

SAPIENZA
UNIVERSITÀ DI ROMA

Facoltà di Ingegneria dell'Informazione,
Informatica e Statistica

Corso di Laurea Magistrale in
Ingegneria Gestionale

Prova Finale di Laurea Magistrale

LONG TERM MULTIVARIATE ENERGY TIME SERIES FORECATING WITH STATE SPACE MODELS

Candidato: Tacconi Francesco

Docenti di riferimento:
Prof. Massimo Panella, Prof. Antonello Rosato

Relatore: Alessio Verdone

Anno Accademico 2025 – 2026

LONG TERM MULTIVARIATE ENERGY TIME SERIES FORECATING WITH STATE SPACE MODELS

Abstract

Time series forecasting remains a cornerstone of modern industrial planning, particularly within the energy sector, where accurate anticipation of future system behavior is essential for grid stability, safety, and efficient resource allocation. As contemporary energy infrastructures grow in scale and complexity, forecasting models are increasingly required to operate over long horizons while handling high-dimensional and non-linear data. Traditional statistical approaches, such as autoregressive and exponential smoothing models, have historically provided a reliable foundation for forecasting tasks. However, their reliance on linear assumptions and rigid parametric structures limits their effectiveness when applied to modern energy datasets characterized by complex temporal dynamics and strong inter-variable interactions.

The emergence of deep learning has fundamentally reshaped the field of time series forecasting by enabling data-driven representation learning directly from raw observations. In this context, a diverse set of neural architectures has been proposed to address long-term forecasting challenges, including recurrent models, attention-based mechanisms, and, more recently, state space formulations. Despite rapid progress, a unified understanding of how architectural bias and data representation interact to capture temporal dependencies and inter-variable correlations remains incomplete.

This thesis presents a comprehensive comparative study of three modern forecasting backbones: Gated Recurrent Units, vanilla Transformer models, and the selective State Space Model known as Mamba. These architectures are evaluated across nine experimental configurations derived from three distinct tokenization paradigms: time-point–based representations, patch-based representations, and variate-centric representations. Experiments are conducted on two benchmark energy datasets focusing on transformer temperature monitoring and large-scale electricity consumption, each exhibiting distinct challenges related to long-term dynamics, non-stationarity, and dimensionality.

In addition to architectural comparison, this work analyzes the transition from autoregressive sequence-to-sequence decoding toward direct generative forecasting strategies based on linear projections. To address the scalability limitations of such decoders in high-dimensional settings, an attention-based pooling mechanism is introduced to mitigate parameter growth while preserving temporal context. The results provide insights into the structural biases required to effectively model temporal dependencies and inter-variable relationships, offering practical guidance for the design of efficient and accurate forecasting systems in industrial energy applications.

Table of Contents

1 Introduction 1

1.1 motivation and context 1

1.2 From Statistical Rigor to Deep Learning Flexibility 2

1.3 Temporal and Variate Correlation in Multivariate Time Series 2

1.4 Limitations of Classical Statistical Models 3

1.5 Deep Learning as a Paradigm Shift 3

1.6 Contributions of This Thesis 4

1.7 Thesis Organization 4

2 Foundations of Time Series Forecasting 5

2.1 Time Series Fundamentals 5

2.2 The Forecasting Task 8

2.3 Classical Statistical Forecasting Models 9

2.4 Limitations and Transition Toward Learning-Based Methods 10

3 Deep Learning Foundations for Sequential Data 11

3.1 Neural Networks and Optimization for Sequential Regression 11

3.2 Core Deep Learning Building Blocks 17

3.3 Recurrent Neural Networks (RNNs) 22

3.4 Summary: TD and VC in Deep Sequential Modeling 26

4 Sequence-to-Sequence and Attention-Based Models 28

4.1 Sequence-to-Sequence Models for Forecasting 28

4.2 Transformer Foundations for Forecasting 42

4.3 Tokenization Strategies for Time Series 52

5 State Space Models and Linear-Time Modeling 60

5.1 The Mathematical Foundation of State Space Models 61

5.2 Structured State Space Models: From LTI Recurrence to S4 65
5.3 Mamba: Selective State Space Models (S6) 69
6 Experimental Setup and Methodology 77
6.1 Forecasting formulation 78
6.2 Tokenization Strategies Under Evaluation 80
6.3 Detailed Model Configurations 84
6.4 Datasets 92
6.5 Data Preprocessing 96
6.6 Training Configuration 98
7 Experimental Results and Analysis 99
7.1 Overview of Evaluation Protocol 99
7.2 Accuracy Analysis 100
7.3 Computational Efficiency Analysis 105
7.4 Model Size, Memory and Trade-off Analysis 111
7.5 Qualitative Forecast Analysis 116
7.6 Summary of Experimental Findings 118
8 Conclusions and Future Work 120
8.1 Summary of the Work 120
8.2 Main Findings 121
8.3 Limitations 123
8.4 Future Work 124
8.5 Final Remarks 126
9 Bibliography 126

1 Introduction

Time series data constitutes the primary record of historical evolution across a wide range of scientific, industrial, and societal systems. Forecasting, defined as the task of leveraging past observations to infer future behavior, plays a critical role in enabling informed decision-making in domains such as finance, healthcare, transportation, climate science, and energy management. As data acquisition technologies continue to advance, the scale, granularity, and complexity of time series datasets have increased substantially, placing growing demands on forecasting methodologies.

In the energy sector, forecasting accuracy is not merely a matter of analytical performance but a prerequisite for operational safety and economic efficiency. Modern power grids must continuously balance supply and demand under conditions of increasing volatility, driven by renewable energy integration, evolving consumption patterns, and aging infrastructure. Within this context, the ability to produce reliable long-term forecasts has become a central requirement for maintaining grid stability and preventing costly failures.

1.1 motivation and context

The motivation of this thesis is rooted in two concrete industrial forecasting challenges: the monitoring of power transformer health and the management of large-scale electricity consumption. Power transformers are critical assets within electrical grids, and their operational lifespan is strongly influenced by internal temperature dynamics. Accurate forecasting of transformer oil temperature enables early detection of hazardous operating conditions and supports preventive maintenance strategies. Similarly, electricity consumption datasets, which track the demand profiles of hundreds of consumers over extended periods, require models capable of capturing both individual usage patterns and collective system behavior.

These applications are characterized by strong temporal regularities, such as daily and weekly cycles, as well as long-term trends influenced by environmental and socio-

economic factors. At the same time, they exhibit complex interactions across multiple measured variables, reflecting the interconnected nature of energy systems. Forecasting in such settings demands models that are not only precise in the short term but robust over extended prediction horizons, where small inaccuracies can accumulate and lead to significant operational risk.

1.2 From Statistical Rigor to Deep Learning Flexibility

For several decades, time series forecasting was dominated by statistical methodologies grounded in linear modeling and probabilistic assumptions. Approaches such as autoregressive integrated moving average models and exponential smoothing techniques provided interpretable and mathematically well-founded solutions, particularly for low-dimensional and stationary data. These models remain valuable baselines and are still widely used in practice.

However, the assumptions underlying classical statistical models often fail to hold in modern industrial datasets. Energy time series frequently exhibit non-linear dynamics, evolving statistical properties, and strong dependencies across multiple variables. Under these conditions, manually specifying model structure and dependencies becomes increasingly impractical.

Deep learning has introduced a paradigm shift by enabling forecasting models to learn representations directly from data. Early neural approaches, such as recurrent neural networks and their gated variants, demonstrated the ability to capture temporal patterns without explicit feature engineering. More recently, attention-based models and state space formulations have further expanded the capacity to model long-range dependencies and complex system dynamics. This transition from rigid parametric models to flexible representation learning forms the methodological foundation of this thesis.

1.3 Temporal and Variate Correlation in Multivariate Time Series

A defining characteristic of multivariate time series is the coexistence of two distinct forms of dependency. The first is temporal dependency, which describes how past observations of a single variable influence its future evolution. This dependency underlies phenomena such as trends, seasonality, and delayed responses. The second is variate

correlation, which captures the relationships between different variables, either at the same time point or across time. In energy systems, such correlations arise naturally from shared physical constraints and operational coupling.

These two dependency structures represent orthogonal axes of information. Accurately modeling long-term temporal dynamics does not necessarily imply effective modeling of inter-variable relationships, and vice versa. Many forecasting models implicitly prioritize one axis over the other through their architectural design or data representation choices. A central premise of this thesis is that understanding and controlling this balance is crucial for effective long-term forecasting in high-dimensional settings.

At this stage, the discussion remains purely conceptual. No specific architectural solutions are assumed. Instead, the goal is to establish a framework that distinguishes temporal dependency and variate correlation as separate, yet complementary, modeling objectives.

1.4 Limitations of Classical Statistical Models

Classical statistical forecasting models are primarily designed to capture temporal dependency within individual series or within small sets of tightly coupled variables. While multivariate extensions exist, they typically rely on fixed linear relationships and become difficult to scale as the number of variables increases. Moreover, these models often struggle with non-linear dynamics and distributional shifts, both of which are common in real-world energy data.

As a result, statistical approaches tend to exhibit limited robustness when applied to long-term multivariate forecasting tasks. Their inability to adaptively learn complex dependency structures motivates the exploration of alternative modeling paradigms capable of jointly addressing temporal dynamics and inter-variable relationships.

1.5 Deep Learning as a Paradigm Shift

Deep learning models approach forecasting as a representation learning problem. Rather than explicitly defining how dependencies should be modeled, neural networks learn internal representations that encode relevant patterns through optimization. This

flexibility allows deep models to capture non-linear temporal behavior and complex interactions across variables, provided sufficient data and appropriate inductive biases.

Recurrent architectures emphasize sequential processing and temporal memory, while attention-based mechanisms enable direct interaction between distant observations. More recent state space approaches seek to combine the strengths of both paradigms by offering scalable long-range modeling with input-dependent dynamics. These developments suggest that architectural design plays a critical role in determining which forms of dependency are most effectively captured.

1.6 Contributions of This Thesis

This thesis makes the following contributions:

- It introduces a conceptual framework that treats **temporal dependencies (TD)** and **inter-variate correlation (VC)** as orthogonal modeling dimensions in multivariate time series forecasting.
- It provides a systematic empirical evaluation of how different architectural backbones interact with alternative tokenization strategies, highlighting the implicit biases introduced by data representation.
- It analyzes the transition from autoregressive sequence modeling to direct generative forecasting and proposes an attention-based pooling mechanism to address scalability issues in high-dimensional settings.
- It benchmarks all configurations on real-world energy datasets, offering practical insights into model behavior under varying levels of dimensionality and non-stationarity.

1.7 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 introduces the theoretical foundations of time series forecasting, reviewing the main concepts used to formulate forecasting problems and the limitations of classical statistical approaches. Chapter 3 presents the deep learning foundations required for sequential regression, including neural networks, optimization principles, and the main architectural building

blocks for temporal modeling. Chapter 4 discusses sequence-to-sequence and attention-based models, emphasizing the transition from recurrent encoder-decoder formulations to Transformer-based forecasting architectures. Chapter 5 introduces state space models and the selective mechanisms that motivate modern linear-time sequence modeling. Chapter 6 describes the experimental setup, including the forecasting formulation, tokenization strategies, model configurations, datasets, preprocessing pipeline, and training protocol. Chapter 7 presents the experimental results and analyzes them from the perspectives of predictive accuracy, computational efficiency, memory usage, architectural trade-offs, and qualitative forecast behavior. Finally, Chapter 8 summarizes the main findings of the thesis, discusses its limitations, and outlines directions for future work.

2 Foundations of Time Series Forecasting

This chapter establishes the foundational concepts required for understanding time series forecasting, with particular emphasis on long-term and multivariate settings. The objective is not to introduce specific learning architectures or training strategies, but rather to clarify how forecasting problems are formulated, categorized, and historically addressed. These foundations provide the conceptual grounding necessary for the deep learning models introduced in the subsequent chapter.

Time series forecasting has evolved through decades of methodological development, driven by increasingly complex data and application demands. By reviewing the structure of time series data, the taxonomy of forecasting tasks, and the capabilities and limitations of classical statistical models, this chapter highlights the challenges that motivate modern learning-based approaches.

2.1 Time Series Fundamentals

A time series represents a collection of observations recorded sequentially over time, forming a chronological trace of an underlying process. Such observations may be sampled at regular or irregular intervals and can take numerical, categorical, or symbolic forms. Time series data arises naturally in a wide range of domains, including energy systems, finance, healthcare, environmental monitoring, and industrial control, where system states evolve continuously and past behavior influences future outcomes.

Unlike static data, time series data exhibits inherent temporal structure. Each observation is context-dependent, deriving meaning from its position within the sequence. As a result, time series analysis focuses not only on individual values, but also on the relationships that unfold across time.

From a probabilistic perspective, a time series can be viewed as a realization of a stochastic process, where each observation corresponds to a random variable indexed by time. In practical settings, only a finite segment of this process is observable, and forecasting aims to extrapolate future values based on the statistical regularities present in historical data.

Time series data is often noisy, high-dimensional, and subject to evolving statistical properties. Sampling frequency can significantly influence observed patterns, as measurements collected at different temporal resolutions may exhibit distinct distributions and dependency structures. These characteristics complicate modeling and motivate careful consideration of how temporal information is represented and processed.

2.1.1 Univariate and Multivariate Time Series

Time series forecasting problems can be broadly categorized based on the number of variables involved. In univariate time series, a single variable is observed over time, and forecasting focuses on predicting its future values based solely on its historical behavior. This setting is common in early statistical modeling and remains relevant for applications where a dominant variable captures system dynamics.

Multivariate time series involve the simultaneous observation of multiple interrelated variables. In this case, forecasting must account not only for temporal dependencies within each variable, but also for correlations across variables. Energy systems provide a canonical example: transformer temperatures, electrical loads, environmental conditions, and operational signals often evolve together, influencing one another over time.

The multivariate setting introduces additional complexity, as relationships between variables may be instantaneous, delayed, or context-dependent. Capturing these interactions effectively is a central challenge in modern forecasting.

2.1.2 Temporal Dependency

Temporal dependency refers to the relationship between past and future values of a variable. This dependency underlies fundamental time series characteristics such as trends, cycles, and delayed effects. In many systems, current observations are strongly influenced by recent history, while longer-term dependencies may encode seasonal or structural behavior.

Temporal dependency is essential for forecasting accuracy, particularly over extended horizons. However, its strength and structure can vary across datasets and variables. Some processes exhibit short memory, where recent observations dominate predictive power, while others require modeling long-range dependencies spanning large portions of the historical record.

2.1.3 Variate Correlation

In multivariate time series, variate correlation captures the relationships between different variables. These relationships may reflect shared physical constraints, causal interactions, or common external influences. For example, electricity consumption across different regions may co-vary due to weather conditions or synchronized human activity.

Variate correlation introduces an additional modeling dimension that is distinct from temporal dependency. A forecasting method may excel at modeling temporal structure within individual variables while failing to capture inter-variable relationships, or vice versa. Recognizing this distinction is crucial for understanding the strengths and limitations of different forecasting approaches.

2.1.4 Structural Components of Time Series

Time series data is often described in terms of several structural components:

- **Trend**, representing long-term directional movement in the data.
- **Seasonality**, capturing recurring patterns at fixed temporal intervals.
- **Random or residual variation**, reflecting stochastic fluctuations not explained by trend or seasonality.

These components may coexist and interact in complex ways. While classical methods often attempt to explicitly separate them, modern approaches frequently model them implicitly through learned representations. Regardless of methodology, awareness of these components remains important for interpreting forecasting behavior and performance.

2.2 The Forecasting Task

Time series forecasting refers to the task of predicting future observations based on historical data. The goal is to infer how a system is likely to evolve beyond a given reference point, using only information available up to that time. Forecasting tasks vary widely in scope and complexity, depending on application requirements and data characteristics.

Forecasting problems are commonly categorized along two primary dimensions: the length of the prediction horizon and the structure of the input variables. These distinctions define the technical challenges a model must address and influence the suitability of different methodological approaches.

2.2.1 Single-Step and Multi-Step Forecasting

Single-step forecasting focuses on predicting the immediate next observation in a sequence. This setting is particularly relevant for applications requiring high temporal resolution and rapid response, such as real-time monitoring or short-term control systems. By limiting predictions to the near future, single-step forecasting avoids many of the instability issues associated with longer horizons.

Multi-step forecasting, by contrast, aims to predict a sequence of future observations over an extended horizon. This setting is essential for strategic planning tasks, including energy load management and infrastructure maintenance. Multi-step forecasting introduces additional challenges, as prediction errors may accumulate and temporal dependencies may weaken over longer horizons.

Several strategies exist for implementing multi-step forecasting, including direct prediction, recursive prediction, hybrid approaches, and multi-output prediction. Each strategy balances computational complexity, stability, and the ability to capture

dependencies across future time steps. The choice of strategy has a significant impact on long-term forecasting performance, particularly in high-dimensional settings.

2.2.2 Target-Only and Covariate-Enriched Forecasting

Another important distinction in forecasting concerns the information made available to the predictive model. In the target-only setting, the forecast is generated exclusively from the historical values of the variable of interest. This formulation is common in univariate forecasting and in settings where the past trajectory of the target already contains most of the relevant predictive signal. Its main advantages are conceptual simplicity, reduced data requirements, and a lower modeling burden.

In covariate-enriched forecasting, the target history is complemented by additional variables that may influence its future evolution. These covariates can include environmental measurements, operational indicators, control signals, or other related series observed in the same system. Incorporating such information may substantially improve predictive performance, especially in long-horizon settings where external drivers shape the overall trend and where the target alone provides only a partial description of the underlying dynamics.

Although covariate-enriched forecasting is generally more expressive, it also introduces additional challenges related to data quality, variable selection, synchronization across sources, and increased model complexity. The relative usefulness of target-only and covariate-enriched formulations therefore depends on the application domain and on the extent to which the forecasted variable is driven by external factors.

2.3 Classical Statistical Forecasting Models

Early forecasting methodologies were rooted in statistical modeling, with an emphasis on linear relationships and probabilistic assumptions. Techniques such as moving averages and exponential smoothing provided simple yet effective tools for trend estimation and noise reduction. These methods assign greater importance to recent observations, reflecting the intuition that recent history is often most informative.

Autoregressive and moving average models formalized temporal dependency by expressing current values as linear combinations of past observations and error terms.

Their combination led to autoregressive moving average models, which became a cornerstone of time series analysis. To address non-stationarity, differencing techniques were introduced, resulting in integrated formulations capable of handling trends and evolving means.

Statistical models are valued for their interpretability and theoretical grounding. However, their reliance on fixed linear structures limits their ability to model non-linear dynamics and complex interactions across variables. As data dimensionality increases, parameter estimation becomes more challenging, and model assumptions are increasingly violated.

2.4 Limitations and Transition Toward Learning-Based Methods

Despite their historical importance, classical statistical models face significant limitations when applied to modern forecasting tasks. Real-world time series often exhibit non-linear behavior, evolving statistical properties, and strong inter-variable dependencies that cannot be adequately captured by fixed linear formulations.

Machine learning methods introduced greater flexibility by learning predictive relationships from data without explicitly specifying model structure. Tree-based methods, such as decision trees and ensemble techniques, demonstrated improved performance in capturing non-linear patterns and feature interactions. However, these approaches often struggle to scale to long sequences and high-dimensional temporal data.

These limitations motivate the adoption of deep learning methods, which treat forecasting as a representation learning problem. By learning hierarchical features directly from sequential data, deep models offer the capacity to jointly capture temporal dependencies and inter-variable relationships. The following chapter builds upon the foundations established here by introducing neural architectures designed explicitly for sequential modeling.

3 Deep Learning Foundations for Sequential Data

The transition from classical statistical forecasting to deep learning is marked by a shift from rigid linear assumptions to high-dimensional, non-linear function approximation. This chapter establishes the mathematical framework of neural networks as regressors and dissects the building blocks used to stabilize training on sequential data.

Notation convention. Throughout this chapter, bold uppercase symbols such as $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{A}$ denote matrices, bold lowercase symbols such as $\mathbf{x}_t$, $\mathbf{h}_t$, and $\mathbf{g}_t$ denote vectors, and non-bold symbols such as t, L, T, V, D, η, and ε denote scalars. In particular, L denotes the lookback length, T the forecasting horizon, V the number of observed variables, and D the latent or hidden representation dimension. This convention is used consistently across feed-forward, recurrent, and optimization formulations.

3.1 Neural Networks and Optimization for Sequential Regression

Time-series forecasting is formally framed as a supervised regression problem. Neural networks generalize classical models by expanding the function class through the composition of non-linear transformations.

3.1.1 Problem Formulation: Structured Inputs and Horizons

Let a multivariate time series be represented as $\mathbf{x}_t \in \mathbb{R}^V$, where t denotes the time index and V is the number of observed variables. Given a lookback window of length L, the historical input segment is written as

$$\mathbf{X}_{t-L+1:t} = [\mathbf{x}_{t-L+1}, \mathbf{x}_{t-L+2}, \dots, \mathbf{x}_t] \in \mathbb{R}^{L\times V}.$$

The forecasting task consists of learning a mapping from this historical window to a future horizon of length T, namely

$$f_{\boldsymbol{\theta}}: \mathbb{R}^{L\times V} \to \mathbb{R}^{T\times V} \quad , \qquad \widehat{\mathbf{Y}}_{t+1:t+T} = f_{\boldsymbol{\theta}}\left(\mathbf{X}_{t-L+1:t}\right),$$

where $\boldsymbol{\theta}$ denotes the trainable model parameters and $\widehat{\mathbf{Y}}_{t+1:t+T} \in \mathbb{R}^{T\times V}$ is the predicted multivariate future trajectory.

This formulation highlights two structural aspects of the forecasting problem. First, the model must extract information along the temporal axis, represented by the lookback

length L. Second, it must account for interactions along the variate axis, represented by the Vobserved variables. These two dimensions correspond to the two complementary dependency structures introduced in Chapter 1: **temporal dependencies (TD)** and **inter-variate correlation (VC)**.

In this perspective, sequential forecasting becomes a structured regression problem in which the design of the architecture determines how information is aggregated across time and across variables.

3.1.2 The Evolution of the Perceptron

The perceptron is the simplest neural model and was originally introduced for binary classification. Given an input vector $\mathbf{x}_i$, the model computes a linear score and applies a sign activation to obtain the predicted label:

$$y_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b),$$

where i denotes the sample index, $\mathbf{x}_i \in \mathbb{R}^P$ is the input vector, $\mathbf{w} \in \mathbb{R}^P$ is the weight vector, $b \in \mathbb{R}$ is the bias term, $d_i \in \{-1,1\}$ is the ground-truth class label, and $y_i \in \{-1,1\}$ is the predicted label. The operator sign $(\cdot)$returns $+1$ if its argument is positive and -1 otherwise. In the classical perceptron learning rule, the parameters are updated only when a sample is misclassified, making the algorithm a simple instance of error-driven online learning.

A single perceptron can represent only linearly separable decision boundaries. As a consequence, it can model simple Boolean functions such as AND and OR, but it cannot represent XOR, which is not linearly separable. This limitation motivates the transition to the Multilayer Perceptron (MLP), where several affine transformations are composed with non-linear activation functions.

Let Ndenote the number of hidden transformations, and let $l \in \{1, \dots, N\}$index the hidden layer. The forward propagation of an MLP can be written as

$$\mathbf{h}^{(l)} = \phi^{(l)} \left(\mathbf{W}^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right), \qquad l \in \{1, \dots, N\},$$

with $\mathbf{h}^{(0)} = \mathbf{x}$, where $\mathbf{W}^{(l)}$ is the weight matrix of layer l, $\mathbf{b}^{(l)}$ is the corresponding bias vector, and $\phi^{(l)}(\cdot)$ is the activation function applied at that layer.

The role of the activation function is crucial. If all hidden layers were purely linear, the composition of the network would collapse into a single affine map, and depth would not provide additional expressive power. For this reason, hidden layers typically employ non-linear activations such as sigmoid, hyperbolic tangent, ReLU, or leaky ReLU. By contrast, the identity activation is commonly used only in the output layer of regression models, where the network must predict unconstrained real-valued quantities.

This distinction is directly related to the universal approximation property. A multilayer perceptron with at least one hidden layer and suitable non-linear activation functions can approximate a broad class of continuous functions on compact domains. In practice, however, the quality of the approximation depends not only on width and depth, but also on optimization, regularization, and data availability.

Activation function	**Formula**	**Typical use**
Sigmoid	$\phi(x) = \frac{1}{(1 + e^{-x})}$	Binary output layers, gates, older hidden layers
Tanh	$\phi(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$	Hidden layers, especially in recurrent models
ReLU	$\phi(x) = \max(0, x)$	Standard hidden-layer activation
Leaky ReLU	$\phi(x) = \max(\alpha x, x),$ $with\ small\ \alpha > 0$	Hidden layers, reduces dead-ReLU effect
Identity	$\phi(x) = x$	Output layer in regression

From a representational perspective, the single-layer perceptron is sufficient for linearly separable Boolean functions such as AND and OR, whereas XOR requires at least one hidden non-linear layer. This classical example illustrates why multilayer architectures are necessary when the target function cannot be represented by a single linear separator.

3.1.3 The Chain Rule: A Dummy Example

To train the network, we must understand how a change in a weight w_1 affects the total loss $\mathcal{L}$. Consider a simplified one-neuron hidden-layer path:

$$x \xrightarrow{w_1} a_1 \xrightarrow{\phi} h_1 \xrightarrow{w_2} a_2 \xrightarrow{\text{Identity}} \hat{y}.$$

The loss for a single prediction is

$$\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2.$$

To update w_1, we use the chain rule to propagate the error backward:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}.$$

Substituting the specific derivatives:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y, \frac{\partial \hat{y}}{\partial a_2} = 1, \frac{\partial a_2}{\partial h_1} = w_2, \frac{\partial h_1}{\partial a_1} = \phi'(a_1), \frac{\partial a_1}{\partial w_1} = x.$$

Therefore,

$$\frac{\partial \mathcal{L}}{\partial w_1} = (\hat{y} - y)\, w_2\, \phi'(a_1)\, x.$$

This product illustrates why vanishing gradients may occur: if $\phi'(a_1)$is small, as in the saturating regions of a sigmoid activation, the entire gradient shrinks, preventing $w_1$from learning effectively.

3.1.4 Optimization Algorithms

Optimization algorithms update the trainable parameter vector $\boldsymbol{\theta}$ so as to minimize the empirical loss function. Let $\mathcal{L}(\boldsymbol{\theta})$ denote the objective defined over the training set, and let $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ be its gradient with respect to the model parameters. In practice, deep neural networks are optimized iteratively, using gradient-based methods that approximate the steepest descent direction from mini-batches of data rather than from the full dataset at every step.

1. Stochastic Gradient Descent (SGD). In standard gradient descent, the gradient of the loss is computed over the entire training set before each parameter update. Although this formulation is conceptually straightforward, it becomes computationally expensive for large datasets and deep models. For this reason, practical training usually relies on mini-batch stochastic gradient descent, where the gradient is approximated using only a subset of samples of size B.

Let $\mathcal{B}_t$ denote the mini-batch used at iteration t, with $|\mathcal{B}_t| = B$. The mini-batch loss can be written as

$$\mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t) = \frac{1}{B} \sum_{(\mathbf{x}_i, \mathbf{y}_i) \in \mathcal{B}_t} \ell\left(f_{\boldsymbol{\theta}_t}(\mathbf{x}_i), \mathbf{y}_i\right),$$

where $f_{\boldsymbol{\theta}_t}$ is the neural network parameterized by $\boldsymbol{\theta}_t$, $\ell(\cdot,\cdot)$ is the sample-wise loss function, $\mathbf{x}_i$ is the input, and $\mathbf{y}_i$ is the corresponding target. Define the mini-batch gradient as

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t).$$

The parameter update is then given by

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\, \mathbf{g}_t,$$

where $\eta > 0$ is the learning rate. This update moves the parameters in the opposite direction of the estimated gradient, thereby seeking a local reduction of the objective function. Compared with full-batch optimization, SGD substantially reduces the computational cost per iteration and introduces gradient noise that often helps exploration of the loss landscape. However, its convergence may be unstable when gradients are noisy, poorly scaled, or highly anisotropic.

2. Adam (Adaptive Moment Estimation). Adam extends SGD by introducing parameter-wise adaptive scaling based on the recent history of both the gradient and its squared magnitude. In particular, Adam maintains an exponential moving average of the first moment of the gradient and an exponential moving average of its second raw moment. This mechanism makes the optimizer more robust in settings characterized by

noisy objectives, heterogeneous gradient magnitudes, and sparse updates, which are common in deep learning for sequential data (Kingma et al., 2014).

Let

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t)$$

denote the gradient of the mini-batch loss at iteration t. Adam computes the first- and second-moment estimates as

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2,$$

where $\mathbf{m}_t$is the first-moment estimate, $\mathbf{v}_t$is the second-moment estimate, and $\beta_1, \beta_2 \in (0,1)$ are decay coefficients controlling the memory of the moving averages. The square in $\mathbf{g}_t^2$ is intended element-wise.

Since both moment estimates are initialized at zero, they are biased toward zero during the initial iterations. Adam therefore applies bias correction, obtaining

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}.$$

The parameter update is then performed as

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon},$$

where $\eta$is the learning rate and $\varepsilon > 0$ is a small constant introduced for numerical stability. The division and square root are applied element-wise. Intuitively, Adam combines a momentum-like mechanism through $\hat{\mathbf{m}}_t$ with adaptive rescaling through $\hat{\mathbf{v}}_t$, so that each parameter receives an update normalized by the recent magnitude of its gradients **(Kingma et al., 2015)**.

In the formulas above, $\boldsymbol{\theta}_t$ denotes the parameter vector at iteration t, $\mathbf{g}_t$ the mini-batch gradient, $\mathbf{m}_t$ and $\mathbf{v}_t$ the moving estimates of the first and second moments, η the learning rate, β_1 and β_2 the exponential decay coefficients, and ε the stabilization constant used to avoid division by zero.

3. Gradient Clipping. In deep networks, especially when training recurrent or long-sequence models, the gradient norm may occasionally become excessively large, leading to unstable parameter updates and numerical divergence. A common safeguard is gradient clipping by norm, where the gradient vector is rescaled whenever its Euclidean norm exceeds a predefined threshold $\tau > 0$. Formally, given the gradient

$$\mathbf{g}_t = \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathcal{B}_t}(\boldsymbol{\theta}_t),$$

the clipped gradient is defined as

$$\tilde{\mathbf{g}}_t = \begin{cases} \mathbf{g}_t, & \text{if } \| \mathbf{g}_t \|_2 \leq \tau, \\ \tau \dfrac{\mathbf{g}_t}{\| \mathbf{g}_t \|_2}, & \text{if } \| \mathbf{g}_t \|_2 > \tau, \end{cases}$$

where $\| \mathbf{g}_t \|_2$ denotes the ℓ_2-norm of the gradient and $\tau$is the clipping threshold. The parameter update is then computed using $\tilde{\mathbf{g}}_t$ in place of $\mathbf{g}_t$. This operation preserves the gradient direction while limiting its magnitude, thereby reducing the risk of exploding updates without altering the optimization rule in ordinary regimes.

3.2 Core Deep Learning Building Blocks

3.2.1 Convolutional Layers

Convolutional layers are linear operators designed to extract local patterns by applying the same learnable kernel across different positions of the input. In the one-dimensional setting relevant to time-series analysis, a convolution acts along the temporal axis and provides an inductive bias toward **locality**, meaning that nearby observations are assumed to be more strongly related than distant ones. This makes convolutions particularly suitable for detecting short-range structures such as local trends, spikes, short periodic fragments, and transient motifs.

For a discrete input sequence $x[t]$ and a kernel $w[\tau]$ of size K, the 1D convolution can be written as

$$s[t] = \sum_{\tau=0}^{K-1} w[\tau]\, x[t-\tau].$$

Here, $s[t]$ denotes the output at time step t, $K$is the kernel size, and the same kernel coefficients are reused across the entire sequence. This parameter sharing distinguishes convolutional layers from fully connected layers and leads to a substantial reduction in the number of trainable parameters. In addition, the local support of the kernel defines a finite **receptive field**, so the representation at each position depends only on a neighborhood of the input rather than on the full sequence.

In practice, the convolutional transformation is usually followed by a pointwise non-linearity such as ReLU, allowing the model to represent non-linear local features. Depending on the architecture, convolution may also be combined with pooling or strided operations in order to reduce resolution and enlarge the effective receptive field. In forecasting applications, however, the main role of 1D convolution is often not global sequence modeling, but rather the extraction of short-term temporal features that can then be processed by more expressive sequential mechanisms. For this reason, convolution is best viewed here as a local feature extractor rather than as the primary mechanism for long-range dependency modeling.

3.2.2 Normalization Strategies: BN vs. LN

Normalization plays a central role in stabilizing the training of deep neural networks by controlling the scale and distribution of intermediate activations. In the current chapter, normalization is introduced as a building block needed to make optimization more stable and to improve gradient propagation in deep sequential models. The two most relevant normalization strategies for our discussion are **Batch Normalization (BN)** and **Layer Normalization (LN)**. The current draft already introduces this distinction, but in a more compressed form.

Batch Normalization normalizes each feature using statistics computed across the examples in a mini-batch (Ioffe et al., 2015). Let $a_i^{(c)}$ denote the activation of feature c for the i-th sample in a mini-batch of size B. The mini-batch mean and variance for feature c are

$$\mu_B^{(c)} = \frac{1}{B}\sum_{i=1}^{B} a_i^{(c)}, \qquad (\sigma_B^{(c)})^2 = \frac{1}{B}\sum_{i=1}^{B}\left(a_i^{(c)} - \mu_B^{(c)}\right)^2.$$

The normalized activation is then

$$\hat{a}_i^{(c)} = \frac{a_i^{(c)} - \mu_B^{(c)}}{\sqrt{(\sigma_B^{(c)})^2 + \varepsilon}},$$

followed by a learned affine transformation

$$y_i^{(c)} = \gamma^{(c)} \hat{a}_i^{(c)} + \beta^{(c)},$$

where $\gamma^{(c)}$ and $\beta^{(c)}$ are learnable scale and shift parameters, and $\varepsilon > 0$ is a small constant introduced for numerical stability. This final affine step is important because it allows the normalization layer to preserve the representational capacity of the original network (Ioffe et al., 2015).

The main intuition behind BN is that normalizing activations reduces the instability caused by changing activation distributions during training. In feed-forward and convolutional networks, this often accelerates convergence, supports larger learning rates, and may even have a mild regularizing effect due to the noise introduced by mini-batch statistics (Ioffe et al., 2015). However, BN also introduces a dependency between examples in the same batch, since the normalized output of one sample depends on the statistics of the entire mini-batch.

Layer Normalization, by contrast, computes normalization statistics across the features of a single sample rather than across the batch dimension (Ba et al., 2016). Let $\mathbf{a}_i \in \mathbb{R}^H$ denote the pre-activation vector of one layer for sample i, where H is the number of hidden units or channels being normalized. The per-sample mean and variance are

$$\mu_i = \frac{1}{H}\sum_{j=1}^{H} a_{i,j}\,, \sigma_i^2 = \frac{1}{H}\sum_{j=1}^{H} (a_{i,j} - \mu_i)^2.$$

The normalized output is then

$$\hat{a}_{i,j} = \frac{a_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}, \qquad y_{i,j} = \gamma_j \hat{a}_{i,j} + \beta_j,$$

where γ_j and β_j are learnable gain and bias parameters associated with feature j. Unlike BN, LN performs the same computation at training and inference time, because it does not rely on batch-level statistics (Ba et al., 2016).

This distinction has important consequences for sequence modeling. Batch Normalization is especially effective in computer vision, where large and stable mini-batches are common and feature statistics are naturally aggregated across examples. In long-sequence forecasting, however, batch sizes are often small because of memory constraints, and the interaction between different sequences inside the same mini-batch may make BN less stable or less natural. Layer Normalization avoids this dependency, making it better suited to recurrent and attention-based architectures operating on variable-length or memory-intensive sequences (Ba et al., 2016). For this reason, LN has become the standard normalization strategy in many modern sequence models, including Transformer-like architectures.

In summary, BN normalizes **across the batch dimension for each feature**, whereas LN normalizes **across the feature dimension for each sample**. The former is highly effective in convolutional vision models, while the latter is generally more appropriate for sequence modeling, especially when training involves small mini-batches or variable-length temporal data.

3.2.3 Residual Connections

Another key building block in modern deep learning is the **residual connection**, introduced in deep residual networks by He et al. (He et al., 2016). Residual learning was proposed to address the optimization difficulties that arise as network depth increases. In particular, simply stacking more layers does not necessarily improve performance, because deeper models may become harder to optimize and may exhibit higher training error than shallower ones.

The central idea is to reformulate a block not as directly learning a target transformation $H(\mathbf{x})$, but rather as learning a residual function $F(\mathbf{x})$ relative to the input. The block output is then written as

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}.$$

Here, $\mathbf{x}$ is the block input, $F(\mathbf{x})$ is the transformation computed by the stacked layers inside the block, and the term $\mathbf{x}$ is added through a shortcut or skip connection. If the input and output dimensions differ, the shortcut may instead use a linear projection:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{W}_s\mathbf{x},$$

where $\mathbf{W}_s$ is a projection matrix used to match dimensions.

This formulation is useful because it is often easier to learn a correction to the input than to learn a completely new transformation from scratch. In the limiting case where the desired mapping is close to the identity, the residual block only needs to learn a small perturbation. Empirically, this improves optimization and facilitates the training of substantially deeper networks (He et al., 2016).

From the viewpoint of gradient flow, residual connections also create a direct path through which information and derivatives can propagate across multiple layers. This does not eliminate all optimization difficulties, but it mitigates the tendency of deep stacks to degrade as depth increases. For this reason, residual additions have become a standard design choice not only in convolutional residual networks, but also in later sequence architectures, where they are typically combined with normalization and feed-forward sublayers.

In the context of this thesis, residual connections are important not because the models under study are classical ResNets, but because the same design principle reappears in modern sequence encoders. Their role is therefore structural: they improve trainability, stabilize deep blocks, and preserve useful information from earlier representations while allowing each block to learn only the incremental refinement required by the task.

3.3 Recurrent Neural Networks (RNNs)

Recurrent Neural Networks represent a class of architectures designed to process sequential data by maintaining an internal "memory." Unlike the feed-forward models discussed in the previous section, which process the input window $\mathbf{X}$ as a static block, RNNs iterate through the sequence, updating a hidden state that summarizes historical observations.

3.3.1 The Recurrence Mechanism

Unlike feed-forward networks, which process the input window $\mathbf{X}_{t-L+1:t}$ as a static block, recurrent neural networks process the sequence one time step at a time. At each step t, the model updates an internal hidden state $\mathbf{h}_t$, which acts as a compressed summary of the information observed so far.

3.3.2 Vanilla RNNs and the Recurrence Relation

In a vanilla recurrent neural network, the hidden state at time t is computed from the current input vector $\mathbf{x}_t \in \mathbb{R}^V$ and the previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^D$. The recurrence is defined as

$$\mathbf{h}_t = \tanh\left(\mathbf{W}_x\mathbf{x}_t + \mathbf{U}_h\mathbf{h}_{t-1} + \mathbf{b}_h\right),$$

where $\mathbf{W}_x \in \mathbb{R}^{D\times V}$ is the input-to-hidden weight matrix, $\mathbf{U}_h \in \mathbb{R}^{D\times D}$ is the hidden-to-hidden recurrent weight matrix, and $\mathbf{b}_h \in \mathbb{R}^D$ is the bias vector. The resulting state $\mathbf{h}_t$ is therefore a D-dimensional latent representation that summarizes the sequence up to time t.

This recurrence introduces a natural inductive bias toward **temporal dependencies (TD)**, since the sequence is processed in chronological order and each hidden state depends explicitly on the previous one. In forecasting applications, this makes vanilla RNNs conceptually well suited to sequential data, because the model can in principle accumulate information over time and update its internal representation as new observations arrive.

However, this same recursive structure also introduces an important limitation. Since the hidden state is repeatedly transformed by the same recurrent dynamics, early information

must survive many successive non-linear updates before it can influence the final prediction. As a consequence, vanilla RNNs often struggle to preserve useful signals over long temporal horizons, especially when the lookback window is large. In practice, the hidden state behaves as a lossy summary of the past, and this makes standard recurrent models fragile when long-range periodicities or delayed effects are important.

As illustrated in Fig.1, the hidden state is propagated recursively across time and acts as the model's sequential memory.

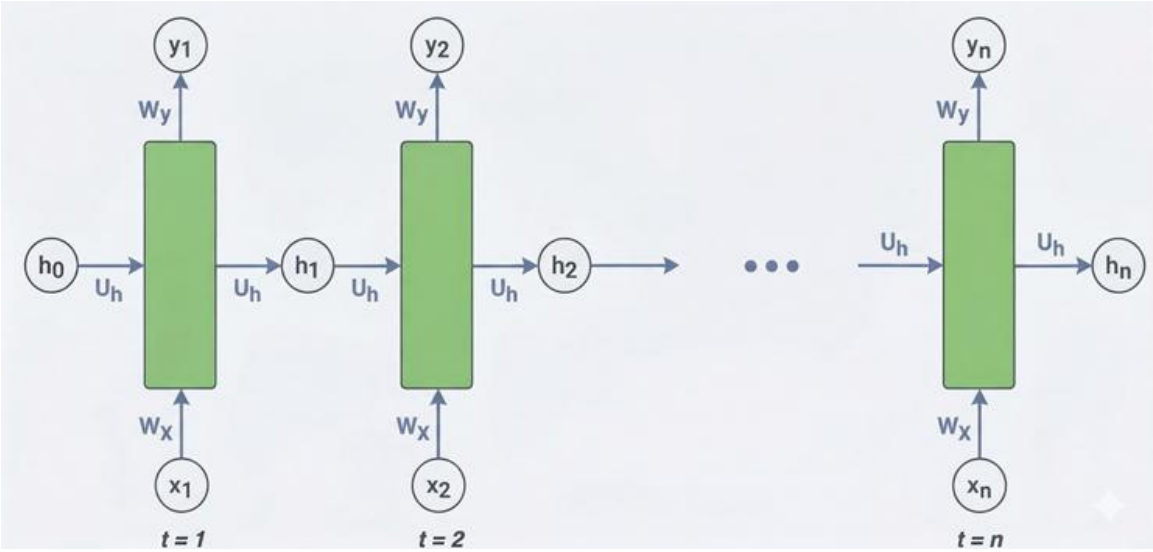


Figure 1: Unrolled representation of a vanilla recurrent neural network (RNN), where the hidden state is propagated sequentially across time and each output depends on both the current input and the accumulated past state.

3.3.3 The Fundamental Flaw: Long-Term Dependency Pathologies

The strength of recurrent neural networks—their ability to process variable-length sequences through a shared recurrent state—is also the source of their main optimization difficulty. During **Backpropagation Through Time (BPTT)**, the gradient of the loss with respect to early recurrent parameters is obtained by repeatedly applying the chain rule across many time steps. As a consequence, the gradient involves a product of Jacobian terms whose norm may either shrink or grow exponentially with sequence length.

This gives rise to the two classical long-term dependency pathologies. In the **vanishing gradient** regime, repeated multiplication by factors with norm smaller than one

progressively suppresses the learning signal, making it extremely difficult for the model to assign credit to events occurring far in the past. In the **exploding gradient** regime, the opposite happens: the norm of the gradient grows rapidly, producing unstable updates and numerical divergence. These effects are especially problematic in forecasting tasks with long lookback windows, where useful predictive information may be distributed over extended temporal horizons.

This limitation has been studied extensively in the literature on recurrent learning and temporal credit assignment, including more recent analyses of BPTT in networks with long-term dependencies (Bird et al., 2021). In practice, two complementary strategies are commonly used to address these issues. The first is **gradient clipping**, which limits the norm of the update and mitigates sudden exploding steps. The second is the introduction of **gated recurrent mechanisms**, such as LSTM and GRU, which create more stable pathways for information and gradient propagation across time.

3.3.4 The Gated Recurrent Unit (GRU)

The Gated Recurrent Unit (GRU) was introduced as a simplified gated recurrent architecture designed to improve the modeling of long-range dependencies while reducing architectural overhead (Chung et al., 2014). More specifically, GRU belongs to the family of gated RNNs developed to mitigate the vanishing-gradient problem that affects standard recurrent networks trained over long sequences. By controlling how information is retained, discarded, and updated across time, gated recurrent architectures provide more stable paths for both memory and gradient propagation.

Given an input vector $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, the GRU computes an update gate $\mathbf{z}_t$, a reset gate $\mathbf{r}_t$, and a candidate hidden state $\tilde{\mathbf{h}}_t$ as

$$\mathbf{z}_t = \sigma\left(\mathbf{W}_z\mathbf{x}_t + \mathbf{U}_z\mathbf{h}_{t-1} + \mathbf{b}_z\right),$$
$$\mathbf{r}_t = \sigma\left(\mathbf{W}_r\mathbf{x}_t + \mathbf{U}_r\mathbf{h}_{t-1} + \mathbf{b}_r\right),$$
$$\tilde{\mathbf{h}}_t = \tanh\left(\mathbf{W}_h\mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h\right),$$

and updates the hidden representation according to

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t.$$

In these expressions, $\sigma(\cdot)$ denotes the sigmoid activation, $\odot$ indicates element-wise multiplication, and $\mathbf{W}_*$, $\mathbf{U}_*$, and $\mathbf{b}_*$ are learnable parameters. The reset gate controls how strongly the previous hidden state contributes to the candidate hidden state, whereas the update gate determines how much of the previous state is preserved in the final hidden state.

From an optimization perspective, the GRU is effective because the hidden-state update preserves a partially linear pathway from $\mathbf{h}_{t-1}$to $\mathbf{h}_t$. This gated interpolation reduces the repeated distortion caused by fully non-linear recurrence and therefore helps mitigate vanishing gradients, making it easier to propagate information and learning signals across longer temporal horizons (Chung et al., 2014).

As shown in Fig.2, the GRU stabilizes recurrent learning by regulating information flow through its reset and update gates.

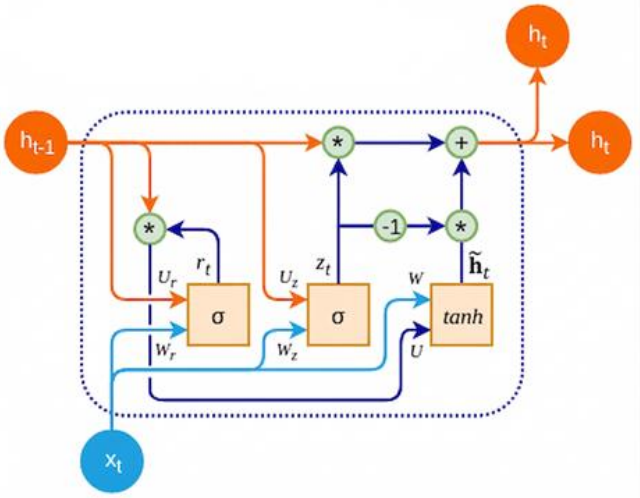


Figure 2: Internal structure of a Gated Recurrent Unit (GRU), illustrating the reset gate, update gate, candidate hidden state, and final hidden-state update used to improve long-range information flow.

3.3.5 Sequence-to-Sequence (Seq2Seq) Paradigm

For forecasting, recurrent units are often arranged in an encoder-decoder structure. The encoder processes the lookback window $\mathbf{X}_{t-L+1:t}$ and compresses it into a fixed-length context vector $\mathbf{c} = \mathbf{h}_L$. The decoder then uses $\mathbf{c}$ to generate the predicted sequence $\widehat{\mathbf{Y}}_{t+1:t+T}$ over a horizon of length T.

In this thesis, we utilize a simplified version in which the encoder, for example GRU-based or Transformer-based, produces a latent representation that is subsequently mapped to the prediction horizon T through a linear head.

3.4 Summary: TD and VC in Deep Sequential Modeling

Before moving to the higher-level architectures discussed in Chapter 4, it is useful to synthesize how the building blocks introduced in this chapter interact with the two main axes of multivariate forecasting: **temporal dependencies (TD)** and **inter-variate correlation (VC)**.

A multivariate time series can be represented as $\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L \times V}$, where L denotes the lookback length and V the number of observed variables. This structure contains two complementary forms of information. Along the temporal axis, the model must preserve and transform TD across successive observations. Along the variate axis, it must capture VC among the observed variables at each time step and, more generally, across the full input history.

The layers introduced in this chapter exhibit different inductive biases with respect to these two objectives. Recurrent layers, such as vanilla RNNs and GRUs, are naturally aligned with TD because they process the sequence recursively and update an internal state over time. Convolutional layers also emphasize TD, but in a local form, since they operate over finite receptive fields and are mainly suited to short-range temporal feature extraction. Fully connected transformations are less structurally constrained: their role depends on how the input is represented, since they mix the features contained in each token without an inherent preference for either axis.

In this sense, the effectiveness of a forecasting architecture depends not only on the expressive power of its layers, but also on how the input representation assigns responsibility for TD and VC. A model may be very effective at preserving temporal information while remaining weak at modeling cross-variate structure, or vice versa. This observation is especially important in multivariate long-term forecasting, where both forms of dependency contribute substantially to predictive performance.

As illustrated in **Fig. 3**, temporal dependency and inter-variate correlation represent two complementary but distinct modeling axes.

The final projection layer, or **linear head**, plays a complementary role. Once the encoder has extracted a latent representation of the historical input, the linear head maps this representation to the output space $\mathbb{R}^{T\times V}$, distributing the learned features across the forecasting horizon T and the V target variables. Although simple in form, this projection is structurally important because it determines how the encoded information is translated into the final multivariate forecast.

This chapter has therefore established the core computational mechanisms required for sequential regression: non-linear function approximation, gradient-based optimization, local feature extraction, normalization, residual refinement, and recurrent temporal processing. The next chapter builds on these foundations by introducing encoder-decoder and attention-based architectures, where the interaction between TD and VC becomes more explicit and more strongly influenced by tokenization design.

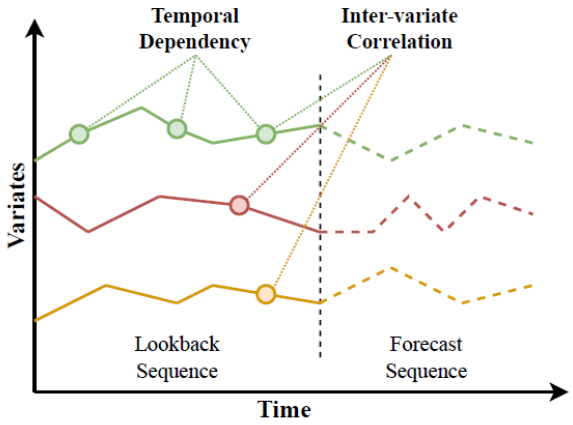


Figure 3: Conceptual illustration of the two complementary dependency axes in multivariate time series forecasting: temporal dependency across the lookback sequence and inter-variate correlation across different variables.

4 Sequence-to-Sequence and Attention-Based Models

The methodology for generating multi-step forecasts in complex energy systems has evolved through a sequence of architectural paradigms. Early approaches relied on recurrent models operating through recursive feedback loops, where predictions were generated step-by-step and fed back into the system. Over time, this paradigm shifted toward direct generative projections, attention-driven mechanisms, and eventually Transformer architectures capable of modeling long-range dependencies through fully parallel interactions.

This chapter describes this transition from a methodological perspective. Building on the recurrent foundations introduced in Chapter 3, we focus here on how forecasting architectures evolved from classical Sequence-to-Sequence (Seq2Seq) models to attention-based mechanisms and, ultimately, Transformer models. The emphasis is on how these structural changes address the challenges of long temporal horizons, multivariate dependencies, and efficient representation of encoder outputs in energy forecasting applications.

4.1 Sequence-to-Sequence Models for Forecasting

4.1.1 Seq2Seq as a Transduction Framework

The Sequence-to-Sequence (Seq2Seq) framework was originally developed for neural machine translation, where a model must transform an input sequence into an output sequence of potentially different length (Cho et al., 2014) (Sutskever et al., 2014)**.** The same abstraction is naturally applicable to multi-step forecasting, where the objective is not to assign a label to a fixed input, but to map an observed historical trajectory into a future one. In this sense, forecasting can be formulated as a sequence transduction problem: the model receives a lookback window of multivariate observations and produces a forecast horizon conditioned on that history.

In the notation adopted throughout this thesis, the historical input window is denoted by $\mathbf{X} \in \mathbb{R}^{L \times V}$, where L is the lookback length and V is the number of observed variables. The

row vector $\mathbf{x}_t \in \mathbb{R}^V$ represents the multivariate observation at time t. Given this input, the forecasting objective is to estimate a future sequence $\hat{\mathbf{Y}} \in \mathbb{R}^{T \times V}$, where T denotes the forecasting horizon. This formulation preserves the temporal ordering of the data while allowing the model to learn a mapping between two sequences with different semantic roles: the first summarizes the observed system evolution, whereas the second represents its expected continuation.

This perspective is particularly appropriate in energy forecasting, where the relation between past and future is rarely reducible to a fixed local rule. Consumption patterns, meteorological drivers, delayed responses, and multi-scale periodicities may all contribute to the forecast, often across different temporal spans. For this reason, the forecasting model must not only encode short-term fluctuations, but also preserve longer temporal dependencies and the interaction among multiple observed variables. Seq2Seq architectures provide an early and influential neural formulation of this requirement by explicitly separating the processing of the historical window from the generation of the forecast trajectory.

4.1.2 Encoder–Decoder Architecture

The core architectural idea behind Seq2Seq models is the separation between an encoding stage and a decoding stage. Rather than attempting to map the historical input window directly to the forecast in a single undifferentiated transformation, the model first builds an internal representation of the observed sequence and then uses that representation to generate the future trajectory. In the original neural machine translation literature, this encoder–decoder decomposition provided a general mechanism for learning conditional mappings between variable-length sequences (Cho et al., 2014) (Sutskever et al., 2014). In forecasting, the same principle is attractive because the model must convert a structured temporal history into a structured temporal continuation, often with a different sequence length and with dependencies distributed unevenly across time.

Let the encoder receive the multivariate input sequence $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_L)$, with $\mathbf{x}_t \in \mathbb{R}^V$. Using a recurrent transition, the encoder updates its hidden representation step by step as

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \boldsymbol{\theta}_{\text{enc}}),$$

where $\mathbf{h}_t \in \mathbb{R}^D$ is the latent state at time t, D is the hidden dimension, and $f(\cdot)$ denotes the recurrent update, typically implemented with an LSTM or GRU. After processing the full lookback window, the encoder produces a sequence of hidden states

$$\mathbf{H} = (\mathbf{h}_1, \dots, \mathbf{h}_L) \ , \mathbf{H} \in \mathbb{R}^{L \times D}.$$

These states form a temporally ordered latent description of the observed history, in which each representation summarizes the input up to its corresponding time step.

In the classical Seq2Seq formulation, the encoder output is then compressed into a context vector $\mathbf{c}$, typically derived from the last hidden state or from a deterministic transformation of the full hidden-state sequence. This vector is intended to summarize the relevant information contained in the input window and to serve as the conditioning signal for the decoder. The decoder, in turn, generates the forecast sequence by modeling a conditional distribution over future outputs:

$$p(\widehat{\mathbf{Y}} \mid \mathbf{X}) = \prod_{\tau=1}^{T} p(\hat{\mathbf{y}}^{\,\tau} \mid \hat{\mathbf{y}}^{\,1}, \dots, \hat{\mathbf{y}}^{\,\tau-1}, \mathbf{c}),$$

where $\hat{\mathbf{y}}^{\,\tau} \in \mathbb{R}^V$ denotes the prediction at forecast step τ. This decomposition makes explicit the autoregressive logic of the original decoder: each new prediction is generated conditionally on the encoded history and on the outputs already produced.

Conceptually, this architecture is important because it preserves the sequential character of the input rather than collapsing it into an unordered feature set. Unlike bag-of-words style representations, which disregard order and therefore treat permutations as equivalent, the encoder–decoder framework remains sensitive to temporal arrangement. The ordering of observations affects the sequence of hidden states, the content of the context representation, and ultimately the forecast itself. This property is essential in time-series forecasting, where identical values arranged in different temporal orders may correspond to very different system dynamics. In this sense, Seq2Seq models inherit from neural machine translation not only the idea of mapping one sequence to another, but also the stronger assumption that order is part of the meaning carried by the data.

At the same time, the encoder–decoder split introduces a useful conceptual asymmetry. The encoder is responsible for extracting temporal structure from the observed window, whereas the decoder is responsible for transforming that latent representation into a forecast trajectory. This division of labor proved highly influential because it made explicit two distinct modeling questions that remain central throughout the rest of this chapter: how the historical sequence should be represented, and how that representation should be converted into multi-step predictions. Later developments in attention and Transformer models can be understood precisely as different answers to these two questions, rather than as a complete abandonment of the Seq2Seq perspective.

4.1.3 Iterated Multi-Step Forecasting (IMS) and Error Accumulation

The original Seq2Seq decoder inherits an autoregressive logic from neural machine translation: the output sequence is generated one step at a time, and each new prediction is conditioned not only on the encoded history, but also on the outputs already produced (Sutskever et al., 2014) (Cho et al., 2014)**.** When this principle is transferred to forecasting, it gives rise to the iterated multi-step forecasting (IMS) strategy. In this setting, the model first predicts the next step, then uses that prediction to support the following one, and continues recursively until the full forecasting horizon is generated. This formulation is natural because it mirrors the causal unfolding of a sequence, but it also introduces a structural fragility that becomes increasingly severe as the horizon grows.

At forecast step τ, the decoder does not rely only on the information extracted from the historical window, but also on the sequence of outputs already generated for steps $1, \dots, \tau - 1$. As a consequence, forecasting errors are not isolated events: once an early step is misestimated, that perturbation may propagate through the remaining trajectory because subsequent predictions are conditioned on an already imperfect generated history.

This is the classical error-accumulation problem of IMS forecasting. During training, the decoder is often optimized under conditions that are more stable than those encountered

at inference time, because the recurrent generation process is guided by the target sequence or by highly informative historical context. During inference, by contrast, the model must rely on its own outputs as it moves through the horizon. The gap between these two regimes makes long-horizon forecasting particularly difficult: small local errors may compound, drift may increase progressively, and the forecast may deteriorate even when the one-step predictor is reasonably accurate. This problem is especially relevant in energy forecasting, where the horizon may span many time steps and the predicted quantities remain mutually constrained by temporal regularities, operating conditions, and exogenous drivers.

The difficulty is not merely numerical; it is also conceptual. In sequence generation tasks such as translation, autoregressive decoding is justified because each output token genuinely conditions the next one in a linguistic chain. In multi-step forecasting, however, the target is often better understood as a structured future segment rather than as a sequence that must necessarily be generated through recursive self-feeding. This distinction is important, because it suggests that the autoregressive decoder is not always the most natural forecasting mechanism, especially when T is large and the objective is to estimate the full future trajectory as coherently as possible. In that sense, the forecasting adaptation of Seq2Seq models inherits both the strengths and the limitations of their NMT origin.

For this reason, IMS should be viewed as an influential starting point rather than as the final solution for long-horizon forecasting. Its importance lies in having established a principled sequence-generation framework for prediction, but its recursive dependence on previously generated outputs creates a source of instability that later architectures explicitly try to remove. This motivates the transition toward attention-based access to the encoded history and, subsequently, toward direct multi-step forecasting strategies in which the horizon is predicted more globally rather than reconstructed through a long autoregressive chain.

4.1.4 Context Vector Bottleneck and the Emergence of Attention

In the classical Seq2Seq formulation, the encoder processes the historical input sequence and summarizes it through a final latent state. In its simplest form, the context vector is taken to coincide with the last encoder state, that is,

$$\mathbf{c} = \mathbf{h}_L,$$

so that the decoder receives a single fixed-length representation of the entire observed window. This formulation is conceptually simple and was one of the key reasons for the early success of encoder–decoder models in sequence transduction (Cho et al., 2014) (Sutskever et al., 2014). At the same time, it reveals a structural limitation immediately: regardless of the length of the input sequence, all relevant historical information must be compressed into one vector of fixed dimension.

This compression becomes problematic when the lookback window is long or when predictive information is distributed unevenly across time. In energy forecasting, the future may depend simultaneously on recent fluctuations, delayed responses, seasonal recurrences, and interactions among multiple measured variables. A single global summary may therefore become too coarse: information from early time steps can be diluted, temporally distant but relevant patterns may be underrepresented, and the decoder is forced to operate through a fixed interface that cannot adapt to the specific needs of each forecast step. For this reason, the limitation of early Seq2Seq models is not only a matter of recurrent memory, but also of representational access. Even if the encoder succeeds in propagating information through the sequence, the decoder still sees the past only through one compressed summary.

Bahdanau et al. addressed this bottleneck by replacing the single static context with a dynamic one that is recomputed for each output step (Bahdanau et al., 2014). Instead of using only $\mathbf{h}_L$, the decoder is allowed to consult the full sequence of encoder states

$$\mathbf{H} = (\mathbf{h}_1, \dots, \mathbf{h}_L), \qquad \mathbf{h}_t \in \mathbb{R}^D, \qquad \mathbf{H} \in \mathbb{R}^{L \times D}.$$

For each forecast step τ, the model constructs a dedicated context vector

$$\mathbf{c}^{\tau} = \sum_{t=1}^{L} \alpha_{\tau,t}\, \mathbf{h}_t, \qquad \mathbf{c}^{\tau} \in \mathbb{R}^D,$$

where the coefficients $\alpha_{\tau,t} \in \mathbb{R}$ are scalar attention weights. These coefficients determine how much attention is assigned to encoder state $\mathbf{h}_t$ when producing the τ-th output. In this way, the model no longer relies on a single immutable representation of the past, but builds a step-specific contextual summary whose content changes as decoding proceeds.

The attention weights are obtained by softmax normalization of raw alignment scores $e_{\tau,t} \in \mathbb{R}$:

$$\alpha_{\tau,t} = \frac{\exp\,(e_{\tau,t})}{\sum_{j=1}^{L} \exp\,(\,e_{\tau,j})} = \mathrm{softmax}(e_{\tau,\cdot})_t$$

Hence, for each fixed forecast step τ, one has $0 < \alpha_{\tau,t} < 1$ and

$$\sum_{t=1}^{L} \alpha_{\tau,t} = 1$$

The score $e_{\tau,t}$ itself is not yet a probability-like quantity; it is only an unconstrained compatibility value that will be normalized by softmax.

The crucial quantity in this mechanism is therefore the score $e_{\tau,t}$, often called the alignment score or energy term. It measures how compatible the encoder state at time t is with the current decoding condition at forecast step τ. In additive attention, this score is computed through a small feed-forward network:

$$e_{\tau,t} = \mathbf{v}^{\top}\tanh(\mathbf{W}_s\mathbf{s}_{\tau-1}+\mathbf{W}_h\mathbf{h}_t+\mathbf{b}),$$

where $\mathbf{s}_{\tau-1} \in \mathbb{R}^D$ is the previous decoder state, $\mathbf{h}_t \in \mathbb{R}^D$ is the encoder state under consideration, $\mathbf{W}_s \in \mathbb{R}^{D_a \times D}$ and $\mathbf{W}_h \in \mathbb{R}^{D_a \times D}$ are learned projection matrices, $\mathbf{b} \in \mathbb{R}^{D_a}$ is a bias term, and $\mathbf{v} \in \mathbb{R}^{D_a}$ maps the hidden compatibility representation to the scalar score $e_{\tau,t} \in \mathbb{R}$. Here, $D_a$denotes the hidden dimension of the additive attention scoring function.

This scoring mechanism is important because it makes the idea of alignment explicit. The decoder does not simply read a stored memory; it learns which parts of the encoded history are most relevant to the output currently being generated. In the original NMT setting, this meant learning soft correspondences between source and target positions. In forecasting, the same principle means that different forecast steps may rely on different temporal regions of the lookback window. Some outputs may depend more strongly on recent observations, while others may draw on more distant temporal patterns. Attention therefore transforms the context from a static compression into an adaptive retrieval process.

This point is also useful for preserving one of the key intuitions of your original chapter. Seq2Seq models are not bag-of-words systems, and attention does not make the sequence unordered. On the contrary, the model remains fundamentally sequence-aware: the encoder states preserve temporal structure, and the attention mechanism learns how to align future outputs with relevant positions in that structured history. What changes is not the importance of order, but the way order is accessed. Instead of forcing the whole sequence into $\mathbf{c} = \mathbf{h}_L$, the model distributes memory across the encoder trajectory and retrieves it selectively according to the decoding step.

From the perspective of this thesis, additive attention should therefore be understood as the first major break with rigid compression-based forecasting. It preserves the recurrent encoder–decoder setting, but it removes the idea that one fixed vector must carry the entire burden of historical representation. Memory becomes distributed across encoder states, context becomes output-dependent, and access to long-range information becomes explicit rather than implicit. This conceptual transition is the real bridge to the next stage of the chapter, where attention ceases to be an auxiliary alignment mechanism and becomes the primary computational principle of the Transformer.

4.1.5 From Autoregressive Decoding to Direct Multi-Step Forecasting

The limitations of iterated multi-step forecasting motivated a gradual rethinking of the role of the decoder in neural forecasting architectures. In the original Seq2Seq formulation, the decoder is a recurrent generator that produces the future one step at a time, mirroring the logic of neural machine translation (Cho et al., 2014) (Sutskever et

al., 2014). This design is coherent when the target sequence must be constructed recursively, but it becomes increasingly problematic in long-horizon forecasting, where the objective is often to estimate an entire future segment rather than to simulate a token-by-token generative process. As a result, the decoder progressively evolved from a full autoregressive sequence model into a much simpler prediction module.

A useful way to interpret this evolution is through three main stages. The first stage corresponds to the classical autoregressive decoder inherited from Seq2Seq learning. Here, the model generates the horizon sequentially, conditioning each step on the encoded history and on previously predicted outputs. This formulation preserves a strict causal chain across the generated sequence, but it also inherits the weaknesses already discussed in the previous subsection: error accumulation, slow inference, and growing instability as the forecasting horizon increases. In forecasting, these limitations are more serious than in translation because the target horizon is not always naturally decomposable into a chain of mutually dependent symbolic outputs.

The second stage emerges when forecasting models begin to relax the recursive decoding requirement and move toward one-shot horizon generation. In this setting, the decoder no longer produces each future step by feeding its own outputs back into itself. Instead, it receives a representation of the observed history and emits the full horizon in parallel. This idea becomes especially clear in architectures such as Informer, whose generative-style decoder is designed to predict a long future window in a single forward pass rather than through a long autoregressive chain (Zhou et al., 2021). The practical importance of this shift lies in two advantages: it reduces inference latency, and it avoids the repeated self-conditioning through which local forecast errors propagate across the horizon. Even when a dedicated decoder block is retained, its role is already substantially different from that of the original Seq2Seq decoder: it becomes less a sequential generator and more a structured transformation from encoded history to future trajectory.

The third stage is the simplification of the decoder into a direct projection head. In many modern long-term forecasting architectures, the decoder is no longer treated as the main modeling component. Instead, most of the representational burden is shifted to the encoder, while the final mapping to the prediction horizon is performed by a

comparatively simple linear head or lightweight output module. This tendency became particularly visible after the critical findings of (Zeng et al., 2023), which showed that increasingly elaborate Transformer-style forecasting pipelines did not necessarily outperform much simpler direct prediction baselines. The implication was not merely that linear models can be strong, but that the forecasting problem may not always require a complex generative decoder once the historical representation has already been learned effectively.

This historical progression should not be interpreted as the complete disappearance of the decoder concept, but rather as a redefinition of its function. In early Seq2Seq models, the decoder was the locus of sequential generation. In intermediate architectures, it became a parallel generator of the future horizon. In many recent models, it is reduced further to an output head that translates latent temporal representations into predictions. This evolution reflects a deeper conceptual shift: long-horizon forecasting is increasingly treated not as recursive sequence generation, but as a direct mapping from an observed temporal context to a future temporal block.

This shift is also consistent with later developments such as PatchTST, where the architecture is fundamentally encoder-centric and the final forecast is obtained through a simple prediction head operating on learned patch representations (Nie et al., 2022). In this class of models, the main question is no longer how to decode step by step, but how to construct informative latent representations of the historical sequence before the final projection stage. The same broad tendency appears in several recent forecasting architectures: architectural sophistication migrates away from the decoder and toward the representation mechanism itself, whether that mechanism is recurrent attention, self-attention, or tokenization design.

For the purposes of this thesis, this decoder evolution is an essential conceptual bridge. It explains why the subsequent Transformer discussion will focus increasingly on encoder representations, self-attention, and tokenization choices rather than on deep autoregressive decoding stacks. At the same time, it clarifies why direct multi-step forecasting became so attractive in energy prediction: by abandoning recursive generation, the model can address the entire horizon more coherently and with lower

exposure to cumulative drift. The remaining challenge, however, is how to connect a rich encoder representation to a large multivariate forecast space without introducing an impractically large number of output parameters. That issue is addressed in the next subsection.

4.1.6 Direct Forecasting and Parameter Explosion

Direct multi-step forecasting removes the recursive dependence of autoregressive decoding, but it immediately raises a new dimensional problem. Once the model decides to predict the entire future horizon in a single forward pass, the encoder representation must be connected directly to a target of size $T \times V$. If the encoder output is

$$\mathbf{H} = (\mathbf{h}_1, \dots, \mathbf{h}_L) \in \mathbb{R}^{L \times D},$$

then the forecasting module must transform this latent representation into

$$\widehat{\mathbf{Y}} \in \mathbb{R}^{T \times V}.$$

In the ECL setting, which is particularly instructive because of its high dimensionality, one may consider $V = 321$ observed variables, $L = 96$ lookback steps, $T = 96$ forecast steps, and latent dimension $D = 512$. These values already make clear that direct forecasting is not only a conceptual simplification of the decoder, but also a non-trivial projection problem.

A naive solution is to flatten the full encoder output and apply one large linear map:

$$\mathbf{h}_{\text{flat}} = \text{vec}(\mathbf{H}) \in \mathbb{R}^{LD},$$

where $\text{vec}(\cdot)$ denotes the flattening of a matrix into a vector by concatenating all its entries. The forecast is then obtained as

$$\text{vec}(\widehat{\mathbf{Y}}) = \mathbf{W}_{\text{out}}\mathbf{h}_{\text{flat}} + \mathbf{b}_{\text{out}},$$

with

$$\mathbf{W}_{\text{out}} \in \mathbb{R}^{TV \times LD}, \qquad \mathbf{b}_{\text{out}} \in \mathbb{R}^{TV}.$$

With the ECL dimensions above, the flattened encoder representation has size

$$LD = 96 \times 512 = 49{,}152,$$

while the output horizon has size

$$TV = 96 \times 321 = 30{,}816.$$

The output matrix therefore contains

$$30{,}816 \times 49{,}152 = 1{,}514{,}323{,}968$$

weights, in addition to 30,816 bias parameters. In other words, the output head alone exceeds **1.5 billion parameters**. This is the parameter-explosion problem in its most direct form: the decoder has been simplified conceptually, but the final projection has become so large that it reintroduces computational burden, memory pressure, and a severe risk of overfitting.

For this reason, direct forecasting architectures usually seek an L-free summary of the encoder states before the final projection. The goal is to derive a compact context vector

$$\mathbf{c} \in \mathbb{R}^D$$

that preserves the most useful historical information while eliminating the explicit dependence of the output head on the full temporal dimension. The simplest possibility is to reuse the last encoder state,

$$\mathbf{c} = \mathbf{h}_L,$$

which is the same compression principle encountered earlier in the classical Seq2Seq formulation. Another simple alternative is average pooling,

$$\mathbf{c} = \frac{1}{L}\sum_{t=1}^{L} \mathbf{h}_t.$$

Both strategies remove the Lfactor from the final projection and therefore solve the dimensional problem at a basic level. However, they remain limited: the last-state solution is rigid, while average pooling is indiscriminate, because it assigns the same importance to all encoded positions. The stronger critique of last-state pooling will

become even more important in the Transformer setting, where the encoder is no longer intrinsically sequential in the recurrent sense.

A more expressive solution is to reuse the additive-attention logic introduced by Bahdanau et al., but with a different functional role (Bahdanau et al., 2014). In the original encoder–decoder setting discussed in the previous subsection, attention is step-dependent: each output step τreceives its own context vector

$$\mathbf{c}^{\tau} = \sum_{t=1}^{L} \alpha_{\tau,t}\, \mathbf{h}_t,$$

because the decoder is autoregressive and must decide, at each step, which encoder states are most relevant for the current prediction. In direct forecasting, this logic changes. Since the horizon is generated in one shot rather than step by step, the model no longer needs a different context for each future position. Instead, it needs a single global summary of the encoded history before the final projection. Additive attention is therefore reused not as a step-wise alignment mechanism, but as a temporal pooling operator:

$$\mathbf{c} = \sum_{t=1}^{L} \alpha_t\, \mathbf{h}_t, \qquad \mathbf{c} \in \mathbb{R}^{D}.$$

The mathematical family is the same, but the role is different: before, attention built $\mathbf{c}^{\tau}$ for each future step; here, it builds one global context vector $\mathbf{c}$for the entire forecast horizon.

The coefficients $\alpha_t \in \mathbb{R}$are scalar attention weights obtained from raw scores $e_t \in \mathbb{R}$ through softmax normalization:

$$\alpha_t = \frac{\exp\,(e_t)}{\sum_{j=1}^{L} \exp\,(e_j)} = \mathrm{softmax}(\mathbf{e})_t, \qquad \mathbf{e} = (e_1, \ldots, e_L) \in \mathbb{R}^{L}.$$

Hence, $0 < \alpha_t < 1$ and $\sum_{t=1}^{L} \alpha_t = 1$.

The raw score $e_t$is not yet a normalized weight; it is only an unconstrained compatibility value. In additive pooling, one may define

$$e_t = \mathbf{v}^\top \tanh(\mathbf{W}_h \mathbf{h}_t + \mathbf{b}),$$

where $\mathbf{h}_t \in \mathbb{R}^D$is the encoder state, $\mathbf{W}_h \in \mathbb{R}^{D_a \times D}$is a learned projection matrix, $\mathbf{b} \in \mathbb{R}^{D_a}$ is a bias term, and $\mathbf{v} \in \mathbb{R}^{D_a}$ maps the hidden scoring representation to the scalar value $e_t \in \mathbb{R}$. Here again, D_a denotes the hidden dimension of the additive attention scoring function. Thus, $\mathbf{v}$ is not a Value vector in the Transformer sense; it is simply the trainable vector that collapses the hidden scoring representation into the scalar needed before softmax normalization.

Once the encoder states have been compressed into $\mathbf{c} \in \mathbb{R}^D$, the final projection becomes

$$\mathrm{vec}(\widehat{\mathbf{Y}}) = \mathbf{W}_c \mathbf{c} + \mathbf{b}_c,$$

with

$$\mathbf{W}_c \in \mathbb{R}^{TV \times D}, \mathbf{b}_c \in \mathbb{R}^{TV}.$$

Under the same ECL example, this means

$$30{,}816 \times 512 = 15{,}777{,}792$$

weights, plus 30,816 bias parameters. The output head is therefore reduced from more than **1.5 billion** parameters to about **15.78 million**, corresponding to a reduction of roughly **99%**. Put differently, the parameter count falls by a factor of $L = 96$. In practical terms, this difference is decisive: it makes the model substantially easier to train, reduces memory usage, and limits the tendency of the output layer to overfit temporal noise instead of learning a robust mapping from latent history to future horizon.

The contrast between flattening and pooled aggregation is therefore more than a technical implementation detail. Flattening keeps the entire temporal grid explicitly exposed to the final linear map, whereas pooling assumes that the encoder has already extracted a structured sequence of latent states and that the remaining task is to summarize them into a compact forecasting context. For the present section, the main conclusion is clear: direct forecasting becomes truly viable only when decoder simplification is accompanied

by a principled strategy for compressing the encoder output before projection. This closes the logic of Section 4.1 and prepares the transition to the Transformer, where attention ceases to be an auxiliary alignment mechanism and becomes the core computational operation of the architecture itself.

4.2 Transformer Foundations for Forecasting

The Transformer marks a decisive shift in sequence modeling. In recurrent encoder–decoder models, attention was introduced as an auxiliary mechanism that helped the decoder retrieve relevant encoder states. In the Transformer, by contrast, attention becomes the main computational principle through which sequence representations are built (Vaswani et al., 2017). This architectural change was introduced originally for neural machine translation, but it also reshaped long-horizon forecasting because it offered a new way to model dependencies across distant positions without relying on recurrent state propagation.

For forecasting, however, this transition also introduces a new difficulty. In language, the token is already a meaningful semantic unit. In time series, by contrast, the token is not given naturally but must be defined by the modeler. A single point in time, a local patch, or even an entire variate history can all be treated as tokens, but these choices do not induce the same structural bias. For this reason, the present section focuses first on the canonical Transformer mechanisms themselves, before the next section examines how those mechanisms are reinterpreted once the tokenization problem becomes central in time-series forecasting.

4.2.1 From Recurrent Attention to Self-Attention

In recurrent Seq2Seq models, attention acts as a step-dependent alignment mechanism. The encoder first produces a sequence of hidden states, and the decoder then uses attention to retrieve the most relevant parts of that sequence when generating each output (Bahdanau et al., 2014). The core computation of the model nevertheless remains recurrent, since information must still propagate through a chain of hidden-state transitions. This creates a structural difficulty for long-context modeling: signals from

distant time steps must survive many successive transformations before influencing the final prediction. As sequence length increases, this long path makes optimization more difficult and weakens the preservation of long-range information, which is one of the reasons recurrent models are more exposed to vanishing-gradient-type problems in long dependencies (Bengio et al. 1994). Attention alleviates this limitation, but does not remove it entirely as long as the main architecture remains recurrent.

The Transformer generalizes this idea by removing recurrence from the representation-building stage (Vaswani et al., 2017). Instead of using attention only as a support mechanism for decoding, it makes attention the central operation through which token representations are constructed. Let the input be a sequence of Ltokens, each embedded in dimension D. The full input can then be written as

$$\mathbf{Z} \in \mathbb{R}^{L\times D}.$$

From this matrix, the model learns three linear projections:

$$\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q, \mathbf{K} = \mathbf{Z}\mathbf{W}_K, \widetilde{\mathbf{V}} = \mathbf{Z}\mathbf{W}_{\widetilde{V}},$$

where $\mathbf{W}_Q \in \mathbb{R}^{D\times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{D\times d_k}$, and $\mathbf{W}_{\widetilde{V}} \in \mathbb{R}^{D\times d_k}$. Here, $\mathbf{Q}$, $\mathbf{K}$, and $\widetilde{\mathbf{V}}$ denote the Query, Key, and Value matrices, respectively. The notation $\widetilde{\mathbf{V}}$ is used instead of $\mathbf{V}$ in order to avoid conflict with the thesis-wide notation for the number of observed variables.

The self-attention output is then defined by the scaled dot-product rule:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \widetilde{\mathbf{V}}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\widetilde{\mathbf{V}}.$$

Since $\mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{L\times L}$, every token is compared with every other token in the sequence. After scaling by $\sqrt{d_k}$, the softmax transforms these pairwise affinities into normalized weights. Each row of the resulting matrix expresses how much relevance a given query assigns to the other tokens in the sequence. The final output is therefore a weighted combination of the Value vectors, so that each token becomes a context-aware representation informed by the whole input rather than only by a recurrent past.

This shift has two major consequences. First, it shortens the effective path through which distant information must travel. In recurrent models, linking two distant positions requires a chain whose length grows with the sequence, and the information carried along that chain may progressively weaken. In self-attention, by contrast, the interaction is established directly through the attention matrix, which makes long-range dependencies easier to access conceptually and easier to optimize in practice (Vaswani et al., 2017). Second, it removes the strictly sequential bottleneck of recurrent computation, because all pairwise token interactions can be computed in parallel once the input embeddings are available. For long-horizon forecasting, these properties make the architecture attractive, since useful predictive signals may come from temporally distant regions of the lookback window.

The gain, however, is not free. Because the attention matrix has size $L \times L$, both time and memory cost grow quadratically in the number of tokens. This is the main structural limitation of vanilla self-attention and one of the main reasons why later forecasting Transformers sought either to reduce the cost of attention itself or to reduce the number of tokens presented to it.

A final caution is already useful here. In language, the Transformer operates on tokens that already behave as semantic units. In time series, a single numerical point rarely carries meaning in the same way. For this reason, although self-attention allows global interaction across the sequence, its effectiveness depends strongly on how the historical signal is tokenized before entering the model. This issue will become central in Section 4.3, where point-wise, patch-wise, and variate-wise tokenizations are compared more explicitly.

4.2.2 Multi-Head Attention (MHA)

A single self-attention map provides only one relational view of the sequence. This is already useful, but it may be too restrictive when different dependency patterns coexist in the data. In multivariate forecasting, the model may need to capture short-range temporal continuity, long-range temporal recurrences, and inter-variate interactions at the same time. Multi-Head Attention (MHA) addresses this limitation by allowing the model to

attend to the same sequence through several learned subspaces in parallel (Vaswani et al., 2017).

Using the standard Transformer notation, for each head $i = 1, \dots, h$, one defines

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \tilde{\mathbf{V}}\mathbf{W}_i^{\tilde{V}}),$$

where $\mathbf{W}_i^Q \in \mathbb{R}^{D \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{D \times d_k}$, and $\mathbf{W}_i^{\tilde{V}} \in \mathbb{R}^{D \times d_v}$ are learned projection matrices for the i-th head. The outputs of the heads are then concatenated and projected back to the model dimension:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \tilde{\mathbf{V}}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}_O,$$

with $\mathbf{W}_O \in \mathbb{R}^{hd_v \times D}$.

The intuition is straightforward. Instead of forcing all dependency structure into one single attention map, the model learns several attention maps at once. Each head can emphasize a different aspect of the sequence. In forecasting, one head may focus on local temporal continuity, another on long-range seasonal correspondences, and another on inter-variate relations. The usefulness of MHA is therefore not merely architectural repetition. It enriches the set of interactions that the model can represent within the same layer.

This also clarifies why MHA partly compensates for the averaging tendency of a single attention map. As noted by Vaswani et al., using only one head can inhibit the model's ability to attend jointly to information from different representation subspaces at different positions (Vaswani et al., 2017). By distributing the computation across several heads, the model can preserve multiple relational views in parallel. In time-series forecasting, where TD and VC may be intertwined rather than separable, this flexibility is particularly useful.

At the same time, MHA does not remove the fundamental structural cost of self-attention. Each head still operates on the token sequence, so the quadratic dependence on token count remains. Increasing the number of heads may enrich representation, but it does not solve the main scalability issue of vanilla attention. This is one of the reasons why later

forecasting Transformers did not rely only on richer attention blocks, but also explored sparse attention, token reduction, and alternative tokenization schemes better suited to long input windows (Zhou et al., 2021; Nie et al., 2023).

4.2.3 Positional Encoding and Temporal Order

A basic limitation of self-attention is that, by itself, it has no intrinsic notion of order. The mechanism compares tokens through pairwise affinities, but it does not know which token comes first, second, or last. For time-series forecasting, this is a critical issue, because temporal order is not a secondary property of the data but part of its meaning. The Transformer therefore requires an explicit mechanism to inject position into the input representation (Vaswani et al., 2017).

If $\mathbf{Z} \in \mathbb{R}^{L\times D}$ denotes the embedded input sequence and $\mathbf{P} \in \mathbb{R}^{L\times D}$ denotes the positional encoding matrix, the actual encoder input is written as

$$\mathbf{Z}^{(0)} = \mathbf{Z} + \mathbf{P}.$$

In this way, each of the L tokens receives not only a content representation, but also a positional mark in the same D-dimensional space. Positional encoding can therefore be understood as a direct injection of temporal order into an otherwise order-agnostic attention mechanism.

In the original Transformer, this positional information is given by fixed sinusoidal functions (Vaswani et al., 2017):

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{D}}}\right), \qquad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/D}}\right),$$

where $pos \in \{1, \dots, L\}$ denotes the token position and $i$indexes the embedding coordinates. These encodings are fixed rather than trainable, and they assign a distinct positional pattern to each token. Their role is simple but essential: they tell the model where each token is located in the sequence before attention begins to mix information across positions.

For the purposes of forecasting, the main point is straightforward. Self-attention gives each token access to the whole sequence, but positional encoding is what restores temporal order within that interaction. Without it, the model would process the input as if it were only a set of tokens rather than an ordered temporal signal. This is particularly important in time series, where even small changes in order may alter the meaning of the observed dynamics.

At the same time, positional encoding should not be confused with a full solution to the representation problem of time-series tokens. It tells the model where a token is, but not whether the token itself is a meaningful unit for forecasting. That broader issue depends on how the signal is tokenized before entering the encoder, and it will become central in Section 4.3.

4.2.4 Transformer Encoder Architecture

The canonical Transformer encoder is built as a stack of M identical layers, each combining self-attention with a position-wise feed-forward transformation (Vaswani et al., 2017). This structure allows the model to refine token representations progressively: self-attention mixes information across tokens, while the feed-forward sublayer transforms each token representation individually in the latent space. Residual connections and layer normalization stabilize this process and help preserve trainability in deep stacks (He et al., 2015; Ba et al., 2016).

Let $\mathbf{Z}^{(\ell)} \in \mathbb{R}^{L \times D}$ denote the input to the ℓ-th encoder layer. The first sublayer is Multi-Head Self-Attention:

$$\mathbf{A}^{(\ell)} = \mathrm{MHA}(\mathbf{Z}^{(\ell)}, \mathbf{Z}^{(\ell)}, \mathbf{Z}^{(\ell)}).$$

Since queries, keys, and values are all derived from the same input sequence, this is a self-attention operation. The output is then combined with the layer input through a residual connection and normalized:

$$\tilde{\mathbf{Z}}^{(\ell)} = \mathrm{LN}\left(\mathbf{Z}^{(\ell)} + \mathbf{A}^{(\ell)}\right).$$

The second sublayer is a position-wise feed-forward network, applied independently to each token representation:

$$\mathrm{FFN}(\mathbf{x}) = \max(0, \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2.$$

Here, $\mathbf{W}_1 \in \mathbb{R}^{D \times D_{ff}}$, $\mathbf{W}_2 \in \mathbb{R}^{D_{ff} \times D}$, $\mathbf{b}_1 \in \mathbb{R}^{D_{ff}}$, and $\mathbf{b}_2 \in \mathbb{R}^{D}$. In the original Transformer, $D = 512$and $D_{ff} = 2048$(Vaswani et al., 2017). Applying again a residual connection followed by layer normalization gives

$$\mathbf{Z}^{(\ell+1)} = \mathrm{LN}\big(\tilde{\mathbf{Z}}^{(\ell)} + \mathrm{FFN}(\tilde{\mathbf{Z}}^{(\ell)})\big).$$

This decomposition clarifies the division of labor inside the encoder. Self-attention is responsible for contextual interaction across tokens, while the FFN enriches the representation of each token after context has been incorporated. Repeating this structure over several layers yields encoder outputs that are no longer simple embeddings of isolated positions, but context-aware representations shaped by the whole input sequence.

Because each encoder layer maps $\mathbb{R}^{L \times D}$ to $\mathbb{R}^{L \times D}$, the outer representation shape is preserved across layers. This makes it straightforward to stack M encoder layers while progressively refining the contextual representation.

Compared with recurrent architectures, this encoder has a different computational profile. A recurrent layer processes the sequence step by step, which preserves a linear dependence on sequence length but also imposes a sequential bottleneck. Self-attention, by contrast, removes that sequential dependence within the layer, allowing all token interactions to be computed in parallel, but it pays for this with quadratic dependence on the number of tokens. This trade-off was summarized explicitly by Vaswani et al. (2017), and it is useful here because it explains both the appeal and the limitation of Transformer encoders for forecasting.

Layer Type	**Complexity**	**Sequential Ops**	**Max Path Length**
Recurrent	$O(L \cdot D^2)$	$O(L)$	$O(L)$
Self-Attention	$O(L^2 \cdot D)$	$O(1)$	$O(1)$

Table 1: Comparison between recurrent layers and self-attention in terms of per-layer computational complexity, sequential operations, and maximum dependency path length, adapted from Vaswani et al. (2017).

As summarized in Table 1, the contrast is clear. Recurrent models such as GRUs scale linearly in the number of sequence steps, but information must pass through a chain whose length also grows linearly with L. Transformer self-attention shortens this path to a constant number of interactions within the layer and removes sequential operations during training, which is one of its main advantages for long-context modeling. At the same time, the quadratic term $O(L^2 \cdot D)$ becomes a serious burden when the input is long. In forecasting, this is one of the main reasons why later work did not simply adopt the vanilla Transformer unchanged, but instead explored more efficient attention mechanisms or alternative tokenization schemes that reduce the effective token count presented to the encoder (Zhou et al., 2021; Nie et al., 2023).

As illustrated in Fig.4, the canonical Transformer combines multi-head self-attention, feed-forward sublayers, residual connections, and positional encoding within an encoder-decoder architecture.

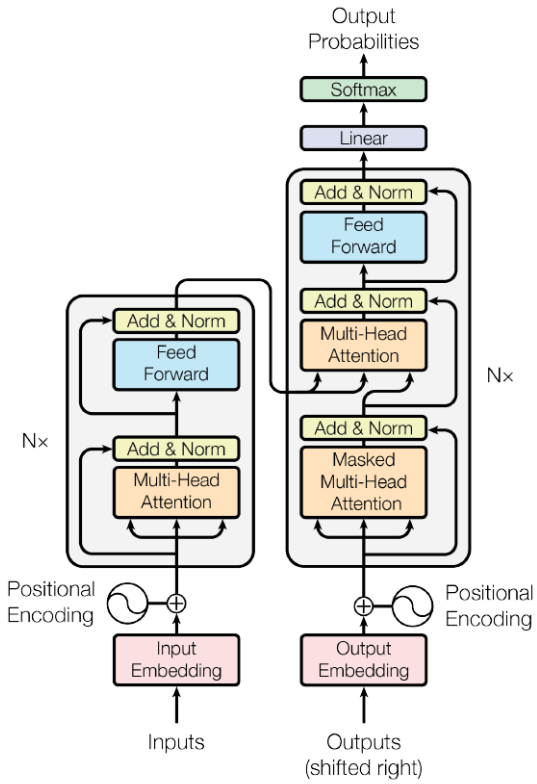


Figure 4: Canonical Transformer architecture, showing the encoder-decoder structure, multi-head attention, position-wise feed-forward layers, residual connections, layer normalization, and positional encoding.

4.2.5 From the Canonical Transformer to Encoder-Centric Forecasting

The original Transformer was introduced as an encoder-decoder architecture for sequence transduction (Vaswani et al., 2017). In time-series forecasting, however, later developments gradually shifted the emphasis away from autoregressive decoding and toward encoder-centric designs. Once the encoder has produced a sequence of context-aware representations, the main problem is no longer how to model dependencies across the input, but how to convert those representations into a forecast efficiently and coherently.

Let the encoder output be

$$\mathbf{H} \in \mathbb{R}^{L \times D},$$

where each row is a context-aware token representation shaped by self-attention across the whole input window. At this stage, the temporal structure of the history has already been processed by the encoder. The remaining issue is therefore how to remove the L-dependence before mapping the representation to the prediction space.

A simple possibility is to retain only the last token representation. In recurrent encoders, this choice has a natural interpretation, because the final hidden state $\mathbf{h}_L$ is the last state of a genuinely sequential process and therefore acts as a compact summary of the preceding history. In Transformer encoders, this interpretation is weaker. Although the last token still corresponds to the last temporal position, its representation is produced through parallel self-attention rather than through a cumulative recurrent state transition. For this reason, the last token is not automatically a privileged global summary in the same sense as in recurrent architectures.

This is why encoder-centric forecasting models often introduce an explicit aggregation step over the full encoder output. In the present chapter, the most coherent solution is to reuse the additive attention already introduced in Section 4.1, not as a decoder-side alignment mechanism, but as a global pooling operator over encoder states. In that case, the encoder output sequence is reduced to a single vector

$$\mathbf{c} = \sum_{t=1}^{L} \alpha_t \, \mathbf{h}_t,$$

where the weights α_t form a normalized importance distribution over the encoded history. Conceptually, the role is different from Bahdanau attention. There, additive attention builds a step-dependent context vector for each output position. Here, the same family of mechanism is reused to construct one global representation for direct forecasting.

This transition is important because it marks a change in architectural logic. In classical Seq2Seq models, the decoder remained the central mechanism that generated the future step by step. In encoder-centric forecasting, by contrast, most of the modeling burden is concentrated in the encoder, while the final prediction stage becomes comparatively simple. Once the historical window has been transformed into a strong contextual representation, forecasting can be treated more directly as a mapping from encoded history to future values.

A further practical issue concerns the size of the encoder output that must be mapped to the forecast. If the encoder returns Ntokens of dimension D, its output has shape $\mathbf{H} \in \mathbb{R}^{N \times D}$. In the standard temporal tokenization, where typically $N = L$, a direct flattening strategy may become expensive because the final prediction head grows with the full token dimension. As discussed in Section 4.1.6, this may lead to a severe increase in the number of output parameters, which is one of the reasons why explicit aggregation becomes useful in encoder-centric forecasting. At the same time, this difficulty is not identical across all forecasting Transformers, because it depends on how the input sequence is tokenized. This point will become clearer in Section 4.3.

This shift also prepares the discussion of the next section. In Transformer-based forecasting, performance depends not only on the attention mechanism itself, but also on what is presented to attention as a token. Once forecasting becomes encoder-centric, the tokenization choice becomes even more important, because it determines both what kind of dependencies the encoder can represent and how expensive that representation becomes computationally.

4.3 Tokenization Strategies for Time Series

4.3.1 The Tokenization Problem in Time-Series Forecasting

In Transformer-based forecasting, the token is the elementary unit processed by self-attention. This point may appear technical, but it is in fact decisive. In natural language, a token already has a relatively clear semantic role, since it corresponds to a word or sub-word unit. In time series, by contrast, the token is not naturally given by the data. It must be defined by the modeling choice itself. A single timestamp, a local subseries, or the full history of one variate can all be treated as tokens, but these alternatives do not induce the same structural bias.

This is especially important in multivariate forecasting, where the input window

$$\mathbf{X} \in \mathbb{R}^{L \times V}$$

contains two distinct forms of information. Along the temporal axis, the model may extract TD, that is, dependencies linking earlier and later observations across the lookback window. Across the variate axis, the model may extract VC, that is, the relationships among the V observed variables. These two structures coexist in the same input, but a given tokenization does not expose them in the same way. Tokenization therefore determines not only what the encoder compares, but also whether attention is oriented more naturally toward TD or toward VC.

For this reason, tokenization should not be understood as a simple preprocessing detail. It determines what kind of unit self-attention receives, what type of relation it can represent most directly, and how the computational cost of the model scales with sequence length. If the input is converted into N tokens of dimension D, vanilla self-attention scales as $O(N^2 D)$. Consequently, changing the tokenization strategy changes not only the meaning of a token, but also the practical burden of attention.

This issue became especially important after the publication of Are Transformers Effective for Time Series Forecasting? (Zeng et al., 2023). That work showed that simple direct linear baselines could outperform several established Transformer forecasters on standard long-term forecasting benchmarks. The importance of this result was not only

empirical, but also conceptual. It suggested that the weakness of early forecasting Transformers might not lie solely in the attention mechanism itself, but also in how time-series inputs were represented before entering the model.

A useful reference point here is the LTSF-Linear baseline introduced by (Zeng et al., 2023). Let the historical input be $\mathbf{X} \in \mathbb{R}^{L \times V}$. For each variate v, the model takes its lookback trajectory $\mathbf{x}_{:,v} \in \mathbb{R}^{L}$ and maps it directly to a forecast $\hat{\mathbf{y}}_{:,v} \in \mathbb{R}^{T}$ through a single linear transformation

$$\hat{\mathbf{y}}_{:,v} = \mathbf{W}\mathbf{x}_{:,v}, \qquad \mathbf{W} \in \mathbb{R}^{T \times L}.$$

Thus, LTSF-Linear does not build a sequence of attention tokens and does not explicitly model VC. It simply learns a direct mapping from the past temporal window to the future horizon for each variate, relying mainly on TD. The significance of this result was therefore not that linear models are universally preferable, but that such a simple temporal projection could outperform several richer time-token Transformers. In this sense, the question became not only whether Transformers are effective, but under what representation of the sequence they become effective again.

This criticism was particularly strong against the standard temporal-token view inherited from earlier sequence models. When each timestamp is treated as one token, self-attention operates over a sequence of isolated temporal points. This naturally favors TD, because attention is applied across time positions. However, time-series data differs from language in an important respect: a single numerical point often has little meaning by itself. Its interpretation depends strongly on neighboring values, local shape, delayed responses, and interactions with other variables. In this sense, preserving temporal order is not enough if the elementary unit itself is too weak.

Later work made this point more explicit. As argued in iTransformer, temporal-token Transformers often fuse multiple variates observed at the same timestamp into one representation, even though those measurements may reflect different physical processes, different scales, or delayed effects (Liu et al., 2023). Under this view, the problem was not necessarily that Transformer components were intrinsically unsuitable for forecasting, but that the inherited time-token formulation was too restrictive for multivariate time

series. The issue, in other words, may have been less the architecture itself than the way it was being used.

The subsequent evolution of Transformer forecasting can therefore be read as a progressive redefinition of the token itself. Early models largely retained the point-wise temporal token, which gives direct access to TD but only indirect access to VC. Later work introduced patch-based representations, in which each token corresponds to a short local subseries rather than to one isolated time point. More recently, inverted formulations proposed variate-wise tokens, in which the full lookback history of one variable becomes the token and attention is redirected more explicitly toward VC. The historical development is therefore not only a sequence of new architectures, but also a sequence of changing assumptions about which elementary unit best exposes the useful structure contained in $\mathbf{X}$.

The rest of this section follows that evolution. Section 4.3.2 discusses point-wise temporal tokens as the original Transformer adaptation to forecasting. Section 4.3.3 then examines patch-wise tokenization, which enriches each token with local temporal structure while reducing token count. Finally, Section 4.3.4 considers variate-wise tokenization, where the axis of attention is effectively inverted and the notion of a token is redefined more radically. Taken together, these developments show that in time-series forecasting the effectiveness of the Transformer depends not only on the architecture itself, but also on how the input window is decomposed into units that allow TD and VC to be represented more appropriately.

4.3.2 Point-Wise Temporal Tokens

The earliest Transformer models for forecasting adopted the most direct tokenization strategy: each timestamp in the input window was treated as one token. If the historical input is $\mathbf{X} \in \mathbb{R}^{L \times V}$, the model reads it as a sequence of L temporal tokens, each containing the V variables observed at that time. These tokens are projected into the latent space, combined with positional information, and then processed by self-attention. The encoder output therefore has shape $\mathbb{R}^{L \times D}$.

This choice was historically natural. The original Transformer was built for token sequences, and the simplest adaptation to forecasting was to treat timestamps as the analogue of words. Under this formulation, self-attention acts directly across temporal positions. In the language of this thesis, point-wise temporal tokenization therefore exposes TD directly.

This is also its main advantage. Because attention is applied along the time axis, the model can relate distant parts of the history without relying on recurrent propagation. In principle, this makes long-range TD easier to access than in recurrent models.

The weakness lies in the token itself. A single timestamp is often a poor and noisy elementary unit. In time series, one isolated value rarely has a clear meaning on its own. Its interpretation depends on the surrounding values: whether it lies inside a rise, near a peak, or within a short fluctuation. As a result, self-attention may compare tokens that are precise as measurements, but weak as units of meaning.

This is easy to see in energy data. A single oil-temperature value may indicate the start of a heating phase, the delayed effect of an earlier load increase, or simply a local fluctuation. Likewise, one isolated demand value does not yet show whether the system is entering a ramp, remaining on a plateau, or descending after a peak. In both cases, the token is exact, but still fragile.

The same issue appears across variables. When all V measurements observed at one timestamp are fused into one token, variables with different physical roles, scales, and delays are mixed immediately. VC is therefore not the main axis of attention. It is first compressed inside each temporal token, and only afterward handled indirectly through temporal interactions. Point-wise tokenization thus favors TD, while VC remains less directly exposed.

The output stage reflects the same geometry. Since the encoder returns a representation in $\mathbb{R}^{L\times D}$, the model must either reduce the token axis first, for example from $\mathbb{R}^{L\times D}$ to $\mathbb{R}^{D}$, and then project $D \to TV$, or flatten the full $L \times D$ structure before prediction. As discussed in Sections 4.1.6 and 4.2.5, the second option may become expensive when L is large.

Point-wise temporal tokens should therefore be understood as the original and most immediate Transformer adaptation to forecasting, but not necessarily as the most suitable one. Their strength is the direct exposure of TD. Their weakness is that the token itself may be too local and too noisy to support strong attention patterns.

4.3.3 Patch-Wise Tokens and Local Temporal Semantics

Patch-wise tokenization is the first major correction of the point-wise setting. Instead of treating each timestamp as one token, the model groups nearby time steps into short local windows, or patches. The intuition is simple: in time series, a short segment is often more meaningful than an isolated point. A patch can already describe a local shape, such as a rise, a drop, a plateau, or a short oscillation.

Operationally, the input $\mathbf{X} \in \mathbb{R}^{L \times V}$ is reorganized into a sequence of N patches for each series, where each patch has length $P$and the stride between consecutive patches is S. If $S < P$, the patches overlap; if $S = P$, they are non-overlapping. In practice, overlapping patches are common because they preserve continuity from one token to the next. A typical simple choice is $S = P/2$, so that consecutive patches overlap by half their length. In PatchTST, the number of tokens is written as

$$N = \left\lfloor \frac{L - P}{S} \right\rfloor + 2,$$

after padding the end of the sequence, and in practice this scales approximately as $N \approx L/S$. After projection into the latent space, the encoder output takes the form $\mathbb{R}^{N \times D}$.

Patch tokenization still works mainly along the temporal axis, so it remains oriented primarily toward TD. The difference is that attention is no longer asked to compare isolated timestamps, but short temporal patterns. The temporal view is preserved, but the elementary unit becomes richer.

Again, the intuition is easier to grasp with a simple example. A single oil-temperature value may be ambiguous. A short patch may already show whether temperature is steadily increasing, stabilizing after a rise, or fluctuating irregularly. In the same way, one isolated consumption value is weak, whereas a short patch may already describe the start

of a ramp or the flattening of a plateau. A patch is therefore easier to interpret than a single timestamp.

This is also why patching can be seen as a denoising move. Point-wise tokens are highly sensitive to spikes, jitter, and small measurement irregularities. A patch absorbs a short neighborhood and carries more stable local structure. In the older language of your draft, patching reduces token noise: instead of asking attention to work on isolated and fragile measurements, it provides tokens that already contain a small temporal context.

The output side also becomes lighter. Since the encoder now returns a representation of shape $\mathbb{R}^{N\times D}$, the model faces the same structural issue as before, but on a shorter token axis. It may still aggregate the token dimension before prediction, or flatten the full $N \times D$structure. The important point is that $N$is now much smaller than L. The scaling problem therefore does not disappear, but it becomes less severe.

Patch-wise tokenization should thus be read as a refinement of the temporal-token view rather than a rejection of it. It still targets TD as the main source of structure, but it does so with richer local units, lower sensitivity to point-wise noise, and a lighter encoder-output geometry.

4.3.4 Variate-Wise Tokens and the Inversion of Attention

The next step in the evolution of tokenization is more radical. Instead of improving the temporal token, variate-wise tokenization changes the axis along which the Transformer is applied. Rather than treating each timestamp as one token, the model treats each observed variable as one token. The input window $\mathbf{X} \in \mathbb{R}^{L\times V}$ is therefore no longer read mainly as a sequence of Ltemporal positions, but as a collection of Vvariate histories.

This is the central idea behind iTransformer. Each variable contributes its full lookback trajectory as the raw material of one token, and that token is projected into the latent space before entering the encoder. The encoder representation therefore has shape $\mathbb{R}^{V\times D}$. Self-attention is no longer applied primarily across temporal positions, but across variables. In the language of this thesis, the main axis directly exposed to attention is no longer TD, but VC.

This inversion addresses one of the main criticisms of temporal-token Transformers. In the point-wise setting, measurements collected at the same timestamp are fused into one token, even though they may correspond to different physical processes and evolve with different delays. In the variate-wise setting, this fusion no longer happens at the token-definition stage. The recent history of oil temperature forms one token, the recent history of load current another, and the recent history of ambient temperature another. The elementary unit becomes more coherent, because each token corresponds to one variable and preserves its own temporal evolution.

A simple dummy example makes the contrast clear. Suppose the model must forecast transformer behavior using oil temperature, load, and ambient temperature. In a temporal-token formulation, these three values are mixed immediately at each timestamp, and attention then acts across time. In a variate-wise formulation, the model can compare the recent history of oil temperature with the recent history of load and with the recent history of ambient temperature. Attention is therefore encouraged to learn which variables move together, which react with delay, and which relations are more informative for forecasting. VC becomes directly visible to the encoder.

TD does not disappear, but it changes role. Each variate token is built from its full lookback history, so temporal structure is still present. The difference is that TD is no longer the main axis of token-to-token interaction. Self-attention is redirected toward VC, while temporal structure is absorbed inside the representation of each variable.

This reformulation also has two practical consequences. First, positional encoding is no longer required in the same way as in temporal-token Transformers. Since the tokens are variates rather than ordered time positions, the model is not required to preserve an order across tokens comparable to temporal order. What matters here is not whether one variate comes before or after another, but how variables relate to one another.

Second, the output stage is structurally simplified. In temporal-token encoders, one must usually reduce a long token sequence before prediction. In the variate-wise setting, the encoder output already has shape $\mathbb{R}^{V \times D}$, so each variate representation can be projected directly from $D$to the forecasting horizon T. In this sense, the output-scaling problem is

not removed by a later pooling step, but bypassed by redefining the token itself. This is also consistent with the more operational formulation adopted later in Chapter 6.

Variate-wise tokenization should therefore be understood as more than a technical variant. It is a genuine reinterpretation of what the Transformer should do in multivariate forecasting. Point-wise and patch-wise tokenization preserve the temporal axis as the main axis of attention, even if patching improves the quality of the token. Variate-wise tokenization, by contrast, redirects self-attention from TD toward VC.

4.3.5 Synthesis: Tokenization as a Structural Bias

The progression from point-wise to patch-wise and variate-wise tokenization shows that, in time-series forecasting, tokenization is not a secondary preprocessing detail but a structural modeling choice. Starting from the same input window $\mathbf{X} \in \mathbb{R}^{L \times V}$, different tokenizations expose different kinds of information to self-attention.

As illustrated in **Fig. 5**, these tokenization strategies define different elementary units for the encoder and therefore induce different structural biases.

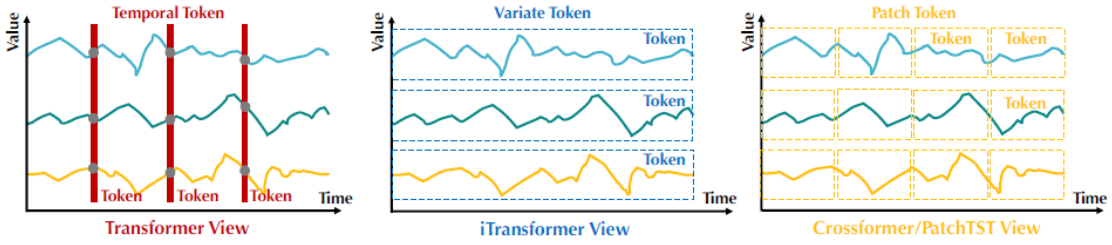


Figure 5: Comparison of tokenization strategies for multivariate time series, illustrating temporal, patch-wise, and variate-wise token representations as used by representative Transformer-based forecasting models.

With point-wise temporal tokens, the encoder output has shape $\mathbb{R}^{L \times D}$. This formulation gives direct access to TD, but it relies on isolated timestamps as tokens, which may be weak and noisy elementary units. VC remains compressed inside each temporal token rather than directly exposed to attention.

With patch-wise tokenization, the encoder output becomes $\mathbb{R}^{N \times D}$, where N depends on patch length and stride and is typically much smaller than L. This setting still favors TD,

but it does so through richer local units. The token becomes more informative, less sensitive to point-wise noise, and easier to handle at the output stage.

With variate-wise tokenization, the encoder output becomes $\mathbb{R}^{V \times D}$. Here the role of attention changes more substantially: the main axis directly exposed to self-attention is no longer temporal but variate-wise. VC becomes the primary relational structure modeled across tokens, while TD is absorbed inside the representation of each variable.

Tokenization also changes the geometry of prediction. In the point-wise setting, one often needs to reduce or flatten a long temporal token axis before projection. In the patch-wise setting, the same issue remains, but on a shorter token axis. In the variate-wise setting, by contrast, each token can be projected directly from D to T, without first collapsing a long temporal sequence of tokens. This is one of the clearest ways in which tokenization affects not only representation, but also the structure of the final forecasting head.

Seen in this way, the recent evolution of Transformer forecasting can be read as a gradual search for more suitable tokens. The main question was not simply whether self-attention should be used for time series, but what kind of elementary unit allows it to model TD and VC in a way that is both meaningful and effective.

5 State Space Models and Linear-Time Modeling

State space models offer a different answer to the long-sequence problem from the recurrent and attention-based architectures discussed in the previous chapters. In recurrent models, long contexts are difficult because information must propagate through many sequential transitions. In Transformer models, long-range interactions become easier to access, but this advantage is paid for with the quadratic cost of self-attention in the number of processed tokens, which becomes increasingly burdensome as the context grows. For this reason, recent work has looked for alternative sequence backbones that can preserve long-range modeling ability without inheriting the full computational cost of attention.

Within this search, structured state space models have emerged as a particularly important direction. Their appeal lies in a simple but powerful idea: instead of modeling a

sequence only through recurrent hidden updates or pairwise token interactions, the model is built around the evolution of an underlying latent dynamical system. This viewpoint comes from classical state space modeling in control and signal processing, but in recent years it has been reformulated as a trainable deep-learning sequence layer. The result is a family of architectures that can be interpreted through three complementary views: as continuous dynamical systems, as discrete recurrences, and, in the linear time-invariant setting, as convolutional sequence operators (Gu et al., 2021; Gu et al., 2022).

This chapter introduces that progression in a deliberately simple way. It first presents the continuous and discrete intuition behind state space models, then explains why the Structured State Space Model (S4) became important for long-sequence modeling, and finally discusses the move toward selective state spaces in Mamba. The emphasis is not on reproducing the full algebra of the original papers, but on clarifying the architectural logic that makes these models relevant for forecasting: long-range memory, linear-time sequence processing, and the transition from fixed linear dynamics to input-dependent filtering and forgetting.

5.1 The Mathematical Foundation of State Space Models

Modern SSMs in deep learning treat sequence modeling as a process of mapping a 1D input signal *x(t)* to an output *y(t)* through an *N*-dimensional latent state *h(t)*.

5.1.1 Continuous-Time State Space Intuition

A classical continuous-time state space model describes how an input signal drives an underlying latent dynamical system and how that latent system produces an observable output. In its simplest form, the model maps a one-dimensional input signal $x(t) \in \mathbb{R}$ to a one-dimensional output signal $y(t) \in \mathbb{R}$ through a hidden state $\mathbf{h}(t) \in \mathbb{R}^N$. The standard linear formulation is

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t), \qquad y(t) = \mathbf{C}\mathbf{h}(t) + Dx(t).$$

Here, $\mathbf{A} \in \mathbb{R}^{N\times N}$ is the state matrix and governs the internal dynamics of the system, $\mathbf{B} \in \mathbb{R}^{N\times 1}$ maps the input into the state, $\mathbf{C} \in \mathbb{R}^{1\times N}$ maps the state back to the output, and $D \in \mathbb{R}$ is a direct skip term from input to output (Gu et al., 2021; Gu et al., 2022). In the

remainder of this chapter, following the common simplification used in the S4 literature, the skip term Dwill be omitted for clarity unless explicitly needed.

This formulation is useful because it separates the visible signal from the latent system that carries memory. The input does not directly determine the output at each instant. Instead, it first modifies the hidden state, and the output is then read from that state. In this sense, the hidden state acts as a compact internal memory of the recent and distant past.

This is also why the state space viewpoint is closely related to the logic of classical filtering and control. In a Kalman-filter style interpretation, the system is not understood only through the measurements that are directly observed, but through an internal state that evolves over time and summarizes the condition of the system. The input drives that evolution, the state stores the system's current memory, and the output reads part of that latent information back into the observable domain. The same general intuition is preserved in modern neural state space layers, even though the goal is no longer classical estimation, but sequence modeling.

The point is especially natural in physical and energy-related settings. One may think of $x(t)$ as an external driving signal or observed measurement, and of $y(t)$ as the quantity of interest to be reconstructed or predicted. The latent state then plays the role of an internal memory of the system. Even in this very simple one-dimensional input-output view, the model already expresses an important idea shared by many sequential architectures: the sequence is interpreted through the evolution of a hidden state rather than as an unordered list of values.

For deep sequence modeling, the appeal of this formulation is not simply that it uses a hidden state, since recurrent networks do that as well. The stronger advantage is that, in the state-space setting, the core sequence operator remains linear and can therefore admit multiple equivalent views. Once discretized, the same model can be interpreted as a recurrence and, under linear time invariance, also as a convolutional sequence operator. This duality is one of the key reasons state space models became attractive for long-context modeling.

The next step is therefore to convert this continuous-time system into a discrete one that can operate on sampled sequences. From there, the chapter will show how the same underlying dynamics can be read first as a recurrence, then as a convolutional kernel, and finally, in Mamba, as a selective scan once strict time invariance is relaxed.

5.1.2 From Continuous Dynamics to Discrete Recurrence

To be used in sequence modeling, the continuous-time system must be converted into a discrete one, because the model no longer receives a continuously observed signal $x(t)$, but a sampled sequence $x_1, x_2, \dots, x_L$. This step is called discretization. Conceptually, discretization transforms the continuous dynamics into an update rule that advances the hidden state from one sampled time step to the next.

After discretization, the state space model takes the recurrent form

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \qquad y_t = \mathbf{C}\mathbf{h}_t.$$

Here, $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$ are the discrete counterparts of the continuous matrices $\mathbf{A}$ and $\mathbf{B}$, while $\mathbf{C}$ preserves the readout from the latent state to the output. In this form, the model can be read as a recurrence: the new state depends on the previous state and on the current input, and the output is obtained by projecting the updated state back to the observable domain.

In the present chapter, this transition is described through the zero-order hold (ZOH) discretization. Under this rule, one assumes that the input remains constant within each sampling interval of size Δ. The resulting discrete matrices are

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \qquad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I})\Delta\mathbf{B}.$$

These expressions show an important point already. The actual sequence computation does not use the continuous matrices $\mathbf{A}$ and $\mathbf{B}$ directly, but their discretized versions $\bar{\mathbf{A}}$ and $\bar{\mathbf{B}}$. Since these depend on the step size Δ, changing Δalso changes the effective discrete dynamics of the model. In this sense, the model is introduced through a continuous-time system, but it operates through discrete transitions on sampled data.

This is the first important bridge with deep sequence modeling. Once written in discrete form, the state space model already resembles a recurrent architecture. It receives the sequence one step at a time, maintains a hidden state, and updates that state as new inputs arrive. State space models and recurrent networks therefore share a common intuition: both process the sequence through an evolving memory. The difference is that, here, the recurrence is not introduced first as a generic neural update rule, but as the discretized version of an underlying dynamical system.

This viewpoint is useful because it gives the recurrence a more structured interpretation. The matrix $\bar{\mathbf{A}}$ controls how past state information persists or decays from one step to the next, while $\bar{\mathbf{B}}$ determines how strongly the current input enters the latent state. If $\bar{\mathbf{A}}$ retains information over many steps, the model can preserve longer temporal influence; if it attenuates the state rapidly, the memory becomes shorter. Even at this basic level, one can already see why the choice of the state matrix will later become central in S4.

The same idea is easy to picture with a simple physical intuition. Suppose the observed sequence is a stream of temperature-related measurements. At each discrete time step, the system keeps part of its previous internal condition and combines it with the new input. The hidden state therefore acts as a running summary of how the system has evolved, rather than as a mere storage of the last observation. The role of the recurrence is to propagate that latent memory across time.

In the setting considered by S4, the system is linear time-invariant (LTI). In simple terms, this means that the same matrices are used at every time step: the state transition $\bar{\mathbf{A}}$, the input map $\bar{\mathbf{B}}$, and the readout $\mathbf{C}$ do not change with t. This fixed-dynamics assumption is what makes it possible to rewrite the recurrence as a convolution with one global kernel. Later, this property will be relaxed in Mamba, where some of these quantities become input-dependent and the model is no longer time-invariant.

However, the discrete recurrent view is still only one side of the story. For linear time-invariant systems, the same recurrence can also be unfolded into a convolutional form. This second view is extremely important, because it is what later allows models such as S4 to combine long-range sequence modeling with efficient parallel training. For this

reason, the next subsection turns from the recurrent interpretation to the corresponding kernel view.

5.2 Structured State Space Models: From LTI Recurrence to S4

Once the continuous-time system has been discretized, a state space model can already be read as a recurrence. The distinctive step of the S4 line of work is to exploit the fact that, under linear time invariance, the same model can also be rewritten through a convolutional kernel. This second view is what makes S4 especially important for long-sequence modeling: the model preserves a recurrent interpretation, but it also acquires a parallel training path through kernel-based sequence processing.

The present section follows that logic in a simple way. It first shows how an LTI recurrence can be unfolded into a kernel, then explains why the state matrix $\mathbf{A}$ is the key object of the model, and finally introduces the practical ingredients that make S4 usable: discretization, structured parameterization, and block-level integration into a deep backbone. The goal is not to reproduce the full algebraic machinery of S4, but to make clear why it became a practical alternative to quadratic-cost attention on long sequences.

5.2.1 From LTI Recurrence to Kernel

After discretization, the state space model takes the recurrent form

$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, y_t = \mathbf{C}\mathbf{h}_t.$$

This is already a valid sequence model: it reads the input one step at a time, updates a hidden state, and produces the output from that state. However, this recurrent form is inherently sequential, so training over long sequences is harder to parallelize efficiently.

The key simplification used in S4 is that the system is linear time-invariant. In practical terms, this means that $\bar{\mathbf{A}}$, $\bar{\mathbf{B}}$, and $\mathbf{C}$ are fixed across time. Since the same transition and readout are applied at every step, the recurrence can be unfolded into a convolutional form.

Assume the initial state is zero, $\mathbf{h}_{-1} = 0$. Then the first hidden states are

$$\mathbf{h}_0 = \bar{\mathbf{B}}x_0, \qquad \mathbf{h}_1 = \bar{\mathbf{A}}\bar{\mathbf{B}}x_0 + \bar{\mathbf{B}}x_1, \qquad \mathbf{h}_2 = \bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \bar{\mathbf{B}}x_2.$$

Applying the readout $\mathbf{C}$, the first outputs become

$$y_0 = \mathbf{C}\bar{\mathbf{B}}x_0, \qquad y_1 = \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}x_0 + \mathbf{C}\bar{\mathbf{B}}x_1, \qquad y_2 = \mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}x_0 + \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}x_1 + \mathbf{C}\bar{\mathbf{B}}x_2.$$

A clear pattern now appears. Each output is a weighted sum of present and past inputs, with weights determined by the same fixed state-space dynamics. The whole sequence transformation can therefore be written as

$$\bar{\mathbf{K}} \in \mathbb{R}^L := (\mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}^2\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}}), \qquad \mathbf{y} = \bar{\mathbf{K}} * \mathbf{x}.$$

This is the main conceptual move behind S4. The model is not only a recurrence over a hidden state, but also a learned convolutional sequence operator. The recurrent view remains useful for interpretation and step-wise inference, while the kernel view becomes attractive for training because the convolution can be implemented efficiently through FFT-based methods. This reduces the sequence-length dependence to about $O(L \log L)$, instead of the quadratic $O(L^2)$ behavior of self-attention.

At this stage, however, one important distinction should be kept in mind. Writing the model as a convolution is not yet the whole solution. The real challenge is to obtain the kernel $\mathbf{K}$ efficiently from the state-space parameters. This difficulty is what makes the matrix $\mathbf{A}$ so central, and it is the reason the next subsection focuses on its structure.

5.2.2 Why the State Matrix $\boldsymbol{A}$ Matters

The kernel $\mathbf{K}$ depends on $\bar{\mathbf{A}}$, $\bar{\mathbf{B}}$, and $\mathbf{C}$, but the matrix $\mathbf{A}$ plays the main structural role. Once discretized into $\bar{\mathbf{A}}$, it governs how information propagates through the latent state from one step to the next. For this reason, it largely determines whether the model preserves useful information over long intervals or forgets the past too quickly.

A naive choice of $\mathbf{A}$ is usually not enough. Earlier work showed that simple state-space layers do not automatically solve long-range dependency problems unless the latent dynamics are chosen carefully. The importance of the HiPPO line of work lies precisely here: it provides a principled way to structure the state dynamics so that the hidden state acts as a more effective memory of the past (Gu et al., 2020; Gu et al., 2022; Gu et al., 2022).

At the same time, there is a second issue, and it is computational. Even if one has chosen a good state matrix for long memory, the kernel

$$\overline{\mathbf{K}} \in \mathbb{R}^{L} := (\mathbf{C}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}\bar{\mathbf{B}}, \mathbf{C}\bar{\mathbf{A}}^{2}\bar{\mathbf{B}}, \dots, \mathbf{C}\bar{\mathbf{A}}^{L-1}\bar{\mathbf{B}})$$

still involves repeated powers of $\bar{\mathbf{A}}$. If this is done naively, kernel construction becomes expensive and numerically awkward, especially when the underlying dynamics come from the HiPPO-based matrix. This is the point where S4 makes its crucial contribution.

The modeling issue and the computational issue should therefore be separated clearly. HiPPO explains why the chosen dynamics are good for long memory. S4 then shows how to make those dynamics practical. The key observation is that the relevant HiPPO-based state matrix can be rewritten in a structured form called normal plus low-rank (NPLR), and then handled through a related diagonal-plus-low-rank representation. The purpose of this reformulation is simple: one would like the matrix to behave almost like a diagonalizable system, because diagonal structure is much easier to manipulate when constructing the kernel. In this sense, the structured reformulation does not replace the HiPPO idea, but makes it computationally usable.

For the present chapter, the main point can remain shallow. S4 works well because it combines two ingredients. First, it uses a state matrix with principled long-memory behavior. Second, it introduces a structured parameterization that makes the corresponding kernel practical to compute. Once that kernel is available, the sequence transformation can then be applied efficiently through FFT-based convolution.

This is why the S4 kernel should be treated here as a learned black-box sequence operator, but not as an arbitrary one. Its behavior is shaped by a very specific choice of latent dynamics and by a very specific structured reformulation of the state matrix. The next subsection makes the passage from the continuous system to the discrete parameters explicit through the chosen discretization rule.

5.2.3 From the S4 Kernel to the S4 Block

At the architectural level, the S4 kernel is not used in isolation, but as the core sequence operator inside a repeatable deep-learning block. If the input to the block is a sequence representation in $\mathbb{R}^{L\times D}$, the block returns an output in $\mathbb{R}^{L\times D}$. In this sense, the outer representation shape is preserved: the block refines the sequence representation, but does not change the number of sequence positions or the model dimension.

This point is important because it makes the S4 block comparable to the Transformer-style layers discussed in Chapter 4. The role of the block is to transform an input sequence of L positions, each represented in dimension D, into a new sequence of the same outer shape, but now enriched by the structured state-space operator. One may therefore think of the output as a more context-aware sequence representation, even though the underlying mechanism is not self-attention but an SSM kernel.

Inside the block, the SSM kernel provides the main long-range sequence transformation. Around this core, practical implementations usually include activation, dropout, and a channel-mixing stage so that information is not processed only independently within each hidden feature. In this way, the kernel supplies the sequence operator, while the surrounding layers adapt it into a full neural block.

Because the block maps $\mathbb{R}^{L\times D} \to \mathbb{R}^{L\times D}$, it can be stacked repeatedly through depth. If one uses M such layers, the model progressively refines the same outer representation shape while increasing the expressive power of the sequence backbone. This dimension-preserving structure is what makes deep stacking straightforward.

As in Transformer-style architectures, these repeated sublayers are typically inserted into a residual and normalization framework. The exact placement of normalization may vary between implementations, for example between pre-normalization and post-normalization, but the overall purpose remains the same: to stabilize deep stacking while keeping the block dimension unchanged across layers.

For the present chapter, this architectural reading is enough. S4 should be understood not only as a recurrence and not only as a convolutional kernel, but as a repeatable deep sequence block whose core operator is a structured state-space kernel.

5.3 Mamba: Selective State Space Models (S6)

The previous sections introduced state space models in their structured, linear time-invariant form, culminating in S4 as an efficient sequence operator with strong long-range modeling ability. The next step in this evolution is Mamba, which keeps the state-space foundation but relaxes the strict LTI assumption in order to make the dynamics depend on the current input itself (Gu et al., 2024). This change is important because it gives the model a selective mechanism: instead of processing every time step with exactly the same fixed rule, the model can adaptively decide what information should be written, propagated, attenuated, or forgotten along the sequence. At the same time, this also removes the simple convolutional view used by S4, so the discussion must now move from fixed kernels to selective recurrent scans.

5.3.1 From LTI S4 to selective time-varying dynamics

After S4, an immediate question was whether the same state-space logic could be made more content-aware. In S4, once the continuous system is discretized, the sequence operator remains linear time-invariant, so the same discrete dynamics are applied at every position of the sequence (Gu et al., 2021; Gu et al., 2022). This gives S4 a powerful and efficient long-range operator, but also makes its behavior rigid: the update rule does not change according to the actual content of the current input.

This rigidity is directly tied to the main computational advantage of S4. Because the dynamics are fixed across time, the recurrence can be unfolded into a single global convolution kernel $\bar{\mathbf{K}}$. In other words, the same LTI property that makes S4 elegant also makes it computationally convenient. The model may be interpreted either as a recurrence or as a convolution, and this dual view is central to the S4 construction.

Mamba changes precisely this point. Its main idea is selectivity: the sequence model should be able to react differently to different inputs, so that relevant information can be retained while irrelevant information can be suppressed (Gu et al., 2024). Instead of using one fixed sequence operator for the whole input, Mamba allows part of the SSM dynamics to depend on the current input itself. This gives the model a simple but important ability that pure LTI SSMs lack, namely input-dependent filtering and forgetting.

More precisely, the selective SSM used in Mamba keeps the state matrix $\mathbf{A}$ fixed, while making Δ, $\mathbf{B}$, and $\mathbf{C}$input-dependent (Gu et al., 2024). If the input sequence is $\mathbf{X} \in \mathbb{R}^{L \times D}$, then at each time step $t$the model produces a step size Δ_t, an input-to-state term $\mathbf{B}_t$, and a state-to-output term $\mathbf{C}_t$ from the current input $\mathbf{x}_t$. The state dimension is still N, the outer representation still lives in dimension D, and the latent state still evolves recurrently, but the effective dynamics now vary along the sequence.

At an intuitive level, the roles of these selective terms are straightforward. The step size Δ_t controls how strongly the state is updated at step t, so it acts as an input-dependent control over the memory timescale. The term $\mathbf{B}_t$ controls how the current input is written into the latent state, while $\mathbf{C}_t$controls how the latent state is read back into the output. Since these quantities depend on $\mathbf{x}_t$, the model can treat different parts of the sequence differently. Some inputs may be written strongly into memory, while others may be largely ignored.

The price of this flexibility appears immediately. Once Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ vary with the input, the model is no longer strictly LTI. Therefore, the old convolutional shortcut is no longer valid: there is no single fixed kernel $\bar{\mathbf{K}}$ that can be precomputed for the entire sequence. Mamba thus gains content-aware dynamics, but loses the clean convolutional training view that made S4 especially convenient.

This transition from fixed LTI dynamics to selective time-varying dynamics is the conceptual heart of Mamba (Gu et al., 2024). S4 had shown how a structured state-space model could efficiently capture long temporal dependencies. Mamba keeps that state-space foundation, but relaxes the fixed-dynamics assumption so that the model can respond to the content of the sequence itself.

As shown in Fig.6, Mamba adopts a wrapper architecture centered on a selective state space modeling component.

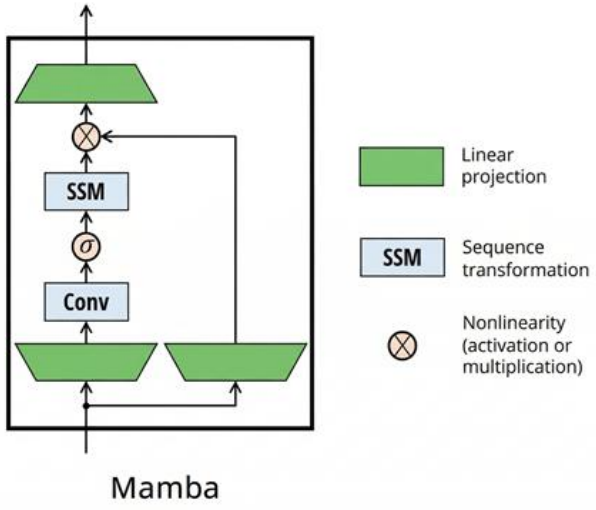


Figure 6: Mamba architecture, illustrating the selective state space block, input-dependent parameter generation, and recurrent sequence processing within the model.

5.3.2 The selective SSM operator

The core sequence operator inside Mamba is the selective state space model, often denoted as S6 in order to distinguish it from the earlier S4 formulation. Conceptually, it keeps the same general state-space backbone introduced in the previous sections: an input sequence is processed through a latent state of dimension N, and the result is mapped back to the outer model dimension D. Thus, at the structural level, the operator still behaves as a sequence-to-sequence map

$$\mathbf{X} \in \mathbb{R}^{L\times D} \longrightarrow \mathbf{Y} \in \mathbb{R}^{L\times D}.$$

So, just like the S4 operator, the selective SSM is dimension-preserving at block level and can be inserted into a deep stack without changing the outer tensor shape.

The key difference from S4 is not the overall input-output shape, but the fact that the internal sequence dynamics are now selective and time-varying. In the Mamba formulation, the state matrix $\mathbf{A}$ is kept as a global learnable parameter, while the other sequence-interaction terms are generated from the current input (Gu et al., 2024). At sequence level, this may be written as

$$\mathbf{B} = s_B(\mathbf{X}), \qquad \mathbf{C} = s_C(\mathbf{X}), \qquad \Delta = \tau_\Delta(\theta_\Delta + s_\Delta(\mathbf{X})).$$

Here, $s_B(\cdot)$, $s_C(\cdot)$, and $s_\Delta(\cdot)$ denote learned input projections, $\theta_\Delta$is a learned bias term, and $\tau_\Delta(\cdot)$is a positive activation, typically chosen as softplus. The important point is that $\mathbf{B}$, $\mathbf{C}$, and Δ now carry a sequence dimension, so they vary across time instead of being fixed once for the whole sequence.

Equivalently, at a single time step t, one may write

$$\mathbf{B}_t = s_B(\mathbf{x}_t), \qquad \mathbf{C}_t = s_C(\mathbf{x}_t), \qquad \Delta_t = \tau_\Delta(\theta_\Delta + s_\Delta(\mathbf{x}_t)).$$

This is the main selective mechanism of Mamba. The model no longer uses one fixed sequence operator for the entire input, but recomputes part of its state-space parameterization from the current input at each position.

Using the usual state-space notation, the selective recurrence may then be written in compact form as

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t, \qquad \mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t,$$

where $\mathbf{h}_t \in \mathbb{R}^N$ denotes the latent state associated with one channel. The discrete quantities $\bar{\mathbf{A}}_t$ and $\bar{\mathbf{B}}_t$ are obtained after discretization using the input-dependent step size Δ_t. Therefore, unlike S4, the effective state transition is no longer fixed across the sequence.

It is useful to state clearly what is learnable in this operator. At the S6 level, the model contains a global learnable state matrix $\mathbf{A}$, together with learned projections that generate Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ from the current input (Gu et al., 2024). Thus, compared with S4, the conceptual change is not that the state-space model disappears, but that part of its parameterization becomes input-dependent. This is exactly what gives the operator its selective behavior.

At an intuitive level, each quantity has a simple role. The matrix $\mathbf{A}$ still defines the basic latent dynamics and therefore acts as the persistent memory backbone of the operator. The step size Δ_t modulates how strongly the state is updated at position t, so it may be viewed as a local control over the memory timescale. The term $\mathbf{B}_t$ determines how strongly the current input is written into the state, while $\mathbf{C}_t$ determines how the current state is read back into the output. Because these terms depend on the input itself, the

operator can react differently to different parts of the sequence. Some inputs may be stored strongly, while others may be attenuated or quickly forgotten.

A further structural point, inherited from earlier SSMs, is that the selective scan acts across the Douter channels in parallel, rather than mixing all channels together inside one large state update. In other words, the SSM part itself is still essentially channel-wise: each channel is processed through its own latent state dynamics, and the channels are handled in parallel across the feature dimension D (Gu et al., 2024). Therefore, the selective SSM is primarily a temporal module. It is designed to model temporal dependencies along the sequence axis, while richer cross-channel interaction is mainly introduced by the surrounding projections and gating structure of the full Mamba block.

This point preserves continuity with the earlier S4 discussion. In both S4 and S6, the state-space operator should be understood first as a temporal module: it scans along the sequence length L, maintains a latent memory of size N, and returns an output again shaped in $\mathbb{R}^{L\times D}$. The difference is that S4 uses fixed dynamics and therefore admits a convolutional kernel view, whereas S6 replaces that static kernel with an input-conditioned recurrent rule. So the selective SSM may be viewed as the time-varying, content-aware continuation of the same state-space idea.

5.3.3 Why selectivity breaks the convolutional shortcut

The introduction of selectivity has one immediate consequence. Since Δ_t, $\mathbf{B}_t$, and $\mathbf{C}_t$ now depend on the current input, the selective SSM is no longer linear time-invariant. As a result, the fixed convolutional kernel view used in S4 is lost, and with it the convenient convolution-based parallelism used during training (Gu et al., 2024). In other words, once selectivity is introduced, the model gains content-aware dynamics but loses the old training convenience of the convolutional formulation. Therefore, an efficient implementation must recover that practicality in a different way. Mamba does so through a hardware-aware selective scan, discussed next.

5.3.4 Hardware-aware selective scan

Once the selective SSM loses the fixed convolutional kernel view, the practical problem becomes clear: the model must regain training efficiency in a different way. Mamba

addresses this point through a hardware-aware selective scan (Gu et al., 2024). The idea is not to restore the old convolutional shortcut, because that is no longer possible, but to make the selective recurrent computation efficient enough in practice.

The main issue is that a naive implementation of the selective recurrence would be too costly in memory traffic. The latent state must evolve across the whole sequence, and if all intermediate expanded states were repeatedly written to and read from large GPU memory, training would become inefficient. For this reason, the Mamba implementation is designed to keep as much of the state update as possible inside faster on-chip memory, while avoiding unnecessary materialization of large intermediate tensors (Gu et al., 2024).

At an intuitive level, the selective scan may be understood as a carefully organized recurrent pass over the sequence. Instead of first building one global convolution kernel and then applying it in parallel, the model processes the sequence through state updates, but does so in a way that is adapted to the memory hierarchy of modern GPUs. In this sense, efficiency is recovered not through the old kernel trick, but through a better implementation of the selective recurrence itself.

The original paper highlights three practical ingredients behind this idea (Gu et al., 2024). First, several operations are fused together, so that the computation does not repeatedly move intermediate results back and forth through slow memory. Second, the recurrence is organized through a scan-style parallel algorithm rather than through a completely naive sequential loop. Third, during training, some intermediate quantities are recomputed in the backward pass instead of being fully stored, which helps reduce memory usage.

For the present discussion, the main conclusion is simple. In S4, efficiency came largely from the LTI equivalence between recurrence and convolution. In Mamba, that equivalence is lost once selectivity is introduced. The model therefore remains efficient by a different route: not by recovering a fixed kernel, but by implementing the selective recurrent scan in a hardware-aware manner. This is the practical mechanism that allows selective state spaces to remain usable at scale.

5.3.5 From the selective SSM to the Mamba block

The selective SSM described above is the core sequence operator of Mamba, but in practice it is not used alone. As in the earlier S4 discussion, it is inserted inside a larger neural block that preserves the outer representation shape

$$\mathbb{R}^{L\times D} \longrightarrow \mathbb{R}^{L\times D}.$$

Thus, if the input of the block is a sequence of Ltokens represented in model dimension D, the output has the same shape. This point is worth stating explicitly: the Mamba block is dimension-preserving at model level, so it can be stacked repeatedly, wrapped by residual connections and normalization layers, and used as a general backbone in the same way as other deep sequence modules (Gu et al., 2024).

At a high level, the block follows a simple logic. The input sequence is first projected from dimension $D$to a wider internal representation. In the standard Mamba design, this expansion factor is usually $E = 2$, so the block temporarily works in a space larger than the original model dimension (Gu et al., 2024). This expanded representation is then split into two parallel flows. One flow is the main content path that will pass through the local convolution and the selective SSM. The other flow acts as a gating path, helping the block modulate how much of the processed information should be passed forward.

The main path first applies a short depthwise one-dimensional convolution, usually with kernel size 4, followed by a SiLU activation (Gu et al., 2024). Its role is not to replace the selective SSM, but to complement it. The local convolution gives the block an explicit bias toward very short-range temporal structure, so it helps capture nearby temporal dependencies. After this local processing, the representation is passed to the selective SSM. This is the component that scans across the sequence dimension, that is, across the Ltokens along the first axis of the input, while maintaining a latent state of size N. In this sense, the selective SSM is the part of the block intended to propagate, update, and retrieve information over longer temporal ranges, and therefore to model longer-term TD through its state-space memory.

In parallel, the second flow provides a multiplicative modulation signal, also associated with a SiLU nonlinearity. This branch is not a residual shortcut in the strict sense, because

it is not added to the main stream. Instead, it acts as a soft gating path: it modulates how much of the processed main signal should pass forward before the final projection back to dimension D (Gu et al., 2024). So, in intuitive terms, one branch extracts and processes temporal information, while the other branch controls its passage in a content-dependent way.

This design is important because it shows that Mamba is not just a recurrence in isolation, but a full sequence block. The selective SSM remains the conceptual core, yet the surrounding projections, local convolution, activation, and gating give the architecture additional flexibility at block level. In particular, the SSM part itself is mainly a temporal operator applied channel-wise in parallel: it scans along the token dimension rather than directly mixing all channels together inside one large state update. Richer feature interaction is introduced mainly by the surrounding projections and gating structure rather than by the scan alone. This is consistent with the general SSM design logic already seen in S4: the state-space operator is the central temporal mechanism, but it lives inside a wider neural block that makes deep stacking practical (Gu et al., 2024).

Once this block is defined, the full model is built by repeating it M times. As usual in deep sequence architectures, each block is placed within a residual and normalization framework, so the representation shape remains stable across depth. For the present thesis, the most important takeaway is therefore structural: the Mamba block is a stackable sequence module of shape $\mathbb{R}^{L\times D} \to \mathbb{R}^{L\times D}$, whose central innovation is the replacement of the old fixed S4 operator with a selective SSM that scans across tokens and aims to preserve relevant information over long temporal ranges. Exact variations in normalization, feed-forward additions, or other implementation refinements may differ across concrete model versions, and can be treated later where needed.

A final computational remark helps place Mamba relative to the recurrent and attention-based mechanisms discussed earlier. In a stacked GRU, the hidden size is typically chosen to match the model dimension D, so the block remains dimension-preserving and can be repeated through depth. Self-attention, by contrast, does not rely on a separate recurrent memory of this type, and its standard cost is therefore written directly in terms

of the sequence length and model dimension. Mamba introduces a different situation: although the block is still stackable as

$$\mathbb{R}^{L\times D} \longrightarrow \mathbb{R}^{L\times D},$$

its selective SSM core maintains an internal latent state of size N, distinct from the outer model dimension D (Gu et al., 2024).

Here, L denotes the number of tokens or sequence positions, $D$the outer model dimension, and N the latent SSM state dimension.

Layer Type	Complexity	Sequential Ops	Max Path Length
GRU / Recurrent	$O(L \cdot D^2)$	$O(L)$	$O(L)$
Self-Attention	$O(L^2 \cdot D)$	$O(1)$	$O(1)$
Mamba / Selective SSM	$O(L \cdot D \cdot N)$	$scan - parallel$	$O(L)$

Table 2: High-level comparison of GRU, self-attention, and Mamba in terms of computational complexity, sequential operations, and maximum dependency path length.

Table 2 should be read only as a high-level comparison. GRU and Mamba both preserve a linear dependence on the sequence length L, because they propagate information through a state update across tokens. Self-attention instead reduces the path length between tokens to a constant, but pays a quadratic cost in L. Mamba therefore keeps the linear-length scaling of recurrent and state-space models, while remaining stackable at block level because its outer input and output still have shape $\mathbb{R}^{L\times D}$, even though its internal selective SSM uses a distinct latent state of dimension N.

6 Experimental Setup and Methodology

This chapter details the systematic framework designed to evaluate the interplay between tokenization paradigms and modern sequence modeling backbones. The purpose is not to reintroduce the theoretical foundations of GRU, Transformer, or Mamba, which have already been discussed in Chapters 4 and 5, but to define clearly how these backbones are instantiated in the present empirical study. In particular, the chapter is organized to isolate the effect of input representation while keeping the forecasting task, datasets, and evaluation protocol as consistent as possible across the compared models. This makes it

possible to study more directly how different tokenization strategies interact with different architectural inductive biases in long-horizon multivariate forecasting.

6.1 Forecasting formulation

6.1.1 Long-term multivariate forecasting task

The objective of long-term time series forecasting is to map a historical look-back window of multivariate observations to a future prediction horizon. Formally, given an input sequence $\mathbf{X} \in \mathbb{R}^{L \times V}$, where L is the look-back length and V is the number of observed variates, the goal is to predict the corresponding future sequence $\widehat{\mathbf{Y}} \in \mathbb{R}^{T \times V}$, where T denotes the prediction horizon. Each row of $\mathbf{X}$ represents the vector of all variates recorded at one timestamp, while each row of $\widehat{\mathbf{Y}}$ corresponds to one future time step to be forecasted. This is the common forecasting task shared by all experimental configurations in the chapter.

In batch form, the same forecasting problem is written as $\mathbf{X} \in \mathbb{R}^{B \times L \times V} \rightarrow \widehat{\mathbf{Y}} \in \mathbb{R}^{B \times T \times V}$, where B denotes the batch size. This batch notation will be used when discussing the detailed tensor transformations inside the model configurations.

Throughout this methodology, the following tensor notation is used consistently:

B: batch size.
L: sequence length.
V: number of variates.
T: prediction horizon.
H: latent dimension for Time-Token models.
D: latent dimension for Variate-Token (Inverted) models.

For ease of reading, tensor shapes such as $B \times L \times V$ or $B \times T \times V$ will later be written in compact form as **BLV** and **BTV** when no ambiguity arises. The same convention will be used in Sections 6.3.2 and 6.3.3 for the intermediate encoder representations.

This convention is important because the chapter compares two different tokenization views of the same forecasting problem. In Time-Token models, the encoder processes a sequence of Ltemporal tokens, each built from the Vobserved variates and embedded into

a latent space of dimension H. In Variate-Token models, the input is instead transposed so that each token corresponds to one variate, represented through its full historical biography and embedded into a latent space of dimension D. Thus, the forecasting task remains the same, but the semantic meaning of a token changes across configurations.

The distinction between temporal dependency and inter-variate correlation, already introduced in previous chapters, is therefore recalled here only in operational terms. Along the temporal axis, the model must use the past Lobservations to forecast the next Tfuture steps. Along the variate axis, the model must determine whether information from one series can improve the prediction of another. The two tokenization paradigms explored in this chapter expose these two axes differently to the encoder backbone, and this is the main experimental variable of the study.

6.1.2 Generative Direct Multi-Step (DMS) Decoding

In this study, forecasting is performed through a Generative Direct Multi-Step decoding strategy. Instead of relying on an Iterated Multi-Step procedure, where future values are generated recursively one step at a time, the model is trained to predict the entire future horizon Tin a single forward pass. In other words, the forecasting function is learned as a direct mapping

$$f_\theta : \mathbb{R}^{L \times V} \rightarrow \mathbb{R}^{T \times V}.$$

This choice follows the same motivation already discussed in earlier chapters. Recursive decoding suffers from error accumulation, because each predicted step may influence the next one. This effect becomes especially problematic in long-horizon forecasting, where small local inaccuracies can propagate and amplify across the decoding trajectory. Direct multi-step prediction avoids this issue by treating the full horizon as one structured regression target. For this reason, all six configurations evaluated in this chapter use a direct linear prediction head rather than an autoregressive decoder.

At implementation level, this means that each encoder backbone first produces a latent representation, and a final generative linear head maps that representation to the whole target horizon. The exact form of this mapping depends on the tokenization strategy. In Time-Token models, the encoder output is organized along the temporal token axis and

must be reduced or pooled before the final projection to $T \times V$. In Variate-Token models, the encoder output is organized along the variate axis, so the projection operates on a different semantic structure, although the final prediction target remains identical. This difference will become important in the detailed model configurations of Section 6.3.

6.2 Tokenization Strategies Under Evaluation

The primary experimental variable in this methodology is the tokenization paradigm. In the present thesis, a token is understood as the elementary semantic unit ingested by the encoder backbone. This choice is not a secondary preprocessing detail, but a structural modeling decision, because it determines which axis of the input is directly exposed to the sequence operator. As already discussed in earlier chapters, different tokenizations induce different biases toward temporal dependency (TD) and inter-variate correlation (VC). Chapter 6 therefore treats tokenization as the central controlled variable of the experimental design, while the backbone family is varied around it.

6.2.1 The Time-Token (TD-Centric) Paradigm

The standard approach in sequence forecasting is to treat each timestamp as one token. In the Time-Token configuration, the input

$$\mathbf{X} \in \mathbb{R}^{L \times V}$$

is viewed as a sequence of Ltokens, where each token contains the Vobserved variates recorded at one time step. This sequence is then projected into a latent space of dimension H, yielding an encoder input of shape

$$\mathbb{R}^{L \times H}.$$

Thus, the encoder backbone processes a temporal token stream whose length is equal to the look-back window.

The inductive bias of this formulation is clear: it directly privileges temporal dependency. Since the token axis is the time axis, the main sequence operator, whether recurrent, attentional, or state-space based, compares and propagates information across different moments in time. In this sense, token-to-token interaction is explicitly TD-centric. VC is

still present, but it is compressed inside each token because all variates observed at the same timestamp are fused before the encoder sees them.

This token definition also reveals the main weakness of the time-token view. A single timestamp often mixes measurements that come from distinct physical processes and may evolve with different delays or different noise characteristics. For instance, in an energy dataset, load, temperature, and oil measurements recorded at the same instant do not necessarily constitute one coherent semantic unit. Their immediate fusion may therefore introduce what the current draft calls **token noise**: the token becomes a heterogeneous container whose internal composition may blur useful single-variate structure before the encoder has a chance to process it. This is one of the main motivations for later considering both patching and inversion.

Even so, the time-token setting remains important because it is the most direct formulation when the objective is to model TD explicitly. GRU naturally fits this regime, since it scans across temporal positions. Likewise, a standard Transformer encoder in this setting applies self-attention across time tokens, while a Time-Token Mamba scans across the same temporal axis using its selective state-space update. Thus, although the three backbones differ in how they process the token sequence, they all receive the same TD-oriented tokenization in this family of experiments.

A further point concerns the output geometry. Since the encoder returns a representation organized along the temporal token axis, the model must still reduce or aggregate this structure before generating the forecasting horizon. In other words, the model does not directly obtain one token per target variate. Instead, it must transform a temporal latent sequence into a prediction tensor of shape $\mathbb{R}^{T\times V}$. This is one reason why the output head in Time-Token models is structurally less direct than in the inverted setting.

Finally, patching may be understood as an intermediate refinement of this same viewpoint. Rather than treating each timestamp as one isolated token, patching groups short temporal neighborhoods into sub-series tokens. As the old draft already notes, this provides two benefits: first, it acts as a denoising step, because each token now contains a small local temporal context instead of one fragile measurement vector; second, it reduces the effective token length seen by attention from $L$to a shorter patched sequence,

thereby reducing computational cost. However, patching still remains within the TD-centric family, since the token axis is still fundamentally temporal rather than variate-wise.

6.2.2 The Variate-Token (VC-Centric / Inverted) Paradigm

The Variate-Token setting redefines the same forecasting problem by changing what counts as a token. Instead of treating each timestamp as one token, the input is transposed so that each observed variate becomes one token described by its full historical trajectory. Starting from $\mathbf{X} \in \mathbb{R}^{L \times V}$, the input is reordered as $\mathbf{X}^{\top} \in \mathbb{R}^{V \times L}$, so that the token axis is no longer time, but the variate index itself. Each token is therefore the history of one variable over the look-back window, and this token is projected into a latent space of dimension D, yielding an encoder input of shape $\mathbb{R}^{V \times D}$.

The main consequence of this inversion is that the encoder no longer compares timestamps directly. Instead, it compares variates. In the language of this thesis, the token-to-token interaction becomes primarily VC-oriented rather than TD-oriented. The encoder is therefore encouraged to learn relations among variable histories, instead of immediately scanning across time positions.

This also makes each token more semantically coherent. In the Time-Token setting, one token mixes many variates observed at the same instant. In the Variate-Token setting, one token corresponds to one physical signal and contains its temporal biography across the whole look-back window. This reduces the heterogeneity inside each token and makes the token itself more interpretable.

A further implication concerns positional encoding. This issue mainly concerns the Transformer configuration, since self-attention is parallel and does not contain an intrinsic notion of order. In the Variate-Token Transformer setting, however, the tokens are variates rather than timestamps, so no natural temporal ordering exists along the token axis. For this reason, positional encoding is removed in the inverted Transformer formulation. In GRU and Mamba settings, this problem does not arise in the same way, because both are sequential operators and positional encoding was already absent even in the Time-Token case.

Another important advantage of the inverted setting is the output geometry. Since the encoder output is already aligned with the variate axis, each encoded variate representation can be projected more directly from latent dimension D to the prediction horizon T. Thus, compared with the Time-Token setting, the final prediction head is structurally more direct. In particular, no additional encoder-output attention reduction over the temporal length L is needed to collapse the encoder representation before forecasting, unlike the issue discussed for Time-Token models in Section 4.2.4.

At an intuitive level, the Variate-Token setting separates the two dependency types more clearly. The first encoder stage, or black-box sequence operator, acts along the first dimension of the tokenized input and therefore primarily processes VC, since that first dimension now indexes variates. Temporal dependency is then handled differently depending on the backbone. In Transformer and Mamba configurations, TD is mainly refined by the FFN stage that follows the black-box operator. In the GRU configuration, by contrast, TD is not delegated to a separate FFN block, but is handled through the GRU's own recurrent update mechanism.

Three practical observations summarize the interest of the inverted formulation:

- **VC becomes directly visible to the encoder.** Token-to-token interaction now operates across variable histories rather than across timestamp-level mixtures.
- **TD is preserved, but no longer occupies the first interaction axis.** In Transformer and Mamba setups, the first black-box stage is mainly VC-oriented, while TD is further processed by the FFN stage. In the GRU setup, TD is maintained through the recurrent dynamics of the GRU itself.
- **The setting may support variate generalization more naturally.** Since each token corresponds to a whole variable history, the model can learn reusable cross-variate patterns at the channel level. In practice, this means that even when only a subset of variables is seen during training, the model may still generalize part of what it has learned to unseen but related variates, as suggested by the kind of partial-variate experiments discussed in the inverted forecasting literature.

6.3 Detailed Model Configurations

After defining the forecasting task and the two tokenization paradigms, the next step is to specify the concrete model configurations used in the experiments. The objective of this section is not to introduce new backbone families, since GRU, Transformer, and Mamba have already been discussed in previous chapters, but to describe how they are instantiated here under a common forecasting framework. In all cases, the model follows the same high-level logic: an input sequence is first tokenized and embedded, then processed by a stack of encoder sublayers, and finally mapped to the future prediction horizon through a direct generative linear head. What changes across configurations is therefore the token definition and the type of encoder sublayer used inside the backbone.

6.3.1 Common Structural Elements

Despite the differences among the six tested configurations, they share a common outer structure. Each model can be summarized as

Input → Token Embedding → Encoder Backbone → Linear Prediction Head → $\widehat{\mathbf{Y}}$.

Thus, all configurations solve the same direct multi-step forecasting task and all produce a final prediction of shape $\mathbb{R}^{T\times V}$. The comparison is therefore controlled at the level of the internal encoder design rather than at the level of the output formulation.

In all configurations, both the Token Embedding layer and the final prediction head are implemented as simple linear projectors. The Token Embedding layer maps the raw token content into the latent space used by the encoder backbone, while the head maps the final latent representation to the target forecasting dimensions. Thus, the main architectural variation across models does not lie in the input embedding or output projection, but in the encoder sublayer used between them.

The encoder backbone is built by stacking M sublayers of the same type. In compact form: Encoder Backbone $=$ [Sublayer] $\times M$.

This stackability is important because each sublayer preserves the outer tokenized representation shape, so deeper encoders can be built while keeping the same interface with the final linear prediction head.

The three sublayer types used in this chapter are the following:

- **Transformer Sublayer** = [MHA + Add&Norm,FFN + Add&Norm]
- **Mamba Sublayer** = [Mamba + Add&Norm,FFN + Add&Norm]
- **GRU Sublayer** = GRU

Thus, Transformer and Mamba follow the same encoder-template logic, differing only in the identity of the black-box sequence operator. In the Transformer case, the black box is multi-head self-attention. In the Mamba case, it is the selective state-space operator. GRU instead uses a simpler recurrent sublayer, without the same Add-and-Norm wrapper and without a separate FFN stage. The practical reason for keeping this simpler GRU form will be discussed later in the configuration notes.

This distinction matters in both tokenization paradigms. In the Time-Token setting, the black-box operator processes the temporal token axis directly, so the first stage is TD-oriented. In the Variate-Token setting, the same operator processes the variate token axis, so the first stage becomes VC-oriented. The subsequent FFN stage in Transformer and Mamba then helps refine TD inside the token representations, while in the GRU case this role is absorbed more directly into the recurrent dynamics. In addition, because GRU and Mamba are sequential operators, their variate-token versions are made bidirectional so that the encoder can better capture channel interactions when the variate axis has no natural ordering.

For clarity, the six experimental configurations are organized into two groups. The first group contains Time-Token models, where the token axis corresponds to time. The second group contains Variate-Token models, where the token axis corresponds to variates. Within each group, the three backbones are instantiated under the same outer forecasting logic, so that the comparison remains focused on tokenization and encoder bias rather than on unrelated output differences.

6.3.2 Configuration Set A: Time-Token Models

In the Time-Token family, each timestamp is treated as one token. Therefore, the input batch **X** starts from shape **BLV**, and after Token Embedding it becomes **BLH**, where H is

the latent dimension used in the Time-Token setting. The encoder then processes the sequence along the temporal axis, so TD is the first interaction target.

A practical consequence is immediate: after the encoder, the representation is still organized over the temporal length L. To produce the final forecast, this L-dependent output must be reduced before the prediction head. Otherwise, a direct projection from the full temporal encoder output would make the head unnecessarily large, causing parameter explosion. For this reason, all Time-Token models use the same final logic:

BLH → temporal pooling → BH → head $(H \to T \cdot V)$ **→ B(TV) → view → BTV**.

Here, **temporal pooling** removes the Ldimension before the head. Depending on the configuration, this is implemented either as last-state pooling or as additive attention pooling, following the same motivation discussed earlier for direct forecasting heads. The final **view** operation simply reshapes the flat 2D output into the final 3D tensor **BTV**.

Time-Token GRU

The first configuration uses a GRU encoder. Since GRU is naturally sequential, it scans the temporal token stream directly and models TD through its recurrent hidden-state update. The model flow is

BLV → Token Embedding $(V \to H)$ **→ BLH → [GRU]** $\times M$ **→ BLH → temporal pooling → BH → head** $(H \to T \cdot V)$ **→ B(TV) → view → BTV**.

Here the GRU itself is the whole sublayer. No Add-and-Norm wrapper and no FFN block are added around it. This simpler form is kept intentionally, since in the present setup the wrapped version was not beneficial.

Time-Token Transformer

The second configuration uses a Transformer encoder. After Token Embedding, sinusoidal positional encoding is added, since self-attention is parallel and does not preserve temporal order by itself. The encoder is then built as a stack of MTransformer sublayers,

Transformer Sublayer = [MHA + Add&Norm, FFN + Add&Norm].

The full flow is

BLV → Token Embedding $(V \rightarrow H)$**+ PE → BLH → [Transformer Sublayer]** $\times M \rightarrow$ **BLH → temporal pooling → BH → head** $(H \rightarrow T \cdot V)$ **→ B(TV) → view → BTV**.

In this case, TD is modeled first through temporal self-attention across the Ltokens, then refined by the FFN stage. The pooled encoder output is finally projected to $T \cdot V$ and reshaped into **BTV**.

Time-Token Mamba

The third configuration uses a Mamba encoder. As in the GRU case, the sequence operator is sequential, so no positional encoding is required. The encoder is built as a stack of MMamba sublayers,

Mamba Sublayer = [Mamba + Add&Norm, FFN + Add&Norm].

The full flow is

BLV → Token Embedding $(V \rightarrow H)$ **→ BLH → [Mamba Sublayer]** $\times M \rightarrow$ **BLH → temporal pooling → BH → head** $(H \rightarrow T \cdot V)$ **→ B(TV) → view → BTV**.

Here, TD is first processed by the selective state-space scan along the temporal axis, then refined by the FFN stage. As in the other Time-Token settings, temporal pooling is required to remove the Ldimension before the final prediction head.

Summary of the Time-Token family

The three Time-Token configurations differ only in the encoder sublayer stacked after Token Embedding:

- **Time-Token GRU**: [GRU] $\times M$
- **Time-Token Transformer**: [MHA + Add&Norm,FFN + Add&Norm] $\times M$
- **Time-Token Mamba**: [Mamba + Add&Norm,FFN + Add&Norm] $\times M$

In all three cases, the first interaction axis is time. This makes the whole family explicitly TD-centric. A second common point is the output stage: all Time-Token models must

remove the temporal dimension before prediction, so they all use **temporal pooling + head + view**.

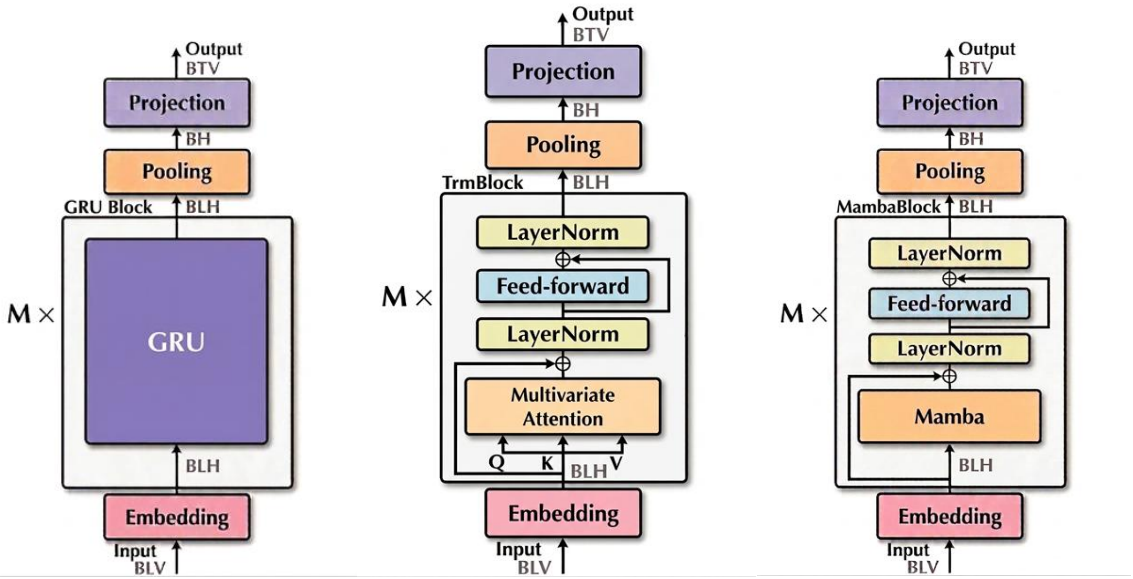


Figure 7: Time-token model configurations, comparing GRU, Transformer, and Mamba under the temporal-token representation in which each timestamp is treated as one input token.

6.3.3 Configuration Set B: Variate-Token Models

In the Variate-Token family, each variate history is treated as one token. Therefore, the input batch first undergoes a **flip** from **BLV** to **BVL**, so that the token axis becomes the variate axis. After Token Embedding, the representation becomes **BVD**, where D is the latent dimension used in the Variate-Token setting. The encoder then processes the sequence along the variate axis, so VC is the first interaction target.

A practical consequence is immediate: unlike the Time-Token case, the encoder output is already aligned with the variate axis, so no temporal pooling is needed before forecasting. The common final logic is therefore

BLV → flip → BVL → Token Embedding ($L \to D$) **→ BVD → Encoder Backbone → BVD → head** ($D \to T$) **→ BVT → flip → BTV**.

Here, the first **flip** converts the input from time-major to variate-major form, while the final **flip** restores the standard forecasting layout **BTV**.

Variate-Token GRU

The first configuration uses a GRU encoder on variate tokens. Since the token axis is now the variate axis, the recurrent scan no longer moves across time positions, but across channels. Because channels have no natural left-to-right order, a unidirectional recurrent pass would be too restrictive. For this reason, the GRU encoder is made **bidirectional** in the Variate-Token setting, so that each variate token can be contextualized from both directions and the encoder can better emulate the global channel interaction that attention provides more naturally.

The full flow is

BLV → flip → BVL → Token Embedding $(L \to D)$ **→BVD → [Bi-GRU]** $\times M$ **→ BVD → head** $(D \to T)$ **→ BVT → flip → BTV**.

Here again, the GRU itself is the whole sublayer. No Add-and-Norm wrapper and no separate FFN stage are added around it. In this inverted setting, VC is first processed by the bidirectional recurrent scan across variates, while TD remains encoded inside each token through the temporal biography already contained in that token and through the GRU dynamics themselves.

Variate-Token Transformer

The second configuration uses a Transformer encoder on variate tokens. Since self-attention is parallel, it can compare all variate tokens directly, making it naturally suited to the VC-first logic of the inverted setting. At the same time, because the token axis is now variate-wise rather than temporal, no positional encoding is used here.

The encoder is built as a stack of M Transformer sublayers,

Transformer Sublayer = [MHA + Add&Norm, FFN + Add&Norm].

The full flow is

BLV →flip →BVL →Token Embedding $(L \to D)$ **→BVD →[Transformer Sublayer]** $\times M$ **→BVD →head** $(D \to T)$ **→BVT →flip →BTV**.

In this configuration, the first black-box stage, here multi-head attention, operates across variates and therefore primarily processes VC. The FFN stage then acts inside each token

representation and helps refine TD. Thus, the Transformer sublayer naturally splits the two roles: black-box first for VC, FFN second for TD.

Variate-Token Mamba

The third configuration uses a Mamba encoder on variate tokens. As in the GRU case, the sequence operator is sequential, so once the token axis becomes the variate axis, the scan moves across channels rather than across time. For the same reason as above, the Mamba encoder is made **bidirectional** in the Variate-Token setting: since variates are unordered, bidirectionality helps the model better approximate a global channel-interaction mechanism.

The encoder is built as a stack of M Mamba sublayers,

Mamba Sublayer = [Mamba + Add&Norm, FFN + Add&Norm].

The full flow is

BLV → flip → BVL → Token Embedding $(L \to D)$ **→ BVD → [Bi-Mamba Sublayer]** $\times M$ **→ BVD → head** $(D \to T)$ **→ BVT → flip → BTV**.

Here, the first black-box stage is the bidirectional selective state-space scan across variates, so it is primarily VC-oriented. The following FFN stage then refines TD inside each token representation. Therefore, as in the Variate-Token Transformer, the two roles are separated more clearly: the first stage models cross-variate interaction, while the FFN stage helps process temporal information already stored inside each variate token

Summary of the Variate-Token family

The three Variate-Token configurations differ only in the encoder sublayer stacked after Token Embedding:

- **Variate-Token GRU**: [Bi-GRU] $\times M$
- **Variate-Token Transformer**: [MHA + Add&Norm,FFN + Add&Norm] $\times M$
- **Variate-Token Mamba**: [Bi-Mamba + Add&Norm,FFN + Add&Norm] $\times M$

In all three cases, the first interaction axis is variate, since the token dimension corresponds to the V channels. This makes the whole family explicitly VC-centric. A second common point is the output stage: unlike the Time-Token family, no pooling is required here, because the encoder output is already organized along the variate axis. The final head maps each token directly from latent dimension D to the forecasting horizon T, after which a final flip restores the standard output shape **BTV**.

A final practical note concerns scan direction, normalization, and decoding. In the Variate-Token family, the encoder scans across variates rather than across time. For Transformer this is natural, since attention already sees all variates in parallel. For GRU and Mamba, instead, the scan is sequential, so a unidirectional pass would tend to privilege closer variates and impose an artificial order over channels. For this reason, both GRU and Mamba are made bidirectional in the inverted setting, so that each variate token can receive context from both sides.

Normalization also changes meaning in this family. Since each token now corresponds to one full variate history, LayerNorm acts on the latent representation of each variate token rather than on a timestamp-level mixture of all variates. In practice, this means that the model normalizes each variate biography individually, which is often more robust when different channels have different scales or non-stationary behavior.

Finally, decoding is simpler than in the Time-Token family. Since the encoder output is already aligned with the variate axis, no temporal pooling is needed. The head maps each token directly from D to T, producing **BVT**, and a final flip restores the standard forecasting layout **BTV**.

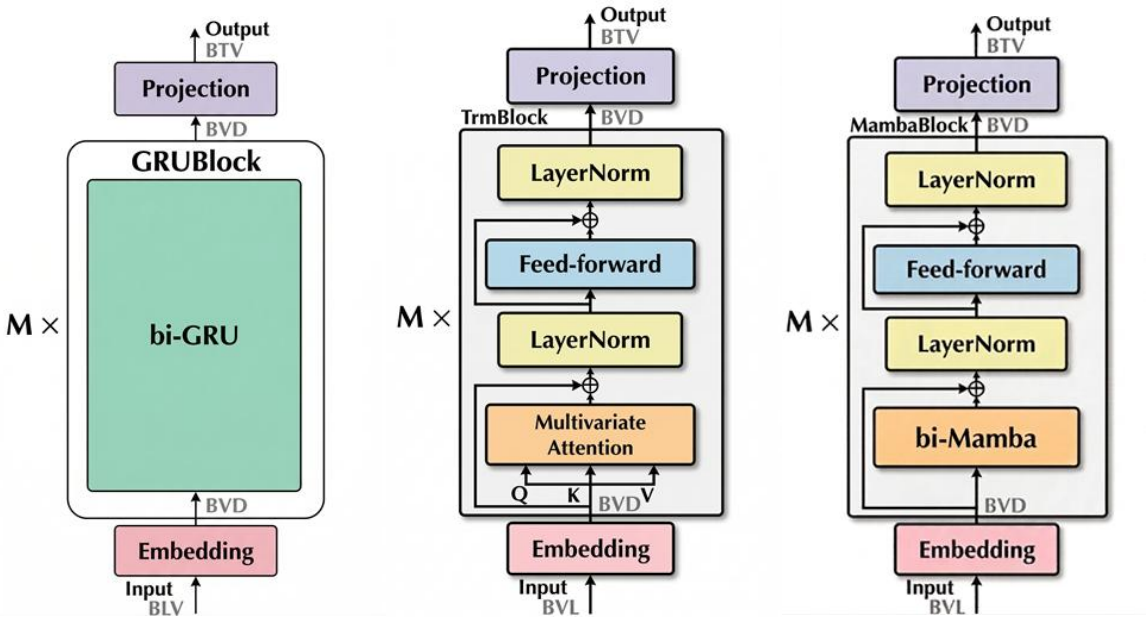


Figure 8: Variate-token model configurations, comparing GRU, Transformer, and Mamba when each token corresponds to the historical trajectory of one variate.

6.4 Datasets

This study evaluates all forecasting configurations on two benchmark multivariate time-series datasets drawn from the energy domain: ETTh1 and ECL. These datasets were selected because they provide a useful contrast in both dimensionality and distributional behavior. ETTh1 represents a lower-dimensional transformer monitoring scenario, whereas ECL represents a high-dimensional electricity consumption setting with a much larger number of variates. Together, they provide a suitable testbed for analyzing how tokenization strategy and encoder architecture behave under different structural conditions.

6.4.1 Electricity Transformer Temperature (ETT)

The ETT dataset was specifically introduced in the *Informer* study to address the scarcity of high-frequency, long-term datasets in the energy sector (*source: https://github.com/zhouhaoyi/ETDataset*). The data was collected from two separate electricity transformers (labeled Station 1 and Station 2) in China over a two-year period from July 2016 to July 2018.

The dataset is divided into four subsets based on sampling frequency:

- **ETTh1 & ETTh2:** Hourly resolution (17,420 timesteps each).
- **ETTm1 & ETTm2:** 15-minute resolution (69,680 timesteps each).

Each subset contains 7 variates. These include the **Oil Temperature (OT)**, which serves as the primary target variable for univariate forecasting, and six load-related features: High Use Fulfilment Load (HUFL), High Use Less Load (HULL), Medium Use Fulfilment Load (MUFL), Medium Use Less Load (MULL), Low Use Fulfilment Load (LUFL), and Low Use Less Load (LULL).

From an application perspective, forecasting transformer oil temperature is useful because it can support transformer condition monitoring and preventive maintenance by anticipating potentially critical thermal behavior.

6.4.2 Electricity Consuming Load (ECL)

The ECL dataset (often referred to as *Electricity*), originally sourced from the UCI Machine Learning Repository, tracks the electricity consumption of 321 individual clients in Portugal between 2011 and 2014 (*source: https://zenodo.org/records/4656140*). Unlike the ETT dataset, which focuses on transformer health, ECL is a benchmark for large-scale consumer demand forecasting.

- **Variates**: 321 distinct time series (one per client).
- **Timesteps**: 26,304 recorded at a 1-hour granularity.
- **Dynamics**: The data exhibits strong daily and weekly seasonality driven by human activity, making it a rigorous test for long-horizon forecasting.

From an application perspective, forecasting electricity load is useful for demand monitoring, consumption planning, and broader grid management tasks.

6.4.3 Distribution Shift Considerations

An additional aspect that differentiates the selected datasets is the degree of **distribution shift between training and test splits across variates**. While all datasets are normalized using statistics computed on the training split, the underlying distributions of the signals may still exhibit shifts over time, which directly impacts forecasting difficulty.

In the **ECL dataset**, the distribution of each channel remains relatively stable between training and test splits. As illustrated qualitatively across multiple sampled channels, both the mean and variance remain closely aligned after normalization, with only minor deviations (Fig. 9). This suggests a **low distribution shift regime**, where the statistical properties of the signals are largely preserved over time. Consequently, models can rely on consistent patterns during training and expect similar dynamics at inference time.

In contrast, the **ETTh1 dataset** exhibits a more pronounced distribution shift across several channels (Fig. 10). Even after per-channel normalization, noticeable discrepancies in mean, variance, and distribution shape can be observed between training and test segments. For instance, some channels display clear shifts in central tendency or significant variance compression/expansion, indicating that the model must generalize across **non-stationary conditions**. This makes ETTh1 inherently more challenging, as the learned representations must remain robust under changing statistical regimes.

This distinction is particularly relevant for multivariate forecasting:

- In **low-shift environments (ECL)**, leveraging a large number of stable variates can enhance predictive performance through richer cross-channel correlations.
- In **high-shift environments (ETTh1)**, the smaller number of variates combined with distribution drift reduces redundancy and increases the difficulty of generalization.

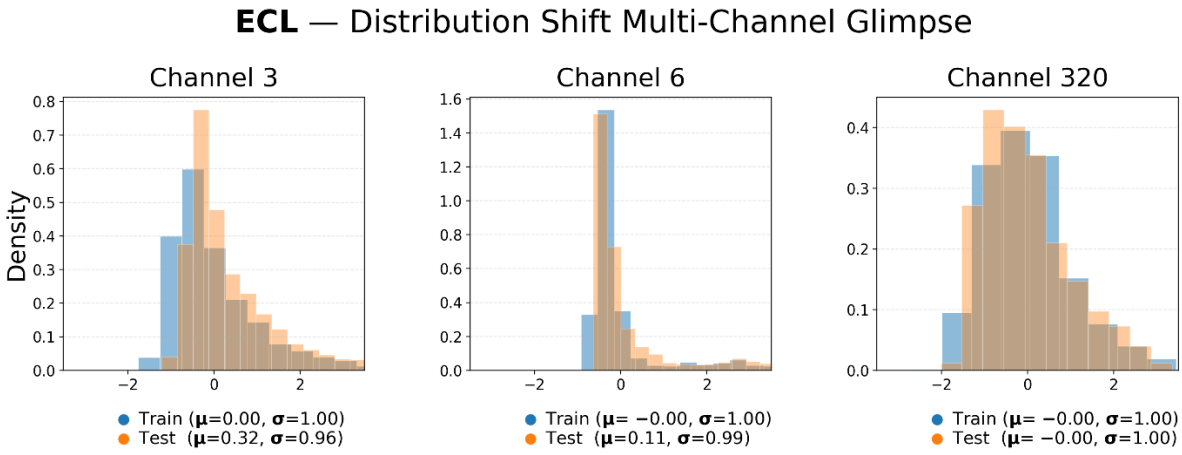


Figure 9: Distribution shift in ECL across sampled channels, showing relatively stable training and test distributions after per-channel normalization.

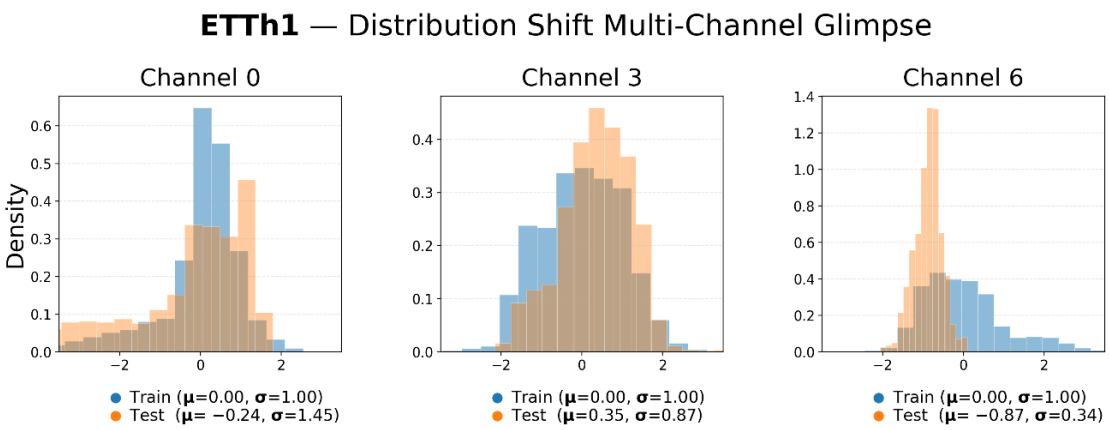


Figure 10: Distribution shift in ETTh1 across sampled channels, showing more pronounced differences between training and test distributions after per-channel normalization.

Summary of Experimental Datasets

The following table summarizes the key characteristics of the datasets used in this research, formatted according to standard SOTA reporting styles:

Datasets	*ETTh1 & ETTh2*	*ETTm1 & ETTm2*	*Electricity (ECL)*
Variates	7	7	321
Timesteps	17,420	69,680	26,304
Granularity	1 hour	15 min	1 hour
Domain	Power Transformer	Power Transformer	Consumer Load

Table 3: Summary of the experimental datasets used in this study, including dimensionality, sampling granularity, and application domain.

6.5 Data Preprocessing

The transformation of raw time-series data into a format suitable for deep learning follows a structured pipeline. To maintain the integrity of the evaluation, we execute the preprocessing in a specific order: temporal splitting, followed by normalization, and finally windowization.

6.5.1 Temporal Splitting

Before any transformation is applied, the continuous time series of length N_{total} is divided into three distinct subsets along the temporal axis: **Training**, **Validation**, and **Test** sets. This strict chronological partitioning prevents any future information from leaking into the training phase.

- **Training Set**: Used as the primary foundation for the model to learn latent patterns and temporal dependencies.
- **Validation Set**: Utilized for hyperparameter optimization and as a trigger for early stopping mechanisms.
- **Test Set**: An unbiased evaluation set used exclusively to simulate real-world deployment on truly unseen future events.

6.5.2 Per-Channel Z-Score Normalization

Once the splits are established, we perform standardization to ensure stable gradient descent across all V variates (channels). We utilize **Z-score normalization**, calculated independently for each channel to preserve unique scale characteristics. To avoid look-ahead bias, the mean (μ) and variance (σ^2) are computed **strictly using only the training split**:

$$x'_{i,v} = \frac{x_{i,v} - \mu_{train,v}}{\sigma_{train,v}}$$

where i is the timestep and $v \in \{1, \dots, V\}$. These training parameters are then applied to the validation and test sets to maintain a consistent feature space.

6.5.3 Windowization and Sample Generation

The final step is the creation of supervised learning samples using a sliding window approach. Each sample consists of a couple *(X, Y),* where *X* is the input window and *Y* is the target horizon.

We define the following parameters for this process:

- **Look-back Window (*L*):** The length of the historical temporal window used as input *X*.
- **Forecasting Horizon (*T*):** The length of the target sequence *Y* that the model aims to predict.
- **Stride (*S*):** The number of steps between the start of one sample and the next. In this study, we set ***S=1*** to capture all available overlapping temporal patterns.

For a given split containing *N* timesteps, the number of generated windows *W* is determined by the following formula:

$$W = \left\lfloor \frac{N - (L + T)}{S} \right\rfloor + 1$$

Every row in the resulting dataset represents a single sample where the window *L* and horizon *T* are concatenated to form the total sample span.

6.5.4 Borderline Information Loss

It is important to note that the windowization process inherently results in a minor loss of information at the "borderlines" between splits. Because a sample *(X, Y)* requires a continuous block of *L + T* timesteps, the trailing *L + T - 1* timepoints at the end of the Training and Validation sets cannot serve as the start of a full window within that specific split. These segments are effectively "lost" to ensure that no window crosses the boundary between, for example, the training and validation data, which would otherwise introduce data leakage.

6.6 Training Configuration

The experimental environment and hyperparameter settings are designed for high-performance deep learning, ensuring efficient processing of complex multivariate sequences.

6.6.1 Hardware and Software Environment

All experiments are implemented in **PyTorch** and conducted on a localized workstation running **Ubuntu Linux**. The primary computational engine is an **NVIDIA GeForce RTX 4080 GPU** with **16GB of VRAM**. This hardware configuration allows for the parallel processing of high-dimensional datasets, such as ECL with its 321 variates, while accommodating significant d_{model} (token embedding) sizes.

6.6.2 Optimization Strategy and Criterion

The model optimization is governed by the following parameters:

- **Criterion**: We utilize **Mean Squared Error (MSE)** as the loss function for all experiments. MSE is the standard regression criterion for forecasting as it heavily penalizes large outliers, encouraging the model to track the mean trend of the series.
- **Optimizer**: We employ the **AdamW** optimizer, which utilizes decoupled weight decay to improve generalization.
- **Learning Rate**: The initial learning rate is set to 10^{-4}.
- **Weight Decay**: A decay factor of 1×10^{-5} is applied to regularize the weights.
- **Scheduler**: We utilize a **ReduceLROnPlateau** scheduler, which reduces the learning rate by a specific factor if the validation loss fails to improve after a set number of epochs.

6.6.3 Training Loop and Reproducibility

The training duration is fixed at a maximum of **20 epochs**. To prevent overfitting and save computational time, we implement an **early stopping** mechanism with a **patience of 10 epochs**. If the validation MSE does not reach a new minimum within this window, training is terminated.

To ensure the scientific validity and reproducibility of the results, all experiments are performed using fixed random seeds. This ensures that the initialization of model weights, dropout layers, and data shuffling remains consistent across different runs, allowing for a fair comparison of architectural performance.

7 Experimental Results and Analysis

7.1 Overview of Evaluation Protocol

This section presents a comprehensive analysis of the experimental results obtained across different model architectures and tokenization strategies. The evaluation considers six configurations, derived from the combination of three encoder types—GRU, Transformer, and Mamba—and two tokenization strategies, namely time-token and variate-token formulations.

Experiments are conducted on two benchmark multivariate time series datasets, ECL and ETTh1, which exhibit distinct structural characteristics in terms of dimensionality and distribution shift, as discussed in Chapter 6. For each configuration, two input window lengths are evaluated, $L = 96$ and $L = 192$, while the forecasting horizon is fixed to $T = 96$.

Performance is assessed along three main axes:

- Forecasting accuracy, measured through Mean Squared Error (MSE) and Mean Absolute Error (MAE).
- Computational efficiency, measured in average milliseconds per iteration (ms/iter).
- Resource utilization, including GPU memory consumption and model size in MB.

All measurements are obtained under identical experimental conditions, with fixed batch size $B = 32$. The ms/iter metric corresponds to a full forward and backward pass over one batch, thus representing the true computational cost of a single optimization step.

Computational Perspective

To interpret the efficiency results, it is useful to recall the main scaling behavior of the considered encoder families. Let N denote the number of tokens processed by the encoder, and d_{model} the embedding dimension.

- GRU (sequential scan): $O(num_layers\ N\ d_{model}^2)$. GRU processes tokens sequentially, making it highly sensitive to large N. Bidirectional variants further increase the computational cost and parameter count.
- Transformer (parallel attention): $O(num_layers\ (N^2 d_{model} + N\ d_{model}\ d_{ff}))$. Attention introduces quadratic dependence on N, but benefits from full parallelization. The FFN contributes an additional cost that becomes important whenever $d_{model}$and d_{ff} are large.
- Mamba (state-space scan): $O(num_layers\ (N\ d_{model}^2 + N\ d_{model}\ d_{ff}))$. Mamba operates linearly in N, but still has non-negligible per-token cost, especially when embedding dimension and FFN size are large.

Across all models that include feed-forward blocks, the FFN contributes a term $O(N\ d_{model}\ d_{ff})$, which becomes particularly relevant when the number of tokens is small and the embedding dimension is large. These differences will be crucial in explaining the efficiency and memory trends discussed later in the chapter.

7.2 Accuracy Analysis

TEST METRICS (T=96)

	L96 MSE	L96 MAE	L192 MSE	L192 MAE
ECL				
gru-variate	0.1502 ± 0.0004	0.2528 ± 0.0008	0.1388 ± 0.0003	0.2391 ± 0.0004
transformer-variate	**0.1489 ± 0.0011**	**0.2460 ± 0.0015**	**0.1335 ± 0.0008**	**0.2314 ± 0.0016**
mamba-variate	0.1505 ± 0.0014	0.2513 ± 0.0017	0.1382 ± 0.0019	0.2387 ± 0.0014
gru-time	0.2574 ± 0.0048	0.3552 ± 0.0043	0.2518 ± 0.0038	0.3527 ± 0.0045
transformer-time	0.2667 ± 0.0030	0.3691 ± 0.0025	0.2657 ± 0.0051	0.3680 ± 0.0033
mamba-time	0.2663 ± 0.0008	0.3646 ± 0.0035	0.2757 ± 0.0125	0.3730 ± 0.0102
ETTh1				
gru-variate	0.4415 ± 0.0063	0.4781 ± 0.0068	0.4490 ± 0.0132	0.4918 ± 0.0115
transformer-variate	**0.4384 ± 0.0061**	**0.4667 ± 0.0051**	**0.4241 ± 0.0100**	**0.4601 ± 0.0067**
mamba-variate	0.4658 ± 0.0044	0.4955 ± 0.0037	0.4645 ± 0.0121	0.4960 ± 0.0076
gru-time	0.6741 ± 0.0455	0.5921 ± 0.0187	0.6896 ± 0.0347	0.5997 ± 0.0215
transformer-time	0.5727 ± 0.0271	0.5560 ± 0.0133	0.6343 ± 0.0520	0.5859 ± 0.0295
mamba-time	0.6499 ± 0.0151	0.5815 ± 0.0033	0.6865 ± 0.0230	0.5980 ± 0.0130

Figure 11: Summary of test-set forecasting metrics at horizon $T = 96$ across the evaluated datasets and configurations. The best results are shown in **bold**, and the second-best results are highlighted in purple.

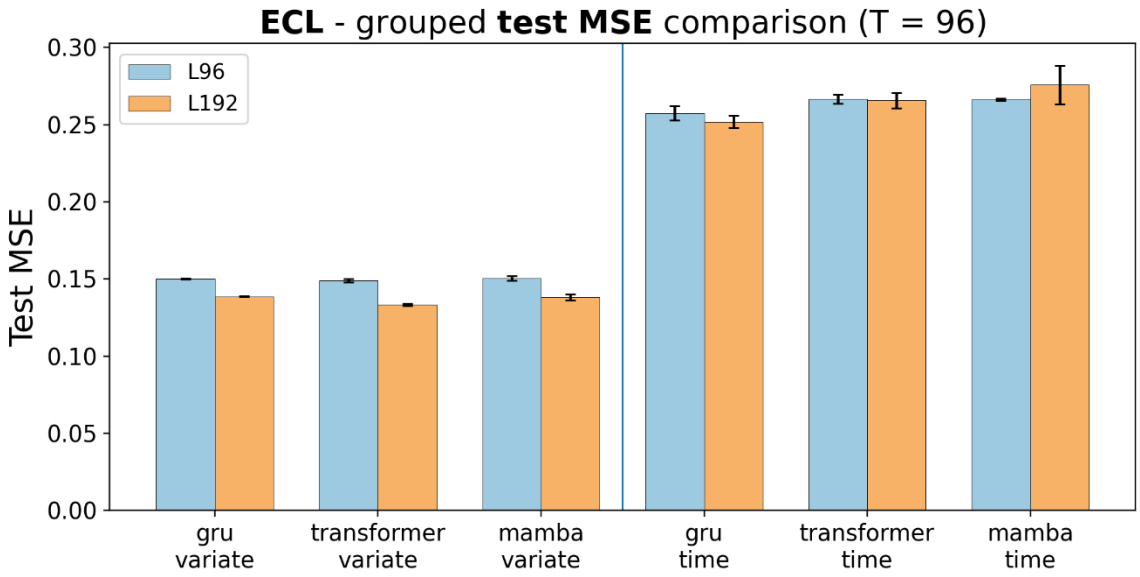


Figure 12: Grouped test MSE comparison on ECL for lookback windows $L \in \{96,192\}$ at forecasting horizon $T = 96$.

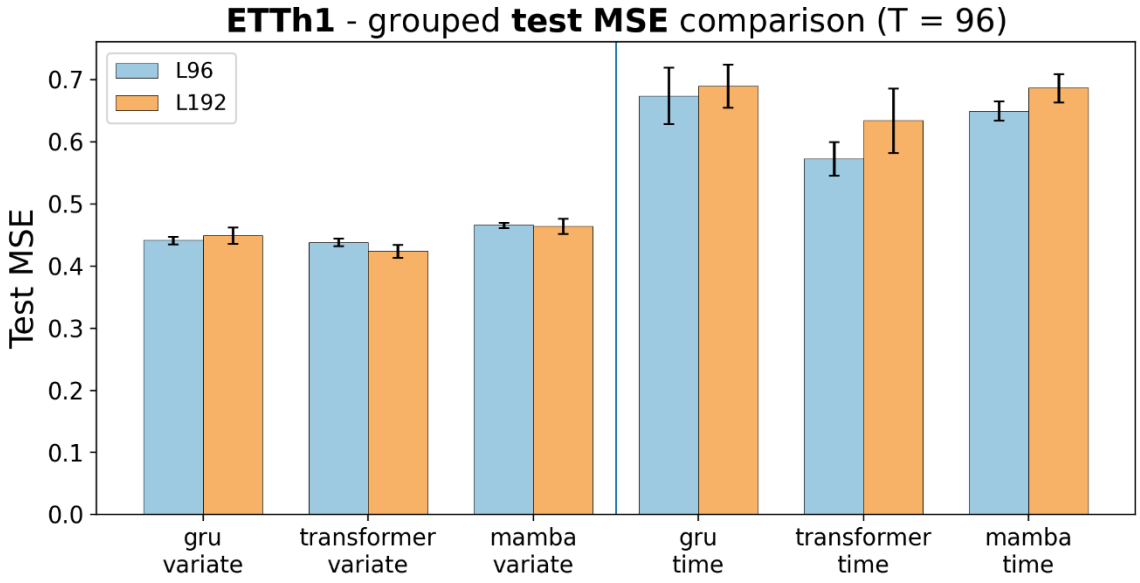


Figure 13: Grouped test MSE comparison on ETTh1 for lookback windows $L \in \{96,192\}$at forecasting horizon $T = 96$.

7.2.1 Variate-token vs Time-token Formulation

A consistent and dominant trend across all experiments is the superior performance of the variate-token formulation over the time-token counterpart. On the ECL dataset, variate-token models achieve MSE values around 0.14–0.15, whereas time-token models remain

above 0.25. On ETTh1, variate-token configurations achieve approximately 0.43–0.46, compared to about 0.57–0.69 for time-token models.

This behavior can be explained by the fundamentally different ways in which information is represented. In the time-token formulation, each token corresponds to one timestep and aggregates all variates into a single embedding. This produces an early compression of multivariate information, which can discard important cross-channel structure and introduce representational noise. In contrast, the variate-token formulation treats each channel as an independent token, preserving channel-wise temporal information and enabling the encoder to model inter-variable relationships more explicitly.

This distinction is particularly important in multivariate forecasting, where correlations between channels play a central role. The results strongly indicate that delaying aggregation until after encoding leads to more expressive representations and consistently better predictive performance.

7.2.2 Model-wise Comparison

Among variate-token configurations, the Transformer encoder consistently achieves the best performance, although the margins over GRU and Mamba remain relatively small. This advantage can be attributed to the attention mechanism, which directly models inter-variate dependencies through global interactions across tokens. Unlike GRU, which processes tokens sequentially, or Mamba, which relies on structured state-space dynamics, the Transformer can attend to all variates simultaneously and capture both local and global dependencies.

At the same time, the difference among encoder architectures is modest compared with the much larger gap introduced by tokenization. This suggests that, in this study, the choice of tokenization strategy has a stronger impact on forecasting accuracy than the choice of encoder family.

7.2.3 Impact of Dataset Characteristics

A significant difference in overall performance is observed between ECL and ETTh1, not only in absolute error values but also in how strongly tokenization affects the results.

On ECL, the gap between variate-token and time-token is particularly pronounced. The dataset contains 321 variates, so in the time-token setting all channels are compressed into a single embedding at each timestep. This aggregation introduces a form of noise amplification, because many heterogeneous signals must be represented within a fixed-dimensional token. Important cross-channel patterns may therefore be diluted or lost. In contrast, the variate-token formulation treats each channel as an independent token, producing $N = V = 321$ structured tokens and allowing the encoder to process each variate explicitly and learn inter-channel relationships more effectively.

On ETTh1, the same advantage remains present but is less pronounced. The dataset contains only seven variates, which reduces both the possible benefit of explicit cross-channel modeling and the severity of the compression introduced by the time-token representation. Moreover, ETTh1 exhibits stronger distribution shift between training and test splits, as discussed in Chapter 6. This makes the forecasting task inherently more difficult and reduces the effectiveness of all models, regardless of tokenization.

Overall, these observations show that the effectiveness of a tokenization strategy depends strongly on dataset structure. Datasets with many stable variates, such as ECL, benefit more from variate-token representations, whereas in low-dimensional and non-stationary settings such as ETTh1, the relative advantage is reduced, although still consistent.

7.2.4 Effect of Increasing *Lookback Window*

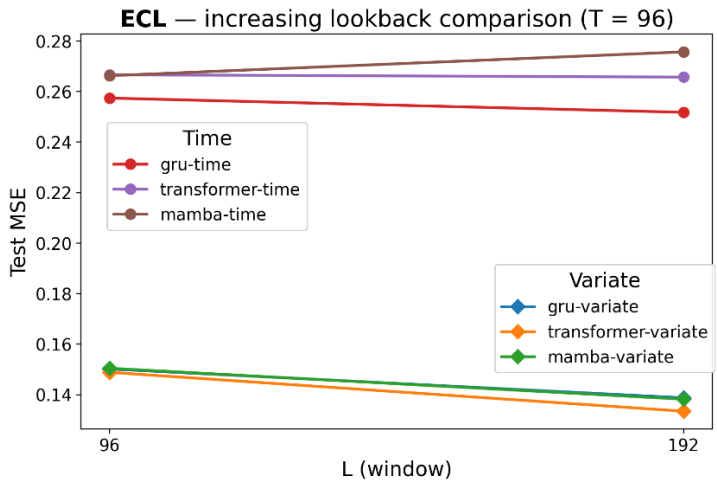


Figure 14: Effect of increasing the lookback window for ECL from $L = 96$to $L = 192$at forecasting horizon $T = 96$.

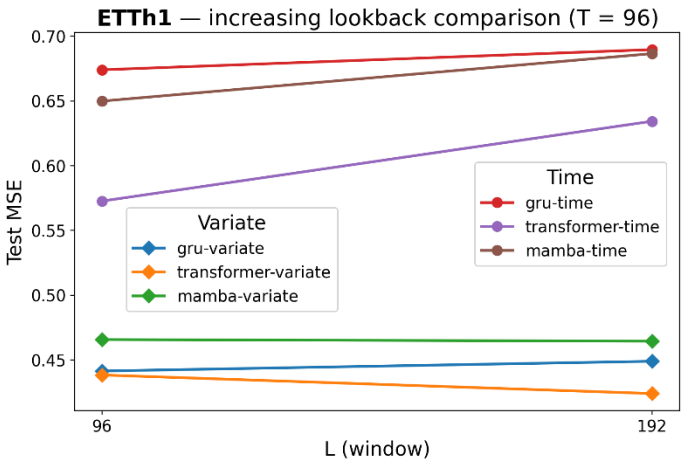


Figure 15: Effect of increasing the lookback window for ETTh1 from $L = 96$to $L = 192$at forecasting horizon $T = 96$.

Increasing the input window from $L = 96$ to $L = 192$ produces distinct effects depending on the tokenization strategy. For variate-token models, a consistent improvement in performance is observed across both datasets. This indicates that providing a longer temporal context per variate helps the encoder capture longer-range dependencies more effectively.

Time-token models, in contrast, show limited gains and in some cases even degraded performance, particularly on ETTh1. Because the time-token representation is more pointwise and more sensitive to noise, extending the sequence length does not necessarily provide more useful information and may instead amplify instability in the learned representation.

7.3 Computational Efficiency Analysis

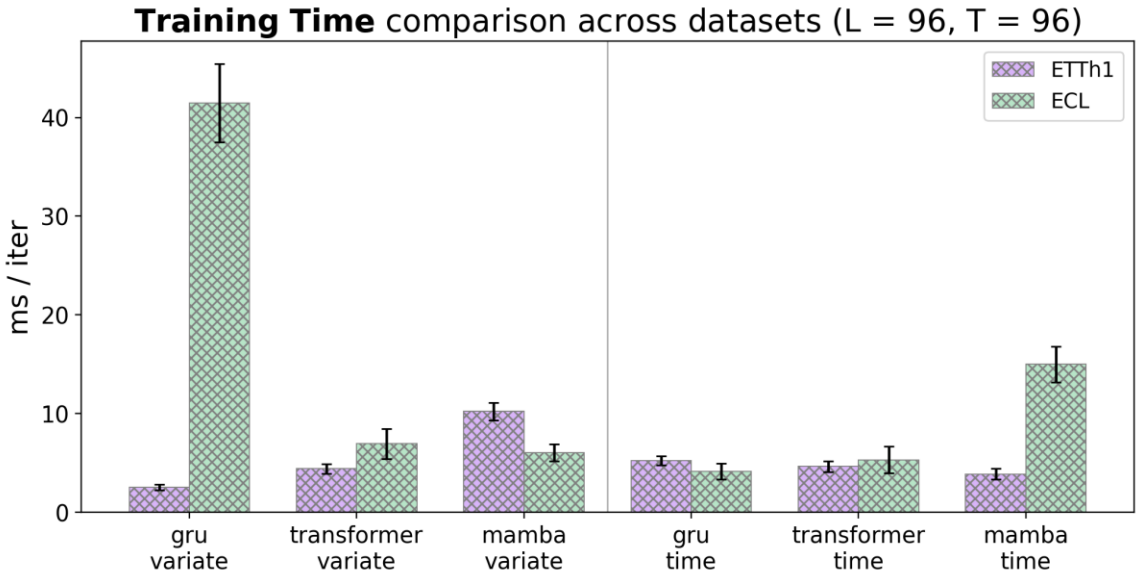


Figure 16: Cross-dataset comparison of training time per iteration at $L = 96$ and $T = 96$ across the evaluated architectures and tokenization strategies.

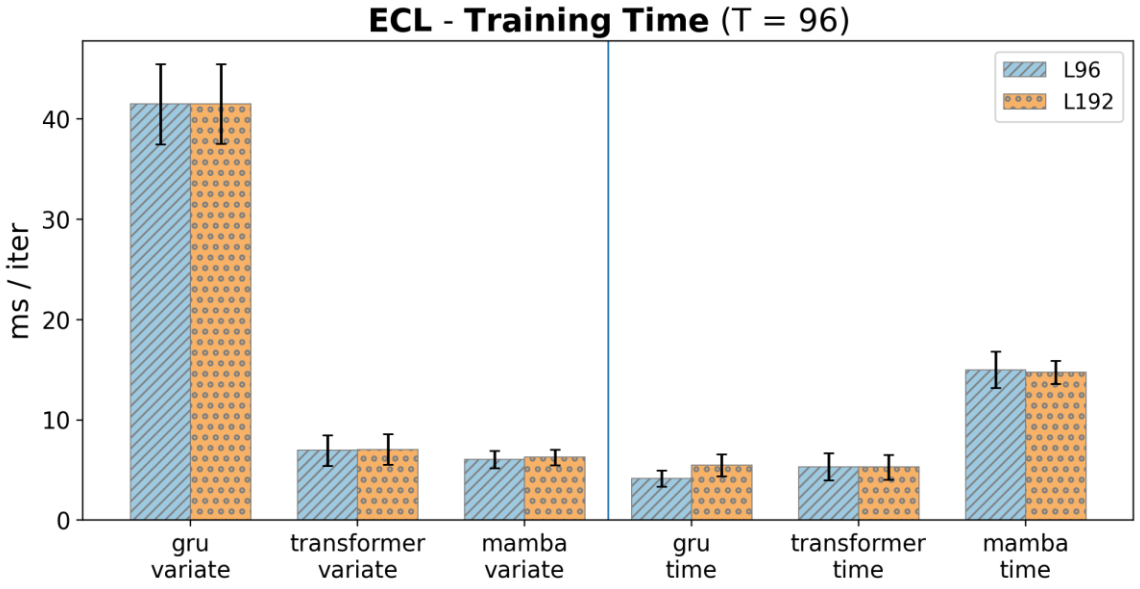


Figure 17: Training time per iteration on ECL for lookback windows $L \in \{96,192\}$ at forecasting horizon $T = 96$.

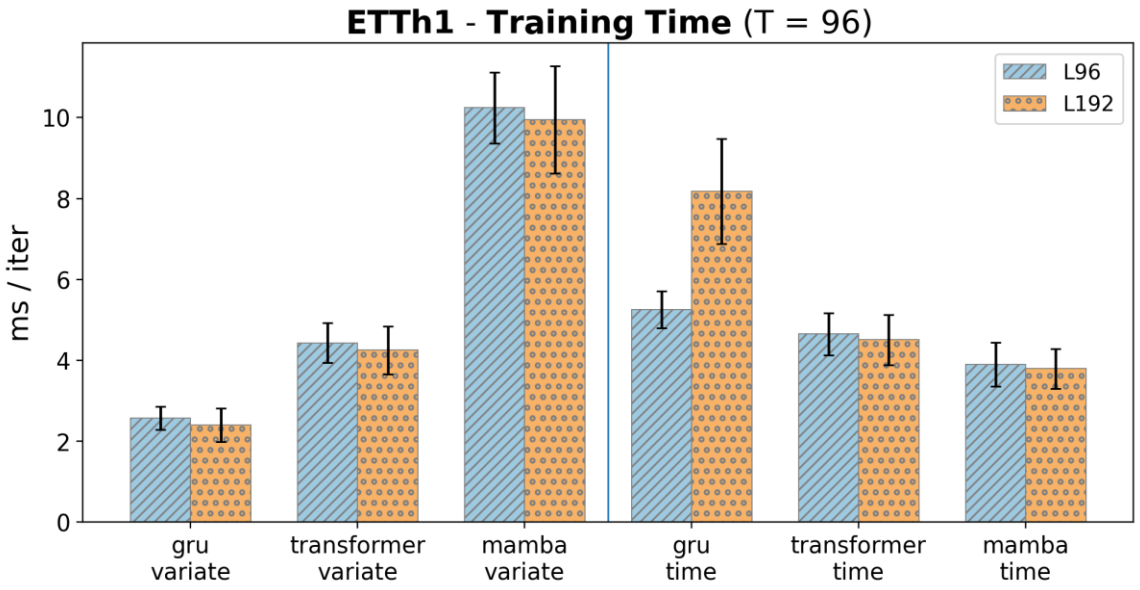


Figure 18: Training time per iteration on ETTh1 for lookback windows $L = 96$ and $L = 192$at forecasting horizon $T = 96$.

7.3.1 Architectural Cost and Scan Behavior

Computational efficiency is evaluated in terms of average milliseconds per iteration (ms/iter). Each iteration corresponds to a full forward and backward pass over a fixed batch of size $B = 32$, including gradient computation and parameter updates. This metric therefore reflects the true training cost per optimization step, independently of convergence speed or early stopping.

The observed efficiency trends are determined by the interaction among token count N, embedding dimension d_{model}, number of layers, architectural scan type (sequential versus parallel), and the presence and size of feed-forward networks (FFNs). In sequential models such as GRU, runtime is strongly affected by the number of scanned tokens. In parallel architectures such as Transformer and Mamba, sequence length can be handled more efficiently, but cost remains sensitive to per-token transformations.

In particular, the FFN must be considered explicitly. Its contribution scales as $O(N\, d_{model}\, d_{ff})$, and becomes especially relevant when d_{model} and d_{ff} are large or when N is small enough that scan-related costs no longer dominate. For this reason, the efficiency comparison cannot be explained only through attention versus scan

complexity: embedding dimension, FFN width, and tuned architectural choices such as bidirectionality and number of layers all play an important role in the final ms/iter values.

7.3.2 Efficiency Trends on ECL

The ECL dataset ($V = 321$) represents a high-dimensional scenario, where both token count and embedding size can become large depending on the tokenization strategy.

Among all configurations, GRU-variate exhibits the highest computational cost, around 40 ms/iter. This behavior is explained not only by the large number of tokens, $N = 321$, but also by the tuned architecture itself: more layers than the time-token counterpart, bidirectional processing, relatively large d_{model}, and full gate computations at every step. The result is a large number of sequential operations, each with high per-step cost. Even though the variate-token formulation avoids early information compression, it forces the GRU to scan a long sequence of tokens with a relatively heavy architecture. This clearly shows that a conceptually simple scan does not guarantee efficiency when the per-step computation is both sequential and expensive.

Across most ECL configurations, increasing the lookback from $L = 96$ to $L = 192$ produces only limited variations in ms/iter. In variate-token models, $N = V = 321$ remains constant and only the embedding projection changes. In time-token models, Ndoubles with the lookback length, but the impact is partly mitigated by GPU parallelism, particularly for Transformer and Mamba.

Mamba-time is slower than one might expect from its linear complexity alone. The main reason is the large embedding dimension induced by the projection from $V = 321$, together with the cost of the FFN and the non-negligible constant factors of state-space operations. Thus, even though the scan is linear in N, the per-token computation remains heavy in high-dimensional settings.

By contrast, Mamba-variate achieves relatively low ms/iter while maintaining competitive accuracy. This can be explained by its tuned hyperparameters: moderate number of layers, controlled d_{model}, and more efficient handling of the large token set through linear scanning. Compared with GRU-variate, it avoids sequential gate computations; compared with Transformer-variate, it preserves similar representational

flexibility without quadratic attention cost. This makes Mamba-variate a particularly balanced configuration in terms of efficiency, memory usage, and accuracy.

A broader structural insight also emerges from ECL. In high-dimensional multivariate settings, the richness of the data can reduce the need for model capacity, especially in the variate-token formulation. Because each token corresponds to the temporal history of one variate, the representation is cleaner and more structured than in the time-token case, where many channels are mixed at every timestep. As a consequence, tuned Transformer and Mamba variate-token models can rely on moderate d_{model}, fewer layers, and controlled FFN sizes while still achieving strong predictive performance.

GRU is an important exception to this trend. Even though it receives the same rich variate-token input, it still requires heavier configurations to compensate for its sequential scan and lack of a direct global interaction mechanism. As a result, it can remain accurate, but only at substantially higher computational cost. This highlights a key difference: data richness can reduce model complexity for parallel architectures, whereas sequential models such as GRU remain constrained by their scan mechanism and therefore often compensate with larger capacity.

7.3.3 Efficiency Trends on ETTh1

ETTh1 ($V = 7$) represents the opposite regime, where the number of variates is small and the computational behavior is no longer driven primarily by token count. In this setting, embedding dimension d_{model} and the FFN term become much more important.

In the variate-token configurations, $N = 7$ is extremely small. This places the models in an FFN-dominated regime, where the contribution $O(N\, d_{model}\, d_{ff})$ becomes central. Because $N$is so limited, the architectural advantage of linear or parallel scan is reduced, and the cost is increasingly determined by the size of the internal transformations, in particular d_{model} and d_{ff}. This explains why Mamba-variate is slower than GRU-variate despite its linear scan complexity, and why Transformer-variate remains faster than Mamba-variate even though both include FFN layers.

The difference between Transformer and Mamba can be attributed to more optimized parallel kernels for Transformer operations, especially matrix multiplications in attention

and FFN blocks, together with lower constant factors than state-space computations. Mamba-variate is also less effective in this regime because the scan operates on a very short sequence, $N = 7$, and cannot fully exploit its state-space dynamics. As a result, the model relies more heavily on the FFN and on embedding-space transformations, which reduces the relative advantage of the scan mechanism.

GRU-variate, by contrast, benefits from the opposite conditions: very small N, absence of FFN layers, and limited impact of sequential processing. For this reason, it remains relatively efficient in ETTh1 despite its sequential nature.

The GRU-time configuration shows a clear increase in ms/iter when moving from $L = 96$to $L = 192$. This is expected because in the time-token setting $N = L$, and GRU must process the sequence step by step. Here, the dependence on sequence length becomes directly visible in runtime.

Memory differences on ETTh1 are also much smaller than on ECL. Since the number of tokens is low and activation tensors remain limited in size, GPU memory usage varies only modestly across configurations. In this dataset, memory therefore becomes a less informative comparative metric than runtime.

7.3.4 Cross-Dataset Insights

Comparing ECL and ETTh1 makes it clear that dataset structure strongly shapes computational behavior. In high-dimensional settings such as ECL, token count can dominate, especially for sequential architectures such as GRU, while embedding size also strongly affects Mamba and Transformer. In low-dimensional settings such as ETTh1, token count becomes much less important, and the FFN contribution together with embedding dimension d_{model}often becomes the dominant source of cost.

This comparison also shows that large models can still be efficient when their computation is highly parallelizable, as in the Transformer case, while smaller models can still be slow if they depend on sequential processing, as in GRU-variate. Therefore, efficiency is not determined by raw size alone, but by the interaction among architecture, tokenization, scan mechanism, d_{model}, and FFN.

A related implication is that the relationship between dataset dimensionality and required model capacity is non-trivial. In high-dimensional settings such as ECL, the availability of many correlated variates allows models to extract meaningful structure with relatively moderate architecture size, particularly in the variate-token formulation. Conversely, in low-dimensional settings such as ETTh1, the limited number of channels and the presence of distribution shift force the model to rely more heavily on internal FFN transformations and embedding-space processing, increasing the relative importance of $d_{model}$and per-token cost.

7.3.5 Summary of Efficiency Behavior

- Sequential architectures such as GRU are highly sensitive to the number of tokens.
- Parallel architectures such as Transformer and Mamba mitigate sequence-length effects, but remain sensitive to embedding dimension and FFN size.
- When the number of tokens is small, the FFN term can become the dominant cost component.
- Hyperparameter tuning, including number of layers, bidirectionality, embedding dimension, and FFN width, substantially affects efficiency.
- Dataset dimensionality plays a central role in determining which computational factors dominate runtime.

7.4 Model Size, Memory and Trade-off Analysis

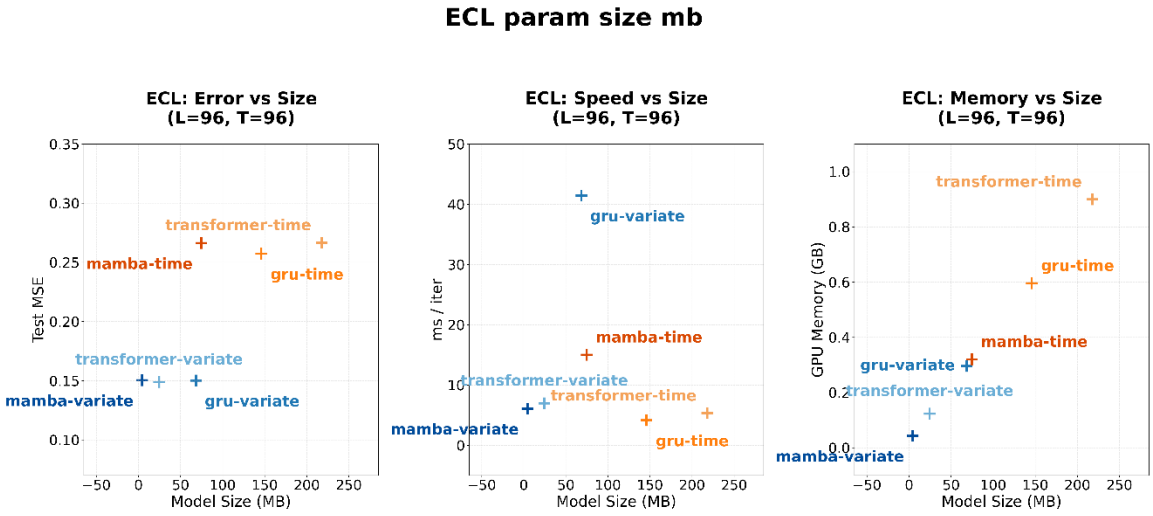


Figure 19: Accuracy, efficiency and memory given model size on ECL, for lookback windows $L = 96$ at forecasting horizon $T = 96$.

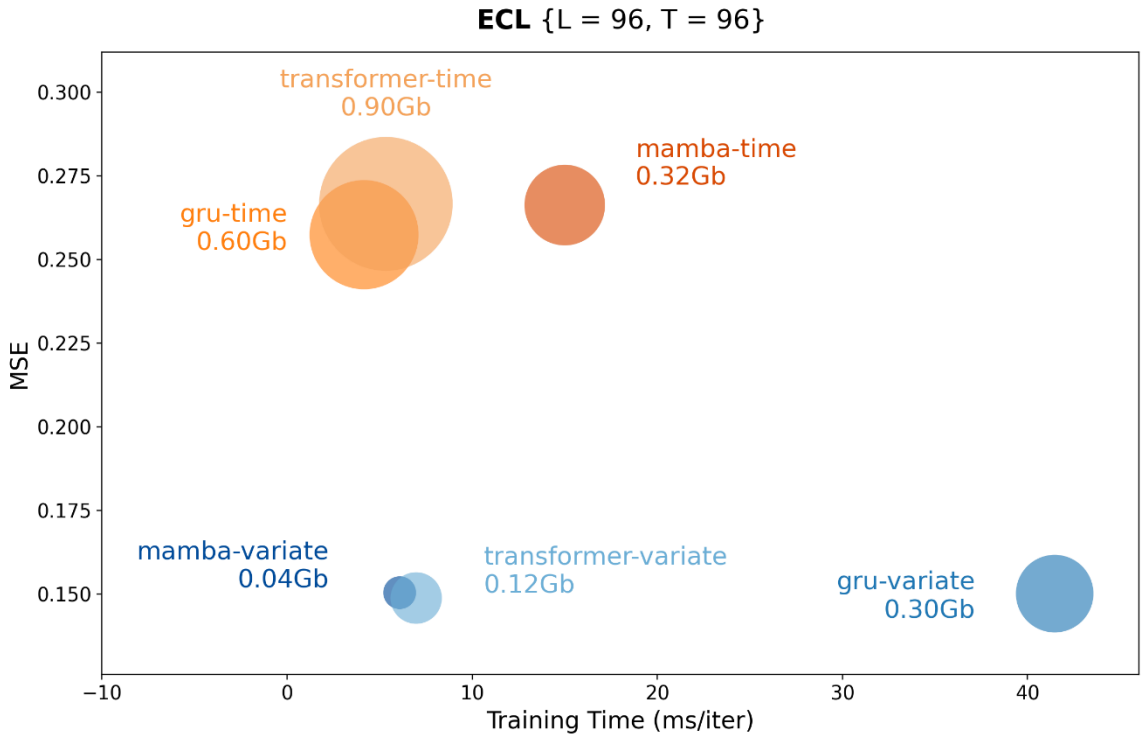


Figure 20: Accuracy-efficiency trade-off on ECL at $L = 96$ and $T = 96$, where bubble size represents GPU memory usage.

This section analyzes the relationship between model size, GPU memory consumption, computational efficiency, and predictive accuracy, with the goal of identifying the most relevant trade-offs across architectures and tokenization strategies.

7.4.1 Model Size vs Efficiency: No Direct Correlation

A first important observation is that model size alone is not a reliable predictor of computational efficiency. Transformer-time models are among the largest configurations, largely because they require large d_{model} and wide FFN layers, yet they still exhibit only moderate ms/iter thanks to parallel execution. Conversely, GRU-variate can be noticeably smaller in terms of parameter size, but remains the slowest configuration because it must process a large number of tokens sequentially.

This confirms that efficiency is governed more by the interaction between architecture and tokenization than by raw parameter count alone. In other words, large models can still be efficient when the computation is parallelizable, whereas smaller models can still be slow when runtime is dominated by sequential scans.

7.4.2 GPU Memory Usage and Tokenization Effects

A clearer relationship emerges when analyzing GPU memory consumption, especially when comparing time-token and variate-token formulations. At first glance, one might expect time-token models to use less memory, since they process fewer tokens than variate-token models and, in the Transformer case, attention scales quadratically with N. However, the experimental results consistently show the opposite: time-token models often require substantially more GPU memory.

This apparent contradiction is explained by the fact that memory usage is dominated less by token count itself and more by embedding dimension d_{model} and FFN width d_{ff}. During training, the largest memory contributions often come from storing intermediate activations for forward and backward passes. In particular, the FFN contributes a cost proportional to $O(N\, d_{model}\, d_{ff})$, which can dominate memory usage when d_{model} and d_{ff} are large.

A concrete example is provided by the ECL Transformer configurations. In the time-token case, the tuned model uses $N = 96$, $d_{model} = 1024$, and $d_{ff} = 4096$. In the variate-token case, the tuned model uses $N = 321$, $d_{model} = 512$, and $d_{ff} = 1024$. Although the time-token model processes fewer tokens, its embedding dimension is twice as large and its FFN width is four times larger. As a result, the activation tensors generated inside the encoder are much larger, and the overall memory footprint becomes higher despite the smaller N.

The reason time-token models require larger d_{model} and FFN layers is structural. In the time-token formulation, each token must aggregate all variates at a given timestamp, meaning that one vector has to encode information from many heterogeneous signals simultaneously. This increases representational complexity, introduces noise due to channel mixing, and pushes the tuned model toward larger embedding dimensions and stronger FFN expansions. In contrast, in the variate-token formulation, each token corresponds to a single channel and already contains a structured temporal signal. The model can therefore work with smaller $d_{model}$and more compact FFN layers.

Although time-token configurations process fewer tokens, they require significantly larger embedding dimensions to represent the compressed multivariate information contained in each token. This leads to larger FFN blocks and larger activation tensors, which dominate GPU memory consumption. In contrast, variate-token models distribute information across a larger number of smaller and more structured tokens. As a result, despite the higher token count and the quadratic attention term in Transformers, variate-token models can still use substantially less memory overall.

An intuitive way to interpret this result is that, from a memory perspective, it is often more efficient to handle many small vectors than a few very large ones. The same intuition also applies to GRU-based models: GRU-time uses larger hidden states to encode all variates at each timestep, whereas GRU-variate can rely on smaller hidden dimensions, making it comparatively more memory-efficient even though it processes more tokens.

This difference is especially pronounced on ECL, where the number of variates is large and the gap in d_{model} and d_{ff} between time-token and variate-token configurations becomes substantial. On ETTh1, the number of variates is small, embedding sizes are more similar across settings, and memory differences therefore become much less visible.

7.4.3 Trade-off Between Accuracy and Efficiency

The combined accuracy-efficiency plots show that variate-token models dominate the Pareto frontier. They achieve lower MSE, comparable or lower ms/iter, and reduced memory usage relative to time-token counterparts. In particular, Transformer-variate provides the best accuracy, while Mamba-variate offers one of the most convincing balances between accuracy, runtime, and resource efficiency.

This balance is especially visible in the ECL setting, where Mamba-variate combines competitive predictive performance with a markedly smaller memory footprint. In the available comparisons, it can be described as requiring about 3x less memory than the corresponding Transformer alternative and about 8x less memory than GRU-variate, while still remaining in a favorable region of the accuracy-efficiency trade-off space.

GRU-variate occupies a different region of this trade-off. It remains accurate, but only at the cost of very high ms/iter. This reinforces the earlier conclusion that sequential processing becomes a bottleneck when the number of tokens is large, even when the model itself is not the largest in terms of parameter count.

7.4.4 Dataset-Dependent Trade-offs

The relationship between model size, computational efficiency, and predictive accuracy also depends strongly on the dataset.

In ECL, the large number of variates strongly favors the variate-token formulation. This leads to excellent predictive performance while often reducing the need for very large d_{model}, deep stacks, or oversized FFN layers, especially for Transformer and Mamba. As discussed in Section 7.3, the richness of the multivariate input allows these architectures to exploit data structure more directly and therefore achieve a strong balance between size, efficiency, and accuracy.

In ETTh1, the situation is different. Although variate-token still improves accuracy, the advantage is smaller because the number of variates is low and distribution shift is stronger. In this regime, Mamba-variate is less effective because its scan operates over a very short sequence and cannot fully exploit the benefits of state-space accumulation. The model therefore depends more heavily on FFN transformations and embedding-space processing. More generally, when $N$is small, FFN cost becomes dominant and the relative benefit of sophisticated scan mechanisms is reduced.

This explains why Transformer-variate remains efficient and effective in ETTh1, whereas Mamba-variate cannot exploit its architectural advantages as strongly as in ECL. It also reinforces a broader point: the best trade-off depends not only on the model family, but on how well its computational strengths match the structure of the dataset.

7.4.5 Summary of Trade-offs

- Model size does not directly determine computational cost.
- Sequential architectures such as GRU can become inefficient even at moderate parameter sizes.
- Parallel architectures such as Transformer and Mamba scale better with token count, but remain sensitive to $d_{model}$and FFN size.
- Variate-token formulations provide better accuracy together with improved resource efficiency in most settings.
- Mamba-variate provides a particularly strong balance between performance and memory usage in high-dimensional settings.
- Dataset characteristics determine which architectural advantages can be fully exploited.

7.5 Qualitative Forecast Analysis

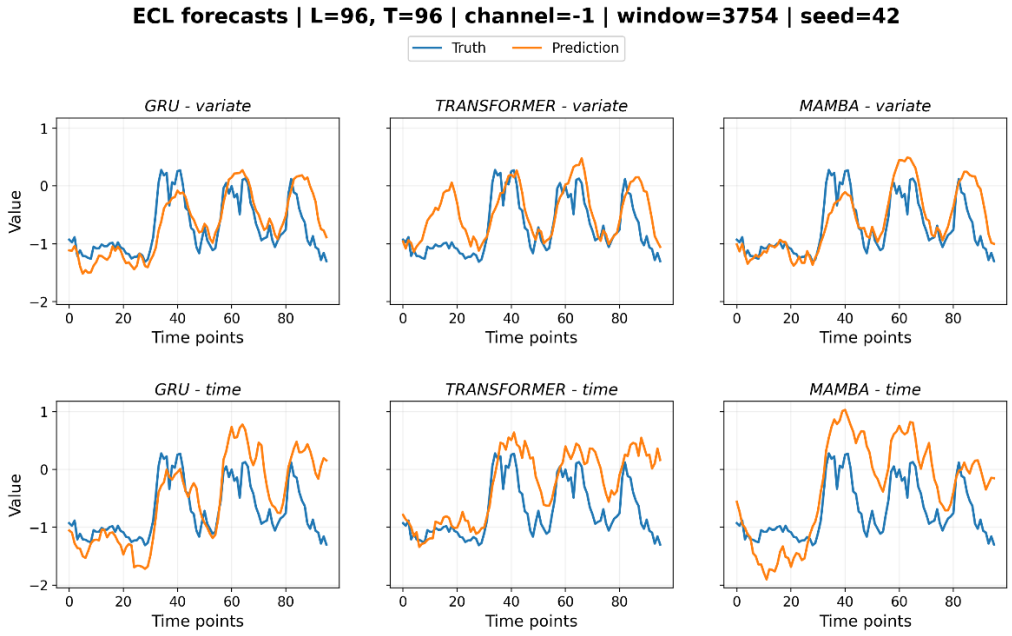


Figure 21: Representative forecast trajectories on ECL at $L = 96$ and $T = 96$, comparing predictions and ground-truth sequences across selected model configurations.

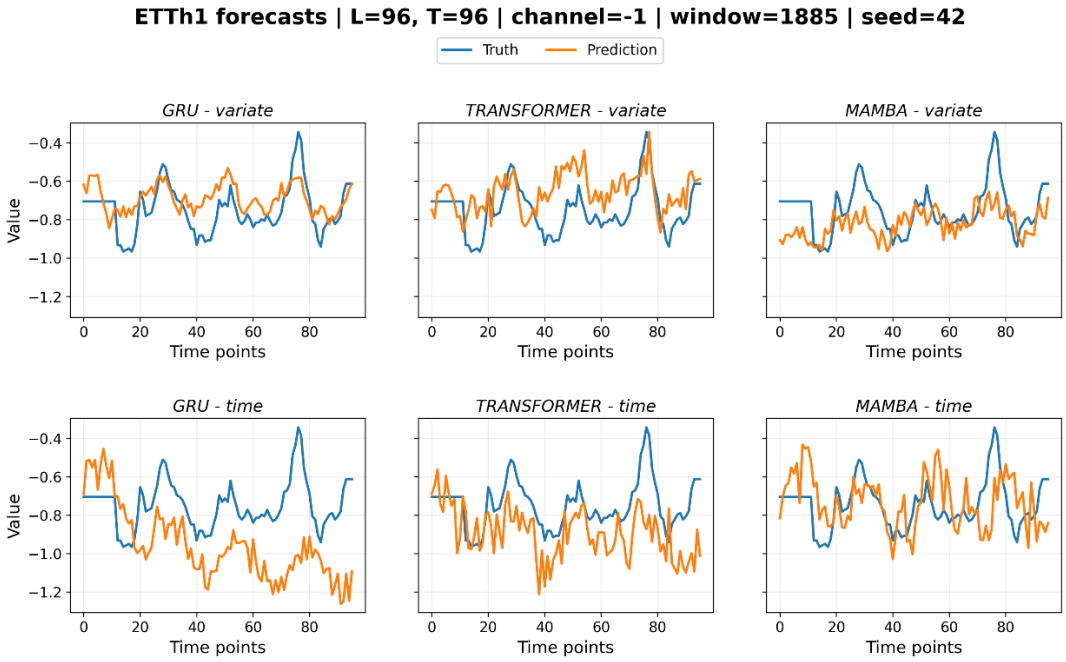


Figure 22: Representative forecast trajectories on ETTh1 at $L = 96$ and $T = 96$, comparing predictions and ground-truth sequences across selected model configurations.

While quantitative metrics such as MSE and MAE provide a global measure of performance, they do not fully capture the qualitative behavior of the models in reconstructing temporal dynamics. To complement the numerical analysis, this section examines representative forecast trajectories for both datasets.

7.5.1 Forecast Behavior on ECL

On ECL, all models capture the dominant periodic patterns, including daily and weekly seasonality. However, clear differences emerge in the reconstruction of amplitude variations and local fluctuations. Variate-token models produce predictions that are smoother, better aligned with the ground truth, and more accurate in reproducing peak magnitudes. Time-token models, in contrast, tend to underestimate peaks, exhibit slight phase shifts, and produce flatter or noisier trajectories.

This behavior is consistent with the structural limitations discussed earlier. In the time-token formulation, each token aggregates all channels at one timestamp, introducing noise and reducing the model's ability to preserve fine-grained channel-specific patterns. In the variate-token setting, each channel is processed independently before cross-channel interactions are modeled, which leads to more faithful reconstruction of temporal structure.

7.5.2 Forecast Behavior on ETTh1

ETTh1 is qualitatively more challenging because of stronger distribution shift and the smaller number of variates. Across all configurations, predictions are less accurate than on ECL and deviations from the ground truth are more pronounced. Nevertheless, the same overall trend remains visible: variate-token models better follow the global trend, show improved stability across the horizon, and produce more consistent outputs, whereas time-token models are more variable, more noise-sensitive, and often less able to capture sudden changes or local extrema.

This observation is particularly meaningful in ETTh1, where the combination of distribution shift and limited channel diversity amplifies the weaknesses of pointwise

representations. Since time-token embeddings are inherently more pointwise and more noise-prone, they become especially fragile in this challenging regime.

7.5.3 Impact of Tokenization on Temporal Reconstruction

The qualitative analysis reinforces one of the central findings of this study: tokenization strategy directly influences the model's ability to reconstruct temporal dynamics, beyond what is captured by aggregate error metrics. Time-token models compress multivariate information early, which leads to loss of structure and increased noise. Variate-token models preserve channel-wise temporal information longer, enabling more faithful reconstruction of both global trends and local variations. This distinction becomes even more important in long-horizon forecasting, where small stepwise errors accumulate over time.

7.5.4 Architecture-specific Observations

Although tokenization is the dominant factor, some architecture-specific behaviors can also be observed. Transformer-based models produce the most stable and consistent predictions, benefiting from global attention. Mamba-based models remain competitive, but may exhibit slight smoothing effects, particularly in short-sequence regimes such as ETTh1 variate-token, where scan length is limited. GRU-based models capture general trends effectively, but often respond more slowly to rapid changes, especially at longer horizons. These qualitative patterns are consistent with the underlying mechanisms: attention provides direct global context, Mamba relies on accumulated state, and GRU processes information sequentially.

7.6 Summary of Experimental Findings

This section consolidates the main insights derived from the experimental evaluation.

7.6.1 Tokenization as the Dominant Factor

Across all datasets, models, and configurations, the variate-token formulation consistently outperforms the time-token approach in terms of forecasting accuracy. This confirms that preserving channel-wise structure before encoding, and modeling inter-

variate relationships explicitly, is more effective than aggregating multivariate information at each timestep.

7.6.2 Dataset Structure Strongly Influences Performance

The two datasets exhibit fundamentally different behaviors. ECL combines a large number of variates with relatively stable structure, leading to strong performance across models and especially large gains for variate-token configurations. ETTh1 combines few variates with stronger distribution shift, leading to overall higher error and a reduced, but still consistent, advantage for variate-token models. This highlights the importance of considering data characteristics when selecting both architecture and representation.

7.6.3 Computational Trade-offs Depend on Architecture

Efficiency depends on the interaction among token count N, embedding size d_{model}, and scan mechanism. GRU is highly sensitive to token count because of sequential processing, and becomes inefficient in high-dimensional variate-token settings. Transformer benefits from parallelization and remains efficient even when model size is large. Mamba offers a compromise between linear scan complexity and expressive power, but remains sensitive to embedding dimension and sequence length.

7.6.4 FFN and Embedding Size Drive Memory Usage

GPU memory consumption is dominated primarily by embedding dimension $d_{model}$and by the FFN blocks inside the encoder, rather than by the number of tokens alone. As a consequence, time-token models, which often require larger $d_{model}$and larger FFN expansions to represent compressed multivariate information, consume more memory. Variate-token models, despite processing more tokens, can often use smaller embeddings and smaller FFN layers, and therefore achieve better memory efficiency overall.

7.6.5 Mamba-variate as a Balanced Configuration

In high-dimensional settings such as ECL, the Mamba-variate configuration emerges as a particularly effective compromise: it achieves competitive accuracy, significantly lower memory usage, and moderate computational cost. This makes it a strong candidate for real-world settings where both performance and resource constraints matter.

7.6.6 Final Insight

Overall, the results demonstrate that the interaction between tokenization strategy, model architecture, and dataset characteristics is the key determinant of forecasting performance and efficiency. Rather than identifying a single universally best model, the experiments show that optimal performance is obtained by aligning representation, architecture, and data structure.

8 Conclusions and Future Work

8.1 Summary of the Work

This thesis investigated the interaction between tokenization strategy, sequence modeling architecture, and dataset characteristics in multivariate time series forecasting. In particular, the analysis focused on two alternative input formulations—time-token and variate-token—combined with three encoder families: GRU, Transformer, and Mamba.

The experimental study was conducted on two benchmark datasets, ECL and ETTh1, selected for their markedly different properties. ECL represents a high-dimensional multivariate setting with a large number of channels, while ETTh1 is a lower-dimensional dataset characterized by a more evident distribution shift between training and test segments. Across these datasets, all configurations were evaluated under the same protocol in terms of forecasting accuracy, computational efficiency, GPU memory usage, and model size.

The goal of this thesis was not only to compare architectures, but also to understand how the representation of the input sequence influences both predictive performance and computational behavior. In this respect, the experimental analysis showed that tokenization is not a secondary implementation detail, but a central design choice that shapes the effectiveness and efficiency of the entire forecasting pipeline.

8.2 Main Findings

8.2.1 Tokenization as the dominant design factor

The most important result of this work is that tokenization strategy consistently has a stronger impact than the choice of encoder architecture. Across both datasets and across all tested model families, the variate-token formulation produced better results than the time-token formulation.

This trend was especially evident on ECL, where variate-token models achieved a substantial improvement in forecasting accuracy. The reason is closely related to the structure of the data: when each variate is treated as an independent token, the encoder can preserve channel-wise temporal information and explicitly model interactions across variables. In contrast, the time-token formulation compresses all channels into a single embedding at each timestep, which introduces early information mixing and can amplify noise.

The same tendency was also observed on ETTh1, although with smaller margins. This indicates that the advantage of variate-token is robust, but also that its magnitude depends on the dataset characteristics.

8.2.2 The role of dataset structure

A second key finding is that dataset structure strongly influences both the relative performance of the models and the relevance of specific architectural properties.

In ECL, the large number of variates makes the variate-token formulation particularly effective. Each token corresponds to a single channel with a structured temporal history, and this allows the model to exploit inter-variate relationships more directly. In such a setting, strong performance can be obtained even with relatively moderate architectural capacity, especially for Transformer and Mamba in the variate-token configuration.

In ETTh1, the situation is different. The number of channels is much smaller, reducing the representational advantage of treating each variate as a token. Moreover, ETTh1 exhibits stronger distribution shift and non-stationarity between training and test

segments, which makes the forecasting task more difficult for all models. As a result, the advantage of variate-token remains present, but becomes less pronounced.

These observations suggest that there is no universally optimal configuration independent of the data. Instead, the design of a forecasting model should be guided by the dimensionality, stability, and structural properties of the target dataset.

8.2.3 Architectural trade-offs

The experiments also highlighted clear differences among the three encoder families.

GRU remains a competitive baseline in some settings, but its sequential scan makes it highly sensitive to the number of tokens. This becomes a major limitation in high-dimensional variate-token regimes, where the sequence length is large. In these cases, GRU often requires heavier tuned configurations—more layers, bidirectionality, and larger hidden dimensions—to reach competitive performance, with a significant increase in computational cost.

Transformer provides the strongest overall accuracy, especially in the variate-token setting, thanks to its ability to model global interactions between tokens. Although self-attention scales quadratically with the number of tokens, the architecture benefits from highly optimized parallel implementations. In practice, this allows Transformer to remain computationally viable even when model size is relatively large.

Mamba emerges as a particularly interesting alternative. It combines linear scan behavior with expressive learned transformations and, in high-dimensional variate-token settings such as ECL, achieves a very favorable balance between predictive performance, runtime, and memory usage. However, its advantages are less evident in very short-sequence regimes, such as ETTh1 variate-token, where the scan length is too small to fully exploit the state-space mechanism.

8.2.4 Efficiency depends on more than sequence length

Another important conclusion is that computational efficiency cannot be inferred from token count alone. A shorter sequence does not necessarily imply a cheaper model.

Time-token models often process fewer tokens, but this apparent advantage is frequently offset by the need for larger embedding dimensions and larger feed-forward networks (FFNs). In practice, the computational and memory cost is strongly influenced by the interaction between the number of tokens N, the embedding size d_{model}, and the FFN expansion d_{ff}. This explains why some time-token models, despite operating on shorter sequences, still require more GPU memory and can remain computationally demanding.

Conversely, variate-token models may process more tokens, but each token is more structured and usually allows the encoder to operate with smaller embeddings and more compact FFN blocks. This leads to a more efficient distribution of information across the sequence and, in many cases, to better overall trade-offs between cost and accuracy.

8.2.5 Memory efficiency as a practical advantage of Mamba-variate

A particularly relevant practical outcome concerns the Mamba-variate configuration. While it does not systematically outperform Transformer in accuracy, it consistently provides a much lower GPU memory footprint, especially in the high-dimensional ECL setting.

This makes Mamba-variate an attractive option when memory becomes a limiting factor. The current experiments do not show that Mamba benefits more than Transformer from increasing the lookback window from $L = 96$to $L = 192$. However, they do show that Mamba remains the lightest configuration in terms of GPU memory. For this reason, it can be regarded as a promising candidate for future experiments with substantially longer input windows, where memory efficiency may become decisive.

8.3 Limitations

Despite the breadth of the empirical analysis, the present study has some limitations.

First, the evaluation was conducted on only two datasets. Although ECL and ETTh1 provide a meaningful contrast in terms of dimensionality and distribution shift, they cannot represent the full diversity of real-world multivariate forecasting problems.

Second, the study considered only three encoder families. While GRU, Transformer, and Mamba cover three important design paradigms—sequential recurrence, attention-based

parallelism, and state-space modeling—other architectural variants may reveal additional trade-offs.

Third, the comparison is based on a specific forecasting setting, with fixed horizon and selected input lengths. Extending the analysis to much longer forecasting horizons or substantially larger lookback windows may reveal new behaviors not captured in the current experiments.

Finally, while the empirical results provide clear evidence of the importance of tokenization, they do not yet offer a complete theoretical explanation of why the variate-token formulation is consistently superior. This remains an open question for future investigation.

8.4 Future Work

8.4.1 Scaling to longer lookback windows

One natural extension is to evaluate the models with substantially larger input windows. The current experiments already show that increasing the lookback can benefit variate-token models, especially in more structured settings. In this context, the memory efficiency of Mamba-variate makes it particularly appealing. Even though the present results do not demonstrate a stronger lookback gain than Transformer, the lower GPU usage of Mamba suggests that it may scale more easily when much longer contexts are considered.

8.4.2 Robustness to distribution shift

A second important direction concerns distribution shift, which is especially relevant for datasets such as ETTh1. Since the experiments indicate that ETTh1 is more challenging partly because of non-stationarity between training and test segments, future work could integrate methods explicitly designed to mitigate this problem.

A promising example is Reversible Instance Normalization (RevIN), which normalizes each input instance using its own statistics and restores them at the output through a symmetric denormalization step. Such a mechanism can reduce discrepancies in mean and variance across sequences while preserving the ability to reconstruct the original

scale of the series. Since RevIN is lightweight and model-agnostic, it would be particularly suitable for evaluating whether part of the performance gap observed on ETTh1 can be attributed to distribution shift rather than to architectural limitations alone. This type of correction could be especially interesting when combined with variate-token representations or with patch-based channel-independent settings, where preserving local structure while reducing non-stationary effects may be beneficial.

8.4.3 Patching and channel-independent strategies in low-dimensional settings

The results on ETTh1 suggest that the variate-token formulation is most effective when the dataset contains a sufficiently rich set of channels, so that meaningful inter-variate structure can be learned. When the number of variates is small, extracting strong cross-variate information becomes harder, and the advantages of the variate-token setting may be reduced.

For this reason, a promising direction is the use of patch-based representations, for example with half-overlapping temporal patches, possibly combined with channel-independent processing. In low-dimensional regimes, patching can increase the number of informative tokens extracted from each channel and provide a richer local temporal representation, while channel independence can reduce the need to force cross-variate interactions when those interactions are weak or limited by the small number of channels. This may provide a useful alternative in settings where pure variate-token modeling is less expressive.

An additional practical advantage of the variate-token perspective is its potential flexibility under partial channel availability. In principle, when some channels are missing at test time, a tokenization based on individual variates may degrade more gracefully than a representation that compresses all channels into a single token at each timestep. Similarly, in very high-dimensional settings, it may be possible to train on subsets of channels while still preserving a useful representation of the system dynamics. These aspects were not explored in the present thesis, but they suggest further

applications of token-based multivariate representations beyond the standard full-channel forecasting setting.

8.5 Final Remarks

Overall, this thesis shows that tokenization, architecture, and dataset structure must be considered jointly when designing multivariate forecasting systems. Among the tested configurations, variate-token emerges as the most effective representation, while Mamba-variate appears particularly promising when memory efficiency is important. These findings provide both empirical evidence and practical guidance for future work on more scalable and robust time series forecasting models.

9 Bibliography

Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, *5*(2), 157-166.

Bird, G., & Polivoda, M. E. (2021). Backpropagation through time for networks with long-term dependencies. arXiv preprint arXiv:2103.15589.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in neural information processing systems*, *27*.

Cho, K., Van Merriënboer, B., Gulçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014, October). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 1724-1734).

Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*.

Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

Ioffe, S., & Szegedy, C. (2015, June). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448-456). pmlr.
Ba, J., & Lei, J. R. (2016). Kiros and Geoffrey E. Hinton Layer. Normalization. preprintat https://arXiv. org/abs/160706450.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021, May). Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 35, No. 12, pp. 11106-11115).

Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023, June). Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 37, No. 9, pp. 11121-11128).

Nie, Y., Nguyen, N. H., Sinthong, P., & Kalagnanam, J. (2022). A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730*.

Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., & Long, M. (2023). itransformer: Inverted transformers are effective for time series forecasting. *arXiv preprint arXiv:2310.06625*.

Gu, A., Dao, T., Ermon, S., Rudra, A., & Ré, C. (2020). Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, *33*, 1474-1487.

Gu, A., Goel, K., & Ré, C. (2021). Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*.

Gu, A., Goel, K., Gupta, A., & Ré, C. (2022). On the parameterization and initialization of diagonal state space models. *Advances in neural information processing systems*, *35*, 35971-35983.

Gu, A., & Dao, T. (2024, May). Mamba: Linear-time sequence modeling with selective state spaces. In *First conference on language modeling*.